Open-FDD Documentation

Generated by scripts/build_docs_pdf.py

April 5, 2026

1 Home
2 Open-FDD
  2.1 What it does
  2.2 Quick start
    2.2.1 Optional: extra Caddy hardening (basic auth, full route list, TLS)
  2.3 Documentation
  2.4 Stack
  2.5 Behind the firewall; cloud export is vendor-led
3 System Overview
4 System Overview
  4.1 Architecture
  4.2 Services
  4.3 Campus-based architecture
  4.4 Data flow
  4.5 Ways to deploy
  4.6 Key concepts
5 Getting Started
6 Getting Started
  6.1 Do this to bootstrap
  6.2 External Agentic AI (OpenAI-compatible)
  6.3 Prerequisites
  6.4 What the bootstrap script does
  6.5 After bootstrap
  6.6 Next steps
7 Using the React dashboard
8 Using the React dashboard
  8.1 Navigation
  8.2 Common workflows
  8.3 Data model Brick: danger zone
9 Modular Architecture

10 Modular Architecture
10.1 Module contracts
10.2 Bootstrap mode matrix
10.3 Feature coverage by mode
10.4 Migration stance
10.5 External IoT / “we already have collection + modeling”
11 BACnet Overview
12 BACnet Integration
12.1 Components
12.2 Ports
12.2.1 Multiple sites / gateways (one Open-FDD, several BACnet bridges)
12.3 Discovery and getting points into the data model
12.4 Data acquisition
12.5 Configuration
13 BACnet graph context
14 BACnet graph context
14.1 Why this context matters
14.2 What must be discoverable from the graph
14.3 Live system interpretation
14.3.1 1. What is connected?
14.3.2 2. What kind of HVAC system is this?
14.3.3 3. What analysis should apply?
14.4 SPARQL assets in the repo
14.5 BACnet telemetry baseline query
14.6 Backend SPARQL and auth
14.7 BACnet scraper / DIY gateway context
14.8 Engineering direction
14.9 Where this context is saved
15 BACnet, RDF, and BRICK
16 BACnet, RDF, and BRICK
16.1 Roles
16.2 Flow
16.3 Try it (Open-FDD Swagger)
16.4 Summary
17 BACnet-to-fault verification
18 BACnet-to-fault verification
18.1 Goal
18.2 Required evidence chain
18.3 Overnight expectation
18.4 Important interpretation rule for the current fake bench
18.5 Minimum overnight report contents
18.6 Current practical limitation
19 DIY BACnet gateway RPC contract

20 DIY BACnet gateway RPC contract
20.1 Correct shape for `client_read_property`
20.2 Example result
20.3 Why this matters for overnight verification
21 BACnet
22 BACnet
22.1 Setup (do this before the platform scrapes data)
22.2 Verification and lab (OpenClaw bench)
23 Overview
24 Data model flow
24.1 Flow
24.2 TTL structure
24.3 Data modeling process (discover → tag → import → validate)
24.4 Data-model API
24.5 Validation
25 Sites
26 Sites
26.1 Structure
26.2 Scoping
26.3 API
27 Equipment
28 Equipment
28.1 Structure
28.2 Hierarchy
28.3 API
29 AI-assisted data modeling
30 AI-assisted Brick tagging
30.1 Workflow (export → tag → import)
30.2 API contract (import)
30.3 LLM prompt and agent guidelines
30.4 Rules for this project (what to send the LLM)
30.5 See also
30.6 External agent integration (Open-Claw or any OpenAI-compatible LLM)
30.6.1 External retry loop (prompt chaining)
30.7 Automating tagging (external agents)
30.8 Troubleshooting import failures (fixing your tagged JSON)
30.8.1 `site_id` must be a UUID
30.9 Possible extension: AI assist on Data Model Testing
31 Points
32 Points
32.1 Structure
32.2 Time-series data

32.3 Layers and mapping
32.3.1 BACnet addressing
32.4 API
32.5 Queries
33 LLM workflow (export + rules + validate → import)
34 LLM workflow: one prompt, export JSON, rules, and validated import
34.1 What you upload to the LLM
34.2 Copy/paste prompt template (recommended)
34.3 Where the rules live
34.4 Engineering metadata (fan HP, pumps, coils, design CFM, …)
34.5 Feeds / fed_by (HVAC topology)
34.6 Validate before import (so backend CRUD accepts it)
34.7 Mechanical engineer flow (short)
34.8 See also
35 SPARQL cookbook
36 SPARQL cookbook
36.1 Data Model Protocols vs Data Model Testing
36.2 How to run SPARQL
36.3 Prefixes and graph contents
36.4 Recipe 1: Platform config
36.5 Recipe 2: Sites and counts
36.6 Recipe 3: Equipment and points by site
36.7 Recipe 3b: External references (Brick v1.4)
36.8 Recipe 4: BACnet devices and objects
36.9 Recipe 5: FDD — points and rule mapping
36.10 Recipe 6: FDD — time-series references for rules
36.11 Recipe 7: Integrity and completeness
36.12 Predefined buttons (Data Model Testing)
36.13 SPARQL smoke suite (repo files)
36.14 Related docs
37 External representations (Brick v1.4)
38 External representations (Brick v1.4)
38.1 When ref triples appear in the TTL
38.2 What Open-FDD emits
38.3 Example
38.4 SPARQL discovery pattern
39 Data modeling
40 Data modeling
41 Overview
42 Fault rules
42.1 Where rules live: config path and how to manage them
42.2 Hot reload (edit → run → view)
42.3 Rule types

42.4 YAML structure
42.5 Input resolution (100% Brick)
42.6 Running rules
42.7 Engineering metadata (223P-style) and energy-style analytics
42.8 Cookbook
43 Expression Rule Cookbook
44 Expression Rule Cookbook
44.1 100% Brick-model driven (no column in rules)
44.2 How to define expressions
44.3 AHU rules (reference-style)
44.3.1 Rule A — Duct static below setpoint at full fan speed
44.3.2 Rule B — Blended air temp below expected band
44.3.3 Rule C — Blended air temp above expected band
44.3.4 Rule D — Discharge air cold when heating commanded
44.3.5 Rule E — SAT too low with full heating
44.3.6 Rule F — SAT/MAT mismatch in economizer mode
44.3.7 Rule G — Ambient too warm for free cooling
44.3.8 Rule H — Ambient vs blended mismatch (econ + mech cooling)
44.3.9 Rule I — Ambient vs blended mismatch (econ-only)
44.3.10 Rule J — Discharge above blended in cooling
44.3.11 Rule K — Discharge above setpoint in full cooling
44.3.12 Rule L — Cooling coil delta-T when inactive
44.3.13 Rule M — Heating coil delta-T when inactive
44.4 Central plant
44.4.1 Differential pressure at max pump speed
44.4.2 Plant flow high at max pump
44.4.3 Plant supply temp outside deadband
44.4.4 Chiller runtime over daily limit
44.5 Heat pumps
44.5.1 Discharge cold when heating
44.6 VAV zones
44.6.1 Excessive heating during warm weather
44.6.2 Damper or valve at full open
44.6.3 Zone and IAQ bounds
44.7 Opportunistic rules (economizer & ventilation)
44.7.1 Economizing when outdoor conditions are unfavorable
44.7.2 Mechanical cooling when econ could suffice
44.7.3 Low ventilation (estimated OA fraction)
44.7.4 Preheat over-conditioning
44.7.5 Blended air damper deviation
44.8 Weather station
44.8.1 Unrealistic temperature spike
44.8.2 RH bounds

44.8.3 Wind gust vs sustained
44.9 Sensor validation (bounds & flatline)
44.10 Follow-up (RFCs): multi-ontology selectors & cookbook matrix
45 Test bench rule catalog
46 Test bench rule catalog
46.1 Reference YAML files
46.2 Related docs
47 YAML rules → Pandas (under the hood)
48 YAML rules → Pandas DataFrames (under the hood)
48.1 1. YAML never becomes “one giant DataFrame of rules”
48.2 2. From Postgres rows to a site/equipment DataFrame
48.3 3. What `RuleRunner.run` does to that DataFrame
48.4 4. How a YAML rule dict becomes pandas expressions
48.5 5. Hot reload: YAML vs DataFrame lifetime
48.6 6. Mental model checklist
49 Fault rules for HVAC
50 Fault rules for HVAC
51 Cloud export example
52 Cloud export example
52.1 What the example does
52.2 Run the example
52.3 Use it as your starting point
53 Context and recordkeeping
54 Context and recordkeeping
54.1 Rule
54.2 Good context to commit
54.3 Context anti-patterns
55 Operational states
56 Operational states
56.1 States
56.2 Why this matters
57 Big picture gaps and product suggestions
58 Big picture gaps and product suggestions
58.1 Current themes
58.1.1 1. AI-assisted data modeling needs a stronger review-before-import flow
58.1.2 2. Graph reset behavior is easy to misunderstand
58.1.3 3. Rule-aware minimal polling is a high-impact lever
58.1.4 4. The gap between discovery and FDD-ready modeling is still wide
58.1.5 5. Overnight testing still needs stronger generated evidence
58.1.6 6. Container log viewing should become part of the testing story

58.1.7 7. Open-FDD needs a stronger live-HVAC operations mental model
58.2 Security / hardening context to remember
58.3 Why this doc exists
59 Concepts
60 Concepts
61 MQTT integration (optional)
62 MQTT integration (optional) — Open-FDD + diy-bacnet-server
62.1 What the stack can do today
62.2 Open-FDD product scope
62.3 Related docs
63 Open-Claw Integration
64 Open-Claw Integration (external worker)
64.1 1) Model context endpoint (`GET /model-context/docs`)
64.2 1b) MCP-style discovery (`GET /mcp/manifest`)
64.3 1c) Optional MCP RAG sidecar (`--with-mcp-rag`)
64.4 1d) Mode-aware orchestration with bootstrap
64.5 1e) OpenClaw on a different machine than Open-FDD (split setup)
64.6 2) Automated Brick tagging loop
64.7 3) “Tune faults” worker loop (rule YAML)
64.8 4) Example “worker” prompt for Open-Claw
64.9 5) Recommended “context window” behavior
65 How-to Guides
66 How-to Guides
67 Data model engineering (Brick + 223P MVP)
68 Data Model Engineering (MVP)
68.1 What this adds
68.2 223P and what Open-FDD implements
68.3 How this connects to FDD, PostgreSQL, and the knowledge graph
68.4 Energy penalty and impact calculations (scope)
68.5 Backward compatibility
68.6 JSON pattern
68.7 RDF pattern
68.8 Query examples
68.9 Related docs
69 PyPI releases (open-fdd)
70 PyPI releases (`open-fdd`)
70.1 `openfdd-engine` (optional second PyPI name)
70.2 Release baseline (PyPI)
70.3 1) Before a release of `open-fdd`
70.3.1 Transition checklist (legacy 0.1.x → 2.x on PyPI)
70.3.2 PyPI upload auth (`open-fdd` only)

70.3.3 Fallback: API token
70.3.4 If CI shows `invalid-publisher` or `403`
70.4 2) Publish `open-fdd` (commands)
70.4.1 Local dry run
70.4.2 GitHub Actions
70.5 3) Publish `openfdd-engine` (second PyPI project)
70.6 Scope policy
71 Engine-only deployment and external IoT pipelines
72 Engine-only deployment and external IoT pipelines
72.1 What `--mode engine` starts (Docker)
72.2 Library path — same YAML, any DataFrame
72.2.1 Bring your own `column_map` or resolver
72.2.2 Manifest file + composite priority (workshop / gap-fill)
72.3 Standalone playground (optional)
72.4 Summary
73 The optional openfdd-engine package
74 The optional `openfdd-engine` package (`openfdd_engine`)
74.1 Re-exports only — no duplicated types
74.2 Mental model (three layers)
74.3 When to use `pip install open-fdd` vs `openfdd-engine`
74.4 How the full stack uses the engine (so IoT-only users know what they are skipping)
74.5 Library-only / external data (no Docker engine mode)
74.6 Summary
74.7 See also
75 Grafana SQL cookbook
76 Grafana SQL cookbook —
76.1 BACnet
76.1.1 1. Site Variable
76.1.2 2. Device Variable
76.1.3 3. Point Variable
76.1.4 BACnet only data
76.2 BACnet Plus Fault Data
76.3 Host stats (stack-host-stats)
76.3.1 Dashboard variable (Host)
76.3.2 Host — Memory (used / available / total)
76.3.3 Host — Load (1 / 5 / 15 min)
76.3.4 Host — Swap (used / total)
76.3.5 Host — Disk space (free / used / total)
76.3.6 Host — Disk available (gauge, latest)
76.3.7 Containers — CPU %
76.3.8 Containers — Memory %
76.3.9 Containers — Table (latest)

76.4 Weather
76.4.1 Temp RH Dewpoint
76.4.2 Wind Speed Gusts Direction
76.4.3 Wind Speed Gust Direction
76.4.4 Solar Radiation
76.4.5 Cloud Cover
76.5 Faults
76.5.1 Fault Definitions
76.5.2 Fault Definition Count
77 Grafana dashboards
78 Grafana dashboards (optional)
78.1 React frontend parity
79 Open-FDD integrity sweep
80 Open-FDD integrity sweep
80.1 Cadence
80.2 Source-of-truth order
80.3 Sweep checklist
80.4 Failure classes
80.5 Mode detection
81 Overnight review
82 Overnight review
82.1 Window
82.2 Goals
82.3 Expected outputs
82.4 Alerting discipline
83 OpenClaw context bootstrap
84 OpenClaw context bootstrap
84.1 Store in Git (durable)
84.2 Keep local/private (do not commit)
84.3 Portability principle
84.4 Minimal bootstrap read list for fresh clones
85 Operator framework
86 Operator framework
86.1 Intent
86.2 Usage
87 MCP RAG service
88 MCP RAG service
88.1 Canonical vs derived context
88.2 Bootstrap
88.3 Service endpoints
89 Mode-aware runbooks
90 Mode-aware runbooks
90.1 Engine-only
90.2 Model-only

90.3 Collector + Model
90.4 Full autonomous stack (OpenClaw + MCP)
91 Testing plan
92 Testing plan
92.1 Near-term priorities
92.1.1 0. Continuous PR and CI review
92.1.2 Daytime short-run counterpart
92.1.3 1. Keep authenticated backend graph checks healthy
92.1.4 2. Promote BACnet addressing to a first-class validation target
92.1.5 3. Prove fault calculation from end to end
92.1.6 4. Preserve reusable context for future clones
92.1.7 5. Add nightly docs and link review
92.2 Future role in live HVAC systems
93 Continuous context backup loop
94 Continuous context backup loop
94.1 What gets backed up
94.2 What does not get backed up
94.3 Trigger for an update
94.4 Update sequence
94.5 Why this matters
95 Usage budget and throttling
96 Usage budget and throttling
96.1 Goal
96.2 Practical throttle ladder
96.2.1 Level 0 — Normal
96.2.2 Level 1 — Moderate conservation
96.2.3 Level 2 — High conservation
96.2.4 Level 3 — Survival mode
96.3 Priority order under throttling
96.4 Applying this to the Open-FDD workflows
96.4.1 Every-20-minute integrity sweep
96.4.2 Daytime smoke
96.4.3 Overnight review
96.5 What can be inferred from usage tools
96.6 Important limitation
97 Operations
98 Operations
99 Configuration
100 Configuration
100.1 Platform config (RDF + CRUD)
100.2 Platform keys (RDF / GET-PUT /config)
100.3 Model context endpoint (external agents)
100.4 Services that read config from the API (BACnet scraper)

100.5 BACnet: single gateway, remote gateways, central aggregator
100.6 Edge / resource limits
100.7 Environment
101 Contributing
102 Contributing to Open-FDD
102.1 Table of Contents
102.2 Code of Conduct
102.3 I Have a Question
102.4 I Want To Contribute
102.4.1 Legal notice
102.5 Reporting Bugs
102.5.1 Before submitting a bug report
102.5.2 How to submit a good bug report
102.6 Suggesting Enhancements
102.6.1 Before submitting
102.6.2 How to submit a good enhancement suggestion
102.7 Contributing Rules (Expression Rule Cookbook)
102.8 Your First Code Contribution
102.9 Improving the Documentation
102.10 Styleguides
102.11 Git workflow (branch → PR → sync)
102.12 Commit Messages
102.13 Join the Project Team
102.14 Attribution
103 API Reference
104 API Reference
104.1 High-level overview
104.2 MCP manifest (`GET /mcp/manifest`)
104.3 Model context docs endpoint
105 AI PR review playbook
106 AI PR review playbook
106.1 Priority order
106.2 Review output contract
107 Appendix
108 Appendix
109 Cloning and porting
110 Cloning and porting
110.1 Core portability idea
110.2 What should transfer cleanly
110.3 What usually changes per environment
110.4 What to do when deploying to another site
110.5 Recommended first-pass deployment flow for a new building
110.6 Same-bench OpenClaw clone checklist
110.7 Portability goal

110.8 Engineering principle
111 OpenClaw subscription setup (Codex OAuth)
112 OpenClaw setup: ChatGPT subscription / Codex OAuth (not API key)
112.1 Why this matters
112.2 The key distinction
112.3 First commands to run
112.4 If OpenClaw is still trying to use the API-key provider
112.5 What worked
112.5.1 1. Re-auth the OAuth provider
112.5.2 2. Make sure the default model points to `openai-codex/...`
112.5.3 3. Clean `openclaw.json`
112.5.4 4. Remove stale API-key provider entries
112.5.5 5. Check the agent auth file too
112.5.6 6. Restart the gateway
112.5.7 7. Verify again
112.6 One more gotcha: old sessions
112.7 Practical summary
112.8 Operational context to save for future setups
112.9 Why this doc exists
113 Monitor the fake fault schedule
114 Monitor the fake fault schedule
114.1 Why this matters
114.2 The shared UTC fault schedule
114.2.1 Scheduled points
114.3 Recommended monitor
114.3.1 Example output
114.4 How to reason about anomalies
114.4.1 Case 1: 180°F during UTC minutes 50-54
114.4.2 Case 2: 180°F outside UTC minutes 50-54
114.4.3 Case 3: flatline window but value keeps changing
114.5 How this connects to Open-FDD verification
114.6 Clone note
115 Frontend Evaluation
116 React frontend evaluation
116.1 Quality and stack
116.2 Feature coverage vs OpenAPI CRUD
116.3 Grafana cookbook parity
116.4 WebSockets: keep or remove?
116.5 Legacy static UI and BACnet tree
116.6 Caddy and security
116.7 Summary
117 Plan Rules Crud And Hot Reload

118 Plan: Default Rules, Fix Phantom Definitions, Rules CRUD + Frontend (Hot Reload)
118.1 Goals
118.2 1. Fix phantom definitions (backend)
118.3 2. Default rules (no code change)
118.4 3. Backend: Rules CRUD (upload / delete)
118.5 4. Frontend: Upload, Download, Delete
118.6 5. Hot reload (no change)
118.7 6. Implementation order
118.8 7. Security / validation
118.9 8. Summary
119 Security
120 Security and Caddy Bootstrap
120.1 Reverse proxy: current file vs future hardening
120.2 Architecture: frontend, API, and Caddy
120.3 Frontend and API authentication
120.4 Stack hardening (database, Caddy, secrets)
120.5 Quick bootstrap with Caddy
120.6 Caddyfile for protecting the entire API
120.7 Default password and how to change it
120.8 Services: encryption and exposure
120.9 Hardening best practices
120.10 Throttling and rate limiting
120.10.1 1. No API rate limiting by default
120.10.2 2. Outbound: OT/building network is paced
120.10.3 3. Inbound: rate limiting at the reverse proxy (e.g. Caddy)
120.11 TLS (optional): self-signed certificates
120.12 Troubleshooting
120.13 Security scanning (Trivy)
120.14 Summary

Generated April 5, 2026

---

1 Home

2 Open-FDD

Open-FDD is an open-source knowledge graph fault-detection platform for HVAC systems that helps facilities optimize their energy usage and cost-savings. Because it runs on-prem, facilities never have to worry

about a vendor hiking prices, going dark, or walking away with their data. The platform is an AFDD stack designed to run inside the building, behind the firewall, under the owner's control. It transforms operational data into actionable, cost-saving insights and provides a secure integration layer that any cloud platform can use without vendor lock-in. U.S. Department of Energy research reports median energy savings of roughly 8–9% from FDD programs—meaningful annual savings depending on facility size and energy spend.

The building is modeled in a **unified graph**: Brick (sites, equipment, points), BACnet discovery RDF, platform config, and—as the project evolves—other ontologies such as ASHRAE 223P, in one semantic model queried via SPARQL and serialized to `config/data_model.ttl`.

2.1 What it does

Open-FDD is an **edge analytics and rules engine** for building automation. It:

- **Ingests** BACnet points via diy-bacnet-server (JSON-RPC) and weather via Open-Meteo
- **Stores** telemetry in TimescaleDB and models the building in a **unified RDF graph** (Brick + BACnet + config)
- **Runs** YAML-defined FDD rules (bounds, flatline, hunting, expression) on a configurable schedule
- **Exposes** REST APIs for sites, points, equipment, data-model export/import, bulk timeseries and fault download (Excel-friendly CSV, JSON for cloud), TTL generation, SPARQL validation
- **Visualizes** timeseries and fault results in Grafana

Operators and integrators get full control, lower cost, and no vendor lock-in. Already powering HVAC optimization and commissioning workflows.

2.2 Quick start

```
git clone https://github.com/bbartling/open-fdd.git
cd open-fdd
./scripts/bootstrap.sh
```

Endpoints (direct access):

What	URL	Notes
Frontend (React)	http://localhost:5173	Main UI: dashboard, sites, points, config, faults, plots. Use this for day-to-day workflows.
API (REST)	http://localhost:8000/docs	Swagger UI for integration and scripts. High-level reference: Appendix: API Reference.
Caddy (reverse proxy)	http://localhost:80	Default `stack/caddy/Caddyfile` proxies **`/api*`**, **`/auth*`**, **`/ws*`**, **`/ai*`** to the API (with `/api` prefix stripped) and `/*` to the frontend. See Security and Caddy. Optional hardening (basic auth, TLS) is covered below.
TimescaleDB	127.0.0.1:5432	Database `openfdd` (user `postgres`); host port is loopback-only in compose (Security — stack hardening).
BACnet (diy-bacnet-server)	http://localhost:8080/docs	JSON-RPC API; UDP 47808 for BACnet/IP.
Grafana	http://localhost:3000	**Optional:** `./scripts/bootstrap.sh --with-grafana` (admin/admin). React frontend provides equivalent views.

WebSockets: The API exposes **`/ws/events`** for live updates (faults, CRUD, FDD run). The React app sends the current **access JWT** or **`OFDD_API_KEY`** as **`?token=`** when connecting; REST uses **`Authorization: Bearer`**. See Security & Caddy.

2.2.1 Optional: extra Caddy hardening (basic auth, full route list, TLS)

The **committed `stack/caddy/Caddyfile`** already puts the **API and WebSocket** behind the same **:80** entry as the UI (no **basic auth** in the repo file). To add a **second perimeter** (one browser basic login before the app's own dashboard login), **TLS**, or **many more explicit API paths**, use a custom Caddyfile pattern:

1. **Use a Caddyfile that:**
   - Listens on one port (e.g. `:80`, `:443`, or `:8088`).
   - Optionally enables **basic_auth** for all routes (browser gate in front of the app).
   - Keeps or extends **`/api*`**, **`/auth*`**, **`/ws*`**, **`/ai*`** → **`api:8000`** (strip **`/api`** when using that prefix pattern).
   - Proxies **`/grafana`** to **`grafana:3000`** if you use `--with-grafana`.
   - Proxies **`/*`** to the **frontend** (e.g. `frontend:5173`).
   - When using Caddy basic auth, add **`header_up X-Caddy-Auth <secret>`** on API routes so the API accepts requests that passed Caddy (set **`OFDD_CADDY_INTERNAL_SECRET`** in the API container to the same value).
2. **Full steps:** See Security and Caddy — Quick bootstrap, TLS / stack-hardening notes, default password change, and troubleshooting (401s, WebSocket behind Caddy).
3. **Caddyfile location:** `stack/caddy/Caddyfile`. An extended **example** (many paths + basic auth) is in Security — Caddyfile for protecting the entire API; **HTTPS** starter: `stack/caddy/Caddyfile.https.example`.

2.3 Documentation

Section	Description
System Overview	Architecture, services, data flow
Modular Architecture	Collector/Model/Engine/Interface boundaries and mode-based bootstrap matrix.
Getting Started	Install, bootstrap, first run
Using the React dashboard	

Section	Description
	Overview, Config, Points, Data model, Faults, Plots, Web weather, System resources
BACnet	Discovery, scraping, RPC, RDF/BRICK (knowledge graph, bacpypes3)
Data modeling	Sites, equipment, points (CRUD), Brick TTL, 223P-aligned engineering metadata, SPARQL cookbook, AI-assisted tagging
Fault rules for HVAC	Rule types, expression cookbook, test bench rule catalog
Concepts	Cloud export example — how vendors pull data from the API to their cloud
Operations	Integrity sweep, Overnight review, MCP RAG service, Testing plan
How-to Guides	openfdd-engine vs `open_fdd.engine`, Engine-only / IoT, Data model engineering, Grafana dashboards (optional), Grafana SQL cookbook
Security & Caddy	Basic auth, bootstrap, hardening, optional TLS
Configuration	Platform config, rule YAML
Appendix	API Reference — REST at a glance; Technical reference, Developer guide
Contributing	How to contribute; alpha/beta focus; bugs, rules, docs, drivers

Use the React frontend for config, sites, points, data model, faults, and plots. **API details** (CRUD, config, data-model, download, analytics, BACnet) are summarized in Appendix: API Reference; full OpenAPI at `/docs` and `/openapi.json`. **Grafana** is optional; the React UI provides equivalent timeseries and fault views.

2.4 Stack

Service	Port	Purpose
Caddy	80	Reverse proxy: API/auth/WebSocket/AI routes + frontend (see

Service	Port	Purpose
		Security; optional basic auth / TLS)
API	8000	REST API, Swagger `/docs`, WebSocket `/ws/events`
Frontend	5173	React dashboard (sites, points, faults, plots)
TimescaleDB	5432	PostgreSQL + TimescaleDB
Grafana	3000	**Optional** dashboards (`--with-grafana`); React frontend has equivalent views
diy-bacnet-server	8080	JSON-RPC API (HTTP); optional BACnet2MQTT + experimental MQTT RPC gateway when env vars set (MQTT how-to)
diy-bacnet-server	47808	BACnet/IP (UDP)
Mosquitto (MQTT)	1883	**Optional / experimental:** `./scripts/bootstrap.sh --with-mqtt-bridge` — generic broker for BACnet2MQTT and/or MQTT RPC cmd/ack (MQTT how-to)

2.5 Behind the firewall; cloud export is vendor-led

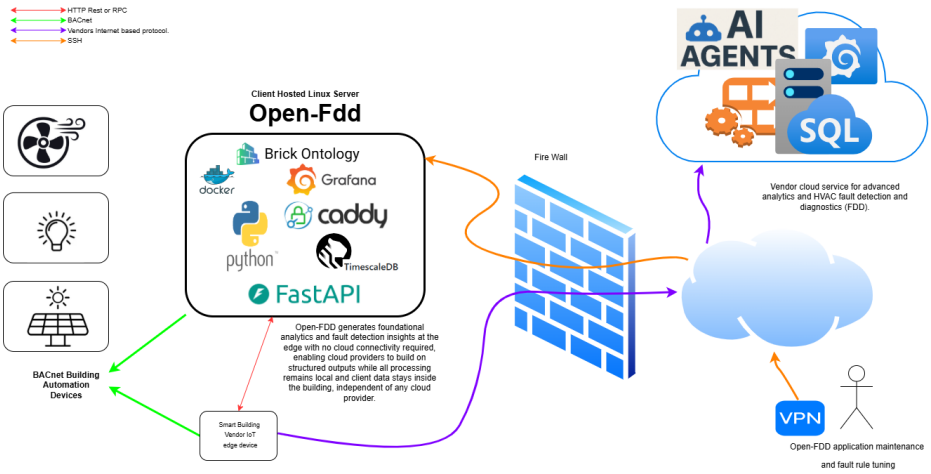
Open-FDD runs **inside the building, behind the firewall**. Analytics and rules run locally; telemetry stays on-premises unless you export it. Cloud/MSI vendors run their **own** export process (e.g. edge gateway or script) that **pulls** from the API over the LAN; Open-FDD does not push to the cloud. See the Cloud export example and `examples/cloud_export.py`. Data stays with the client; the open data model and APIs let you switch vendors without losing data or the model.

3 System Overview

4 System Overview

Open-FDD is an **open-source knowledge graph for building technology systems**, specializing in fault detection and diagnostics (FDD) for HVAC. FDD is often called AFDD when the platform runs it on a schedule (automated FDD). It runs on-premises so facilities keep control of their data and avoid vendor lock-in. This section describes the architecture, services, and data flow.

4.1 Architecture



Open-FDD Edge Platform Architecture

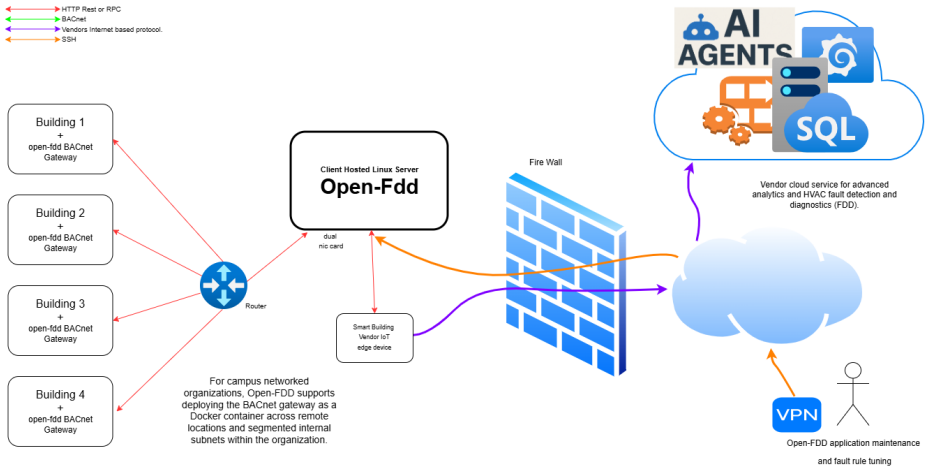
This project is an open-source stack; a cloud or MSI vendor can develop their own Docker container and deploy it on the **same client-hosted server** that runs Open-FDD, pulling from the local API over the LAN. That approach removes the need for a separate IoT or edge device dedicated to the vendor.

4.2 Services

Service	Description
API	FastAPI CRUD for sites, equipment, points. Data-model export/import, TTL

Service	Description
	generation, SPARQL validation. Swagger at `/docs`. Config UI (HA-style data model tree, BACnet test) at `/app/`.
Grafana	Pre-provisioned TimescaleDB datasource only (uid: openfdd_timescale). No dashboards; build your own with SQL from the Grafana SQL cookbook. Use `--reset-grafana` to re-apply datasource provisioning.
TimescaleDB	PostgreSQL with TimescaleDB extension. Single source of truth for metadata and time-series.
BACnet scraper	Polls diy-bacnet-server via JSON-RPC. Writes readings to `timeseries_readings`.
Weather scraper	Fetches from Open-Meteo ERA5 (temp, RH, dewpoint, wind, solar/radiation, cloud cover).
FDD loop	Runs every N hours (see `rule_interval_hours`, `lookback_days` in platform config). Pulls last N days from DB into pandas, **reloads all rules from YAML on every run** (hot reload), runs rules, writes `fault_results` back to DB. No restart needed when tuning rule params.
diy-bacnet-server	BACnet/IP JSON-RPC bridge. Discovery and present-value reads via JSON-RPC; Open-FDD merges discovery into the **data model** (RDF/TTL), not a required CSV.

4.3 Campus-based architecture



Open-FDD Edge Platform Architecture Campus

Remote Open-FDD BACnet gateways (e.g. diy-bacnet-server plus scraper) can be deployed **across each subnet** on the internal campus IT network. Typically each building has its own BACnet network on a unique subnet; a gateway per building or per subnet keeps BACnet traffic local while forwarding data to a **centralized** Open-FDD instance (API, Grafana, FDD loop, database). That gives the campus a single integration point for the cloud-based vendor of choice—one API and one data model for the whole portfolio, without the vendor touching each building's BACnet network directly.

How to set it up: (1) **Remote gateway per building (recommended pattern):** On each subnet run diy-bacnet-server + scraper pointed at the **central Open-FDD API**—create each building's **site** on the central API first and set **`OFDD_BACNET_SITE_ID`** (and related env) so readings and metadata align with the central data model. Prefer **API-mediated** configuration and ingestion over giving every gateway direct **database** credentials. (2) **Central aggregator:** On the central host run DB, API, Grafana, FDD loop (no local BACnet); set **`OFDD_BACNET_GATEWAYS`** to a JSON array of `{url, site_id}` so one scraper polls remote gateways (see Configuration — BACnet). (3) **Advanced / internal-network only:** Pointing **`OFDD_DB_DSN`** at the central Postgres from remote scrapers avoids the API path but **widens exposure** of the database—use only on a trusted LAN with locked-down firewall and ops approval; not the default recommendation. **`run_bacnet_scrape.py`** and per-gateway CSV/env options remain available for special deployments; they are **not** the default product path.

4.4 Data flow

1. **Ingestion:** BACnet scraper and weather scraper write to `timeseries_readings` (point_id, ts, value).
2. **Data model (unified graph):** The building is represented as a **unified graph**—one semantic model combining Brick (sites, equipment, points from the DB), BACnet RDF (from point discovery via diy-bacnet-server), platform config, and room for future ontologies (e.g. ASHRAE 223P). CRUD and **POST /bacnet/point_discovery_to_graph** update this model; SPARQL queries it. One TTL file `config/data_model.ttl` holds the graph; a background thread serializes it to disk every 5 minutes (configurable via `OFDD_GRAPH_SYNC_INTERVAL_MIN`); **POST /data-model/serialize** runs the same write on demand.
3. **FDD (Python/pandas):** The FDD loop pulls data into a pandas DataFrame, runs YAML rules, writes `fault_results` to the database. Fault logic lives in the rule runner; the database is read/write storage.
4. **Visualization:** Grafana queries TimescaleDB for timeseries and fault results.

4.5 Ways to deploy

- **Docker Compose** (recommended): `./scripts/bootstrap.sh`
- **Minimal (raw BACnet only):** DB + Grafana + BACnet server + scraper: `./scripts/bootstrap.sh --minimal` — no FDD, weather, or API.
- **Manual:** Start DB, Grafana, run scrapers and FDD loop from host

---

4.6 Key concepts

- **Sites** — Buildings or facilities.
- **Equipment** — Devices (AHUs, VAVs, heat pumps). Belong to a site.
- **Points** — Time-series references. Have `external_id` (raw name), `rule_input` (FDD column ref), optional `brick_type` (Brick class).
- **Fault rules** — YAML files (bounds, flatline, hunting, expression). Run against DataFrame; produce boolean fault flags. See Fault rules for HVAC.
- **Unified graph** — One semantic model (Brick + BACnet + platform config; future 223P or other ontologies). Stored in `config/`

`data_model.ttl`; maps Brick classes → `external_id` for rule resolution; queryable via SPARQL.

5 Getting Started

6 Getting Started

This page covers **prerequisites** and the **bootstrap script**: how to get the Open-FDD platform running. For configuration, data modeling, and rules, see the Documentation index.

6.1 Do this to bootstrap

1. **Prerequisites:** Docker, Docker Compose, and Git installed (see below).

2. **Clone and run bootstrap** (from any directory):

```
git clone https://github.com/bbartling/open-fdd.git
cd open-fdd
./scripts/bootstrap.sh
```

   Partial module modes are also supported:

```
./scripts/bootstrap.sh --mode collector
./scripts/bootstrap.sh --mode model
./scripts/bootstrap.sh --mode engine
```

   Optional one-liner (HTTPS + login): Piping your password on stdin lets bootstrap create a dashboard user, generate a self-signed certificate for Caddy (then use `https://localhost/`; the browser will warn until you trust the cert), set `stack/.env` for TLS-aware cookies on the API, and start the same full stack as a plain `./scripts/bootstrap.sh`.

```
printf '%s' 'YOUR_PASSWORD' | ./scripts/bootstrap.sh --user
YOURNAME --password-stdin --caddy-self-signed
```

That's it. The script builds and starts the full stack (DB, API, frontend, Caddy, diy-bacnet-server, BACnet scraper, weather scraper, FDD loop), waits for Postgres, runs migrations, and **seeds platform config** via the API (PUT /config) so runtime settings are in the knowledge graph. When it finishes you get:

- **API:** http://localhost:8000 (interactive OpenAPI/Swagger at `/docs` is **disabled** in the shipped API; use the React app and API reference).
- **Frontend:** http://localhost:5173 (or via Caddy http://localhost:80). See Using the React dashboard for what each page does.
- **DB:** `127.0.0.1:5432`/openfdd (postgres/postgres) — bound to loopback only; not exposed on the LAN.

6.2 External Agentic AI (OpenAI-compatible)

Open-FDD does not embed an LLM. Instead, external AI agents (for example an OpenAI-compatible tool like Open-Claw) can take advantage of Open-FDD by calling its APIs:

1. Export the current data model JSON: `GET /data-model/export`
2. Fetch documentation as model context: `GET /model-context/docs` (optionally with `query=...` / keyword retrieval)
3. Import tagged JSON back into the platform: `PUT /data-model/import`

Manual Data Model export/import (JSON) always works without any AI.

See Open-Claw integration and API Reference for endpoint details.

Optional MCP RAG profile:

```
./scripts/bootstrap.sh --with-mcp-rag
```

This starts an MCP-style retrieval sidecar at `http://localhost:8090` using derived index artifacts from canonical docs.

6.3 Prerequisites

- **OS:** Linux only (Ubuntu Server latest, or Linux Mint). Maintainers primarily test on **x86_64** (e.g. Ubuntu Server). Community reports indicate **ARM64** (e.g. Raspberry Pi 4/5) can work with Docker, but

you must validate image availability and resource sizing (RAM/CPU) for your environment. The bootstrap script and Docker stack are not supported on Windows. Keep the system updated:

```
sudo apt update && sudo apt upgrade -y
```

- **Docker and Docker Compose:** Required. Install Docker Engine and Docker Compose (or `docker-compose`). See Docker install for your distro. Your user must be able to run **`docker ps`** (typically: member of the **`docker`** group, then `newgrp docker` or log out/in).

- **Python + argon2-cffi (host):** Whenever you configure **`--user`**, bootstrap hashes the Phase-1 app password with **Argon2** using **host** Python (`./.venv/bin/python` if present, else **`python3`**) — that includes **interactive prompt**, **`--password-stdin`**, **`--password-file`**, and **`OFDD_APP_PASSWORD`** (see `verify_argon2_for_login_bootstrap` in `scripts/bootstrap.sh`). Install **`argon2-cffi`** in that environment. On Ubuntu 24.04+, system pip may refuse installs (**PEP 668**); use a venv, e.g. `python3 -m venv .venv && .venv/bin/pip install argon2-cffi`. Read-only check: `./scripts/bootstrap.sh --doctor` — missing argon2 is a **failure** there unless you passed **`--mode collector`**, **`--no-auth`**, or **`--allow-no-ui-auth`** (then it is a **warning** only).

- **Troubleshooting:** See the root README section *Ubuntu prerequisites* and *Common bootstrap failures* (Docker permissions, missing argon2, PEP 668, venv).

- **Git:** To clone the project:

```
git clone https://github.com/bbartling/open-fdd.git
cd open-fdd
```

- **BACnet (default data driver):** The default data driver is BACnet. Bootstrap **automatically** builds and starts diy-bacnet-server as its own Docker container (plus the BACnet scraper). Run **BACnet discovery** from the UI or API, then add points to the **data model** (with `bacnet_device_id` / `object_identifier`)—the scraper reads **only the database + graph**, not a CSV file. See BACnet → Setup and BACnet overview. To run without BACnet (e.g. central-only with remote gateways), start only the services you need (e.g. `docker compose --profile grafana up -d db api fdd-loop weather-scraper grafana` from `stack/` if you want optional Grafana).

6.4 What the bootstrap script does

`scripts/bootstrap.sh` (run from the **repo root**):

1. Ensures **diy-bacnet-server** exists as a sibling repo (clones it if missing).
2. Runs **docker compose up -d –build** from `stack/` (builds all images, starts all services).
3. Waits for **Postgres** to be ready (~15s).
4. Applies **database migrations** (idempotent; safe on existing DBs).
5. **Seeds platform config** via PUT /config (waits for API, then sends default or `stack/.env` values into the RDF graph).
6. Optionally runs **–reset-data** if you passed that flag (deletes all sites + data-model reset; for testing).

Default full stack: `./scripts/bootstrap.sh` (no flags) starts TimescaleDB, API, **diy-bacnet-server** (BACnet/IP bridge), **BACnet scraper**, weather scraper, FDD loop, and **Caddy** reverse proxy (HTTP on `:80` unless you add TLS flags below). Same service set as the optional one-liner in Do this to bootstrap.

Standard full-stack bootstrap with self-signed TLS (Caddy) and app login: uses **JWT** and **`Authorization: Bearer`**, not HTTP Basic Auth (Security — authentication).

```
        printf '%s' 'YOUR_PASSWORD' | ./scripts/bootstrap.sh --user
YOURNAME --password-stdin --caddy-self-signed
```

Bootstrap options: Run `./scripts/bootstrap.sh --help` for the full list. Summary:

Option	Effect
(none)	Build and start full stack (DB, API, frontend, Caddy, BACnet server, scrapers, FDD loop). Prints API key if generated. Grafana and MQTT broker **not** started by default.
`--with-grafana`	**Optional:** include Grafana at http://localhost:3000 (admin/admin). Add a `/grafana` route in Caddy when you extend the Caddyfile (see Security).
`--with-mqtt-bridge`	**Optional / experimental:** start Mosquitto (`:1883`) for a **generic**

Option	Effect
	broker. Pass **`BACNET2MQTT_*`** / **`MQTT_RPC_*`** env through `stack/.env` to **diy-bacnet-server** for BACnet2MQTT and/or the experimental MQTT RPC gateway (MQTT integration).
`--with-mcp-rag`	**Optional:** include MCP RAG service at http://localhost:8090 (derived from canonical docs and generated docs text).
`--mode MODE`	Module mode: `full` (default), `collector`, `model`, `engine`.
`--minimal`	DB + BACnet server + bacnet-scraper only. No FDD, weather, or API. Add `--with-grafana` for Grafana.
`--verify`	Health checks only: list containers, test DB; exit. Does not start or stop.
`--test`	Run tests and exit. With explicit `--mode`, runs that mode only. Without explicit `--mode` (default full), runs matrix: `collector`, `model`, `engine`, `full`.
`--build SERVICE ...`	Rebuild and restart only listed services, then exit. Services: `api`, `bacnet-server`, `bacnet-scraper`, `caddy`, `db`, `fdd-loop`, `frontend`, `grafana`, `host-stats`, `mcp-rag`, `mosquitto` (with `--with-mqtt-bridge`), `weather-scraper`.
`--build-all`	Rebuild and restart all services; then exit.
`--frontend`	Before start: stop frontend container and remove `frontend_node_modules` volume so the next `up` runs a fresh `npm ci`. Use after changing `frontend/package.json`; the frontend service also runs `npm run build` on every start.

Option	Effect
`--update`	Git pull open-fdd and diy-bacnet-server (sibling), then rebuild and restart (keeps DB).
`--maintenance`	Safe Docker prune only (no volume prune).
`--reset-grafana`	Wipe Grafana volume and re-apply provisioning. DB and other data retained.
`--reset-data`	Delete all sites via API and POST /data-model/reset (testing).
`--retention-days N`	TimescaleDB retention in days (default 365). Env: `OFDD_RETENTION_DAYS`.
`--log-max-size SIZE`	Docker log max size per file (default `100m`). Env: `OFDD_LOG_MAX_SIZE`.
`--log-max-files N`	Docker log max files per container (default 3). Env: `OFDD_LOG_MAX_FILES`.
`--install-docker`	Attempt Docker install (Linux) then continue.
`--skip-docker-install`	Explicitly skip Docker install (no-op; use with scripts that call bootstrap after install).
`--caddy-self-signed`	Self-signed HTTPS for Caddy (`:443`, `:80` → HTTPS): writes certs under `stack/caddy/certs/`, sets `OPENFDD_CADDYFILE` and `OFDD_TRUST_FORWARDED_PROTO=true` in `stack/.env`.
`--caddy-tls-cn HOST`	With `--caddy-self-signed`: certificate CN/SAN (default `openfdd.local`).
`--caddy-http-only`	Revert to the default HTTP-only Caddyfile on `:80`; removes `OPENFDD_CADDYFILE` from `stack/.env` and sets `OFDD_TRUST_FORWARDED_PROTO=false`.
`--no-auth`	

Option	Effect
	Removes auth-related keys from `stack/.env`: **`OFDD_API_KEY`**, app-user keys (**`OFDD_APP_USER`**, **`OFDD_APP_USER_HASH`**, **`OFDD_JWT_SECRET`**, token TTLs), and **`OFDD_BACNET_SERVER_API_KEY`**. Open-FDD API then skips Bearer/JWT enforcement; diy-bacnet-server gets an empty **`BACNET_RPC_API_KEY`** so RPC Bearer middleware is off.
`--user NAME`	Dashboard user: writes `OFDD_APP_USER`, Argon2 hash, `OFDD_JWT_SECRET`, and token TTLs into `stack/.env` (requires a password — next rows).
`--password-file PATH`	Read the dashboard password from a file (first line); avoids putting the password on the command line.
`--password-stdin`	Read the dashboard password from stdin (pipe or redirect into bootstrap; see Security — authentication and `bootstrap.sh` header comments).
(env)	Alternative: set **`OFDD_APP_PASSWORD`** for the dashboard password when using `--user`, or use the interactive prompt if neither file nor stdin is used.

Bearer tokens and API keys (`stack/.env`): With a normal bootstrap (no `--no-auth`), secrets live in **`stack/.env`** (gitignored). They are **not** the same token—each service uses the one that matches its role:

Variable	Consumed by	Role
`OFDD_API_KEY`	**Open-FDD API**	Machine **`Authorization: Bearer`** for REST (Swagger **Authorize**, BACnet

Variable	Consumed by	Role
		scraper → `GET /config`, scripts, agents).
`OFDD_APP_USER`, **`OFDD_APP_USER_HASH`**, **`OFDD_JWT_SECRET`** (+ TTL keys)	**Open-FDD API** (`/auth/login`, JWT validation)	Dashboard login; the browser keeps a **short-lived JWT** and sends **`Authorization: Bearer`** with that token on API calls—not the dashboard password.
`OFDD_BACNET_SERVER_API_KEY`	**Open-FDD API** and **bacnet-scraper** (outbound to the gateway)	**`Authorization: Bearer`** on JSON-RPC to **diy-bacnet-server**. Docker Compose passes the **same value** into the gateway container as **`BACNET_RPC_API_KEY`**. When that env is non-empty, the gateway enforces Bearer on RPC routes except **`POST /server_hello`**; when empty, RPC auth is disabled.

You normally only edit **`stack/.env`**; do not set **`BACNET_RPC_API_KEY`** separately unless you override compose env. Standalone diy-bacnet-server (outside this stack) uses **`BACNET_RPC_API_KEY`** in **its** environment only—see the diy-bacnet-server README.

Dashboard login and piping passwords into `--user` are covered in more detail (including maintenance one-liners) under **Security — authentication**.

To **update** an existing clone: `git pull` then `./scripts/bootstrap.sh`, or `./scripts/bootstrap.sh --update`. Rebuild single services: `./scripts/bootstrap.sh --build api`.

6.5 After bootstrap

- **Grafana (if you used `--with-grafana`):** Open http://localhost:3000. A TimescaleDB datasource is provisioned (`openfdd_timescale`); build dashboards with the Grafana SQL cookbook. If provisioning is wrong, run `./scripts/bootstrap.sh --reset-grafana` (keeps DB data).
- **Minimal mode:** If you used `--minimal`, only DB, BACnet server, and scraper run (plus Grafana **only** if you also passed `--with-grafana`). No API by default; use scraper logs (and Grafana if enabled) to confirm data flow. To add the full stack later, run `./scripts/bootstrap.sh` without `--minimal`.

6.6 Next steps

- **How-to Guides** — Grafana dashboards (optional) and SQL cookbook.
- **Configuration** — Platform config, rule YAML, services that read config from the API.
- **Security & Caddy** — Basic auth, throttling, TLS.
- **Appendix: API Reference** — REST endpoints at a glance. Interactive Swagger/OpenAPI (`/docs`) is **disabled** in the default API build; use the React app, this appendix, or export OpenAPI from a dev configuration if you need a schema browser.

For **BACnet** (discovery and data model): BACnet and BACnet overview. For data modeling and fault rules: Data modeling, Fault rules for HVAC. **Data model export/import (JSON)** works without any AI—you can always export, tag manually or with an external LLM, and import.

7 Using the React dashboard

8 Using the React dashboard

The Open-FDD **React frontend** is available at **http://localhost:5173** (direct) or **http://localhost/** when **Caddy** is the entry point (see Security). Use it after Getting started. When auth is enabled, sign in at **`/login`** (JWT in the browser session + HttpOnly refresh cookie; see Security — authentication). Docker Compose defaults to **`VITE_API_BASE=/api`** so the UI talks to the API through Caddy on the same host/port.

8.1 Navigation

Page	Purpose
Overview	High-level summary: site selector, FDD/weather status, links into the app. The UI does not embed or run an LLM. **External agents** should follow the **help links** on Overview (published docs, PDF, LLM workflow, Open-Claw integration) rather than duplicating endpoint prose in the UI.
OpenFDD Config	View and edit platform config (GET/PUT `/config`): FDD rule interval, lookback, BACnet, Open-Meteo, **`graph_sync_interval_min`** (how often the API writes the merged graph—and Brick `ref:` from the DB—to `data_model.ttl`). **Overview** does not edit this; use **OpenFDD Config** → form → Save. Current settings are shown in a read-only summary first.
BACnet tools	**Step 1 — Sites:** create or delete sites (same API as before; moved here from the Data model page). **Step 2 — BACnet discovery:** gateway selector, Who-Is, point discovery, **Add to data model**.

Page	Purpose
	Optional BACnet tools: read/write RPC and diagnostics (not required for discovery or export/import).
Points	List points (optionally by site). Shows external ID, equipment, Brick type, FDD input, unit, polling, last value and time. Optional BACnet panel for discovery from this page; primary discovery flow is **BACnet tools**.
Data model	Equipment tree, points, export/import, TTL view, SPARQL. Edit equipment and points; run data-model export/import and SPARQL. **Sites** are created on **BACnet tools** (Step 1). Includes a **Danger zone** at the bottom (see below).
Faults	Active fault states and definitions. Filter by site/equipment.
Plots	CSV Plotter workbench (Plotly-style). Load Open-FDD export CSV by site/range/points or drag-drop any CSV; choose X and multiple Y columns; toggle lines/points/both; optionally overlay faults and export CSV joined with `fault_*` 0/1 signals.
Web weather	Open-Meteo weather charts (temp, RH, wind, radiation, etc.) when weather is enabled.
System resources	Host and container metrics (when host-stats is running): memory, load, disk, **per-container CPU/memory** (table + time-series charts). Status badges (green/yellow/red) indicate resource pressure.
Stack status	Overview shows API, BACnet gateway, and MQTT bridge status with **green** / **yellow** / **red** indicators (e.g. MQTT bridge connected = green, enabled but disconnected = yellow).

8.2 Common workflows

- **Change platform config:** OpenFDD Config → edit fields → Save. Changes take effect on the next FDD run or scraper cycle.
- **Discover BACnet points: BACnet tools** → Step 1 (site if needed) → Step 2 (Who-Is, point discovery, Add to data model). You can also use the BACnet panel on **Points**.
- **Inspect or edit the data model:** Data model → browse equipment and points, export/import, view TTL, run SPARQL.
- **Plot and explore CSV:** Plots → choose source (**Open-FDD** or **Upload CSV**) → load/drop file → pick X and Y columns → chart. Optional: select faults and export CSV joined with fault activity columns (`fault_<fault_id>`).
- **Check faults:** Faults → see active faults and definitions; combine with Plots to correlate with sensor data.

For API integration (curl, scripts), see Appendix: API Reference and Swagger at http://localhost:8000/docs.

8.3 Data model Brick: danger zone

{: #data-model-danger-zone }

The **Data model** page ends with a **Danger zone** card. There are two separate actions (lower risk first in the UI, maximum risk last):

Tier	UI label	What it does
Lower risk	Reset graph to DB-only	Calls `POST /data-model/reset` only. Clears the in-memory RDF graph, removes BACnet discovery triples and orphan blank nodes, rebuilds Brick triples from the **current** Postgres state, and rewrites `config/data_model.ttl`. **Does not** delete sites, equipment, or points. Use on a test bench when discovery left stale BACnet RDF but you want to keep DB rows. Use **Check integrity** on the same page if

Tier	UI label	What it does
		you need orphan warnings (reset does not run that check).
Maximum risk	Remove all sites and reset graph	Deletes **every site** via `DELETE /sites/{id}` (cascade removes equipment, points, and related data), then calls `POST /data-model/reset`. **Irreversible** data loss for that relational model (including time-series tied to those points), not "just" clearing the TTL file.

How this fits: Postgres holds the authoritative sites/equipment/points (including `external_id` for time-series columns). The in-memory graph merges Brick (from DB) with BACnet discovery RDF. The TTL file on disk is a snapshot of that graph; the API also persists periodically and on import/reset.

For the same **help links** as the Overview page (GitHub Pages, PDF, LLM prompt, Open-Claw integration), see README — Online Documentation in the repo.

9 Modular Architecture

10 Modular Architecture

Open-FDD supports incremental modular operation while keeping one-command orchestration via `scripts/bootstrap.sh`.

10.1 Module contracts

Module	Concern	Current services
Collector	BACnet collection and ingestion	`db`, `bacnet-server`, `bacnet-scraper`
Model		`db`, `api`, `frontend`, `caddy`

Module	Concern	Current services
	Brick/data-model CRUD and SPARQL	
Engine	Pandas/YAML FDD execution	`db`, `fdd-loop`, `weather-scraper`
Interface	API, websocket, MCP/OpenClaw orchestration	`api`, `frontend`, `caddy`, `mcp-rag`

10.2 Bootstrap mode matrix

Use:

```
./scripts/bootstrap.sh --mode <full|collector|model|engine>
```

- `full` (default): all major services.
- `collector`: BACnet collector only.
- `model`: API/UI/modeling path.
- `engine`: FDD loop path.

Optional overlays still apply (`--with-grafana`, `--with-mcp-rag`, `--with-mqtt-bridge`).

10.3 Feature coverage by mode

Feature	collector	model	engine	full
BACnet gateway + scrape	yes	no	no	yes
CRUD API + docs	no	yes	no	yes
React frontend + Caddy	no	yes	no	yes
SPARQL/ model workflows	no	yes	no	yes
	no	no	yes	yes

Feature	collector	model	engine	full
FDD loop (Pandas/ YAML)				
Weather loop	no	no	yes	yes
MCP RAG sidecar	optional overlay	optional overlay	optional overlay	optional overlay

10.4 Migration stance

The repository remains a single integrated codebase while contracts and modes are formalized. This avoids a hard split now and enables later package/repo extraction with fewer regressions.

10.5 External IoT / “we already have collection + modeling”

If another system owns **ingestion** and **semantic modeling**, you can still use Open-FDD rules in two ways: **Docker `--mode engine`** (DB + `fdd-loop` + weather) or **library-only** `RuleRunner` on **pandas** with the **same YAML** as the platform. See Engine-only deployment and external IoT pipelines. For **`openfdd_engine`** vs **`open_fdd.engine`** vs the container entrypoint, see The optional openfdd-engine package.

11 BACnet Overview

12 BACnet Integration

Open-FDD uses diy-bacnet-server as a BACnet/IP-to-JSON-RPC bridge. Discovery and scrape feed the same **data model** (building as a knowledge graph). The gateway uses **bacpypes3**’s built-in RDF (BACnetGraph) for discovery-to-RDF; Open-FDD merges that TTL and queries via SPARQL.

12.1 Components

Component	Purpose
diy-bacnet-server	BACnet/IP UDP listener + HTTP JSON-RPC API. Discovers devices and objects; exposes present-value reads. Interactive OpenAPI/Swagger is disabled on the gateway; use **BACnet tools** in the React app or JSON-RPC.
BACnet scraper	Platform service. Polls diy-bacnet-server on a schedule; **reads points from the data model** (points with `bacnet_device_id` and `object_identifier`) and writes readings to TimescaleDB.
Data model	Sites, equipment, and points with BACnet addressing (`bacnet_device_id`, `object_identifier`, `object_name`). Configured via the **React frontend** (Config, Data model, Points) or the API. Single source of truth for what to scrape.

12.2 Ports

Port	Protocol	Use
47808	UDP	BACnet/IP
8080	HTTP	JSON-RPC API (no browser docs in default stack)

Only **one** process on the host can use port 47808 (BACnet/IP). Run discovery **before** starting diy-bacnet-server so they do not conflict. diy-bacnet-server runs with `network_mode: host` so it binds to the host's network interface.

12.2.1 Multiple sites / gateways (one Open-FDD, several BACnet bridges)

Use **`OFDD_BACNET_GATEWAYS`** (JSON array of `{ "url", "site_id", … }`) so the scraper and **BACnet tools → Gateway** dropdown can target **remote** diy-bacnet-server instances (one container or VM per site on the OT LAN). Keep **`OFDD_BACNET_SERVER_URL`** as the **default** gateway (often the local site: `http://host.docker.internal:8080` from the API container).

Binding diy-bacnet HTTP to **127.0.0.1 only** on the host is awkward with Docker: Caddy in bridge mode reaches the gateway via **`host.docker.internal`**, which is not the same socket as loopback on Linux. Typical choices: (1) leave the gateway listening on all interfaces on the host (`--public` in compose) and **restrict port 8080 with a host firewall**, or (2) run Caddy with **`network_mode: host`** and `reverse_proxy 127.0.0.1:8080` for `/bacnet/*` if you truly need loopback-only RPC. Remote gateways use their **reachable URL** in `OFDD_BACNET_GATEWAYS` either way.

12.3 Discovery and getting points into the data model

All BACnet configuration for the **default stack** is done via the **data model** and the **React frontend** (or the API). The bundled scraper does not use a BACnet CSV file.

1. **Run Who-Is and point discovery** — Use the React frontend (Config or Data model → BACnet panel) or the API. From the BACnet panel you can run **Test connection**, **Who-Is range**, and **Point discovery** (these proxy to diy-bacnet-server). diy-bacnet-server must be running (e.g. `./scripts/bootstrap.sh` starts it).
2. **Graph and data model** — Use **POST /bacnet/point_discovery_to_graph** (device instance) to put BACnet devices and points into the **in-memory** BACnet graph. Whether the **file** `config/data_model.ttl` updates immediately depends on the **write_file** flag on that request; Brick **`ref:`** external references are

still rebuilt from **Postgres** on full serialize. After discovery, ensure points exist in the DB (CRUD or import) and run a serialize path if you need `ref:` on disk right away—see External representations → When ref triples appear in the TTL. Create points in the DB via the frontend or CRUD (set `bacnet_device_id`, `object_identifier`, `object_name`), or use **GET /data-model/export** → LLM/human tagging → **PUT /data-model/import**.

3. **Run the scraper** — The BACnet scraper loads points that have `bacnet_device_id` and `object_identifier` from the database and polls only those via diy-bacnet-server.

See Points for the BACnet fields and Appendix: API Reference for endpoints. **Port:** Only one process can use port 47808 (BACnet/IP). For API/UI discovery, diy-bacnet-server is already running.

12.4 Data acquisition

The BACnet scraper loads points from the DB where `bacnet_device_id` and `object_identifier` are set; groups by device; calls diy-bacnet-server JSON-RPC (present-value) for each; writes `(point_id, ts, value)` to `timeseries_readings`. Throttling depends on how many points are defined and the poll interval. See Security → Throttling.

12.5 Configuration

Scraper config comes from **environment** and, when the API uses Bearer auth, from **GET /config** (the React Config page controls the interval).

- **`OFDD_BACNET_SERVER_URL`** — diy-bacnet-server base URL (e.g. http://localhost:8080). Required for RPC.
- **Scrape interval** — When **`OFDD_API_KEY`** is set (e.g. in `stack/.env`), the scraper calls GET /config and uses **`bacnet_scrape_interval_min`** from the data model (Config page). Otherwise it uses **`OFDD_BACNET_SCRAPE_INTERVAL_MIN`** env (default: 5). See Configuration → Services that read config from the API.
- **`OFDD_DB_*`** — TimescaleDB connection

13 BACnet graph context

14 BACnet graph context

This document captures BACnet-specific context the **OpenClaw lab** and operators should preserve for future clones, labs, and researchers.

The goal is simple: **do not rely on tribal knowledge**.

14.1 Why this context matters

In live HVAC monitoring, the Open-FDD knowledge graph is not just metadata. It is the bridge between:

- BACnet devices and point addressing
- semantic equipment typing (AHU, VAV, zone, plant, meter, etc.)
- rule applicability
- FDD algorithm selection
- future optimization logic

If we know, from the graph:

- which BACnet device a point belongs to
- which object identifier is being scraped
- which Brick/equipment type the point belongs to
- which Open-FDD rule inputs are mapped

then we know much more than “the point exists”. We know which fault rules should fire, and later we can know which optimization or supervisory logic should apply.

14.2 What must be discoverable from the graph

For BACnet-backed verification, the test harness should always be able to retrieve or verify:

1. **BACnet device inventory** — device instance, device address, device object presence in the graph

2. **BACnet point addressing** — object identifier, point label / object-name alignment, point-to-device membership
3. **Equipment semantic context** — AHU / VAV / zone / plant / building associations, Brick class of the point, whether the point is marked for polling
4. **Rule relevance context** — which YAML fault rules are active, what `rule_input` mappings exist for the point/equipment, what rolling-window parameters matter for that rule
5. **Operational intent** — whether the point is used for FDD only, useful for future optimization / supervisory control, operator-facing or only diagnostic

14.3 Live system interpretation

In a future live deployment, this BACnet graph context should support three increasingly valuable questions:

14.3.1 1. What is connected?

- Which BACnet devices exist?
- Which points are actually mapped?
- Which points are being polled?

14.3.2 2. What kind of HVAC system is this?

- Is the equipment an AHU, VAV, boiler, chiller, plant, or meter?
- What topology relationships exist?
- Which points matter for diagnostics vs operations?

14.3.3 3. What analysis should apply?

- Which FDD rules should run?
- Which optimization or supervisory algorithms are sensible here?
- Which results are expected if the fake BACnet devices deliberately enter a fault state?

This is the beginning of a machine-readable engineering context, not just a UI convenience.

14.4 SPARQL assets in the repo

Relevant reusable queries live under `openclaw/bench/sparql/README.md`, especially:

- `04_bacnet_devices.sparql`
- `08_bacnet_telemetry_points.sparql`
- `05_brick_rule_mapping.sparql`
- `10_ahus.sparql`
- `13_vav_boxes.sparql`
- `19_hvac_equipment.sparql`
- `21_points.sparql`

14.5 BACnet telemetry baseline query

`openclaw/bench/sparql/08_bacnet_telemetry_points.sparql` returns:

- `point_label`
- `brick_class`
- `unit`
- `device_instance`
- `device_address`
- `object_identifier`

That is a strong starting point for proving that BACnet addressing is present in the graph and matched to polling points.

14.6 Backend SPARQL and auth

Graph validation requires `POST /data-model/sparql` with a valid `OFDD_API_KEY` when API auth is enabled. Load the same secret the stack uses (`stack/.env`) in the shell or agent context that runs overnight checks.

Some backends return a simplified shape `{"bindings": [...]}` rather than strict SPARQL JSON `{"results": {"bindings": [...]}}`. Test tooling should accept both.

14.7 BACnet scraper / DIY gateway context

The BACnet scraper gateway OpenAPI is typically at **http://localhost:8080** (Swagger `/docs`). Notable JSON-RPC capabilities include:

- `client_whois_range`
- `client_point_discovery`
- `client_read_property`
- `client_read_multiple`
- `client_supervisory_logic_checks`
- `client_read_point_priority_array`

The bench can validate not only that Open-FDD has graph references, but also that the BACnet-side source system is queryable for independent verification.

14.8 Engineering direction

Evolve toward a formal verification chain:

1. fake BACnet device emits deterministic fault pattern
2. BACnet gateway can independently read that device/point state
3. Open-FDD graph confirms the point is modeled and addressable
4. Open-FDD scrape path lands the telemetry
5. active YAML rules and rolling windows imply a specific fault expectation
6. Open-FDD fault APIs/UI confirm the result

That chain is the real standard for trustworthy FDD validation.

14.9 Where this context is saved

This page lives under **docs/bacnet/** so humans cloning the repo can find it immediately, review it, and extend it without asking the original authors what was “meant.”

15 BACnet, RDF, and BRICK

16 BACnet, RDF, and BRICK

Open-FDD treats the **building as a knowledge graph**: sites, equipment, and points (and BACnet topology) live in a single semantic model. CRUD and **POST /bacnet/point_discovery_to_graph** update this model; SPARQL queries it.

16.1 Roles

Layer	Responsibility
diy-bacnet-server	BACnet RPCs: Who-Is, point discovery (returns JSON objects per device). Open-FDD calls these.
Open-FDD	Calls point discovery, builds clean BACnet TTL from the JSON, merges into the in-memory graph (Brick from DB + BACnet). CRUD owns sites, equipment, points. SPARQL runs over the combined model; sync writes `config/data_model.ttl`.

16.2 Flow

1. **Point discovery** — Open-FDD calls **POST /bacnet/point_discovery** (gateway RPC) for a device instance; gateway returns a list of objects (object_identifier, object_name, etc.).
2. **Point discovery to graph** — **POST /bacnet/point_discovery_to_graph** (same instance, `update_graph: true`) turns that JSON into clean BACnet TTL and updates the in-memory graph; optionally writes `config/data_model.ttl`. SPARQL and **GET /data-model/ttl** see Brick + BACnet.
3. **Points in DB** — Create or edit points via CRUD (set `bacnet_device_id`, `object_identifier`, `object_name`). Optionally **GET /data-model/export** then LLM or human adds Brick types then **PUT /data-model/import**.

4. **Scrape** — Data-model driven (points with `bacnet_device_id` / `object_identifier`).

16.3 Try it (Open-FDD Swagger)

1. Who-Is
- Open **http://localhost:8000/docs**. Find **POST /bacnet/whois_range**. Body: `{"request": {"start_instance": 1, "end_instance": 3456800}}`. Execute; note a device instance (e.g. `3456789`).

2. Point discovery to graph
- Find **POST /bacnet/point_discovery_to_graph**. Body: `{"instance": {"device_instance": 3456789}, "update_graph": true, "write_file": true}`. Execute. Open-FDD calls the gateway, gets point discovery JSON, builds BACnet TTL, and updates the in-memory graph and file.

3. Check
- **GET /data-model/ttl** or **POST /data-model/sparql** (e.g. `SELECT ?dev WHERE { ?dev a bacnet:Device }`).

16.4 Summary

- **Building as knowledge graph**: one semantic model (BRICK + BACnet), updated by CRUD and by **point_discovery_to_graph**.
- **BACnet in the graph** is produced by Open-FDD from point discovery JSON (clean TTL); SPARQL validates and queries the combined model.

17 BACnet-to-fault verification

18 BACnet-to-fault verification

This document defines the verification practice we need for both development testing and future live HVAC deployments.

18.1 Goal

Use **independent BACnet-side evidence** plus **Open-FDD-side evidence** to prove that fault detection is behaving correctly.

This is stronger than merely checking whether a page loads or whether a fault appears somewhere in the UI.

18.2 Required evidence chain

The target verification chain is:

1. **Fake BACnet device schedule is known**
   - use the deterministic schedules in `openclaw/bench/fake_bacnet_devices/README.md`
   - know which devices and points should be healthy vs faulted during the review window
2. **DIY BACnet server RPC confirms source values**
   - use RPC methods such as:
     - `client_read_property`
     - `client_read_multiple`
     - `client_point_discovery`
     - `client_supervisory_logic_checks`
   - confirm the real present values seen by the BACnet gateway
3. **Open-FDD graph confirms BACnet addressing is modeled correctly**
   - query the data model for BACnet devices, point addressing, object identifiers, equipment relationships, polling points
   - use the SPARQL assets in `openclaw/bench/sparql/README.md`
4. **Open-FDD rules context is understood**
   - identify the relevant YAML rule(s) under **`stack/`** (mounted into containers as the live rules directory—not duplicated under `openclaw/`)
   - note the rule inputs and rolling-window assumptions
   - make sure the point/equipment mapping actually supports the rule logic
5. **Open-FDD fault outputs match the expected behavior**
   - compare expected fault windows against:
     - `/download/faults`
     - `/faults/state`
     - `/faults/active`
     - frontend fault views when useful
6. **Result is classified clearly** — PASS, FAIL, or INCONCLUSIVE

18.3 Overnight expectation

The overnight process should increasingly automate the following:

- pull BACnet device and point-address evidence from the Open-FDD graph
- validate that expected BACnet devices are present
- validate that expected point object identifiers are present
- compare the modeled BACnet points to the fake-device schedules
- identify which YAML rules are most relevant to the current fake faults
- compare Open-FDD fault outputs to expected rule behavior
- write a durable report for the morning review (template: `openclaw/reports/overnight-bacnet-verification-template.md`)

18.4 Important interpretation rule for the current fake bench

Do not treat an out-of-bounds spike like `180.0` on `SA-T`, `RA-T`, `MA-T`, or `ZoneTemp` as an automatic product bug.

On this bench that value is intentionally injected by the fake devices during the UTC out-of-bounds window defined in `openclaw/bench/fake_bacnet_devices/fault_schedule.py`.

The right question is not simply "did a spike happen?" but: **did the spike happen in the scheduled UTC window, and did Open-FDD surface the expected `bad_sensor_flag` over the expected rolling window?**

18.5 Minimum overnight report contents

Every overnight BACnet verification report should include:

- date/time window
- target branch context (`master` and optional dev branch if relevant)
- expected BACnet devices
- observed BACnet devices from SPARQL
- expected point/object identifiers
- observed point/object identifiers from SPARQL
- RPC observations from DIY BACnet server
- relevant YAML rule files

- rolling-window notes
- observed Open-FDD fault outputs
- PASS / FAIL / INCONCLUSIVE summary
- follow-up actions

18.6 Current practical limitation

Full unattended graph verification can still be blocked by missing backend auth (`OFDD_API_KEY`) for `POST /data-model/sparql` from the test bench.

That does not change the target practice. It means the environment must be corrected so the verification chain can run unattended.

19 DIY BACnet gateway RPC contract

20 DIY BACnet gateway RPC contract

The DIY BACnet gateway at **:8080** is a **JSON-RPC** API, not a plain REST body-per-method API.

This matters because a naive POST like:

```
{
  "device_instance": 3456790,
  "object_type": "analog-value",
  "object_instance": 1,
  "property_identifier": "present-value"
}
```

will fail with JSON-RPC validation errors.

20.1 Correct shape for `client_read_property`

Use a JSON-RPC envelope:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "client_read_property",
  "params": {
    "request": {
      "device_instance": 3456790,
      "object_identifier": "analog-value,1",
      "property_identifier": "present-value"
    }
  }
}
```

20.2 Example result

```
{
  "jsonrpc": "2.0",
  "result": {
    "present-value": 72.0
  },
  "id": 1
}
```

20.3 Why this matters for overnight verification

A malformed request should be classified as **tooling / API-contract mismatch**.

It should **not** be treated as proof that:

- BACnet is down
- the gateway is unreachable
- Open-FDD ingest is broken

Only after using the correct JSON-RPC contract should the overnight workflow make BACnet-side health judgments.

See also BACnet overview and Swagger at http://localhost:8080/docs when the stack is running.

21 BACnet

22 BACnet

BACnet is the **default data driver** for Open-FDD. Discovery, diy-bacnet-server, and the BACnet scraper are documented here. **Swagger (diy-bacnet-server):** http://localhost:8080/docs

---

22.1 Setup (do this before the platform scrapes data)

Recommended: data model

1. **Start the platform** (including diy-bacnet-server) — e.g. `./scripts/bootstrap.sh`.
2. **Discover devices and points** — In the Config UI (`/app/`) use the BACnet panel: Who-Is range, then Point discovery per device. Or call the API: `POST /bacnet/whois_range`, `POST /bacnet/point_discovery`.
3. **Graph and data model** — Call **POST /bacnet/point_discovery_to_graph** (device instance) to put BACnet devices and points into the in-memory graph and sync `config/data_model.ttl`. Create points in the DB via CRUD (set `bacnet_device_id`, `object_identifier`, `object_name`) or use **GET /data-model/export** → LLM/human tagging → **PUT /data-model/import**.
4. **Scraper runs automatically** — The bacnet-scraper container runs `python -m open_fdd.platform.drivers.run_bacnet_scrape`; it loads BACnet addresses from the knowledge graph (`points`) and polls only those. Configuration is via the frontend and data model only.

22.2 Verification and lab (OpenClaw bench)

Page	Description
BACnet graph context	What the graph must expose for BACnet-backed verification and rules.

Page	Description
BACnet-to-fault verification	Evidence chain from fake devices through RPC, SPARQL, rules, to faults.
DIY BACnet gateway RPC contract	JSON-RPC envelope for `client_read_property` and similar calls.

Example SPARQL files for modeling checks live under `openclaw/bench/sparql/` in the repository (not on this docs site).

23 Overview

24 Data model flow

Open-FDD uses a **unified graph**: one semantic model that combines Brick (sites, equipment, points), BACnet discovery RDF (from bacpypes3 in diy-bacnet-server), platform config, and **engineering / 223-style topology** (`ofdd:*` schedule fields and `s223:*` connection patterns from **Data Model Engineering**). CRUD, discovery, and engineering import all feed this graph; **all backend queries are SPARQL-driven** (rdflib Graph parse + SPARQL; no grep or text search on the TTL). Rules resolve inputs via `ofdd:mapsToRuleInput` in the TTL. See Data model engineering (Brick + 223P MVP) for how that layer ties to FDD, PostgreSQL, and optional **energy-penalty-style** analytics.

24.1 Flow

```
Sites + Equipment + Points (DB)  ← single source of truth
  └─ equipment.metadata.engineering (optional JSON → ofdd/s223 in
TTL)
         │
         ▼
  Data-model export / CRUD
         │
         ▼
  Brick TTL (config/data_model.ttl)  ← Brick section reserialized
on every create/update/delete; same file can include BACnet
discovery + engineering RDF (one file for SPARQL)
         │
```

```
        ▼
FDD column_map (external_id → rule_input)
        |
        ▼
RuleRunner
```

CRUD and Brick TTL sync: The database is the single source of truth. Every **create**, **update**, or **delete** on **sites**, **equipment**, or **points** (via API or data-model import) triggers a reserialize: the Brick TTL file (`config/data_model.ttl`, or `OFDD_BRICK_TTL_PATH`) is regenerated from the current DB and written to disk. So the Brick model is always in sync with CRUD. Deleting a site, device (equipment), or point also cascades to dependent data (timeseries, fault results, etc.) as in a typical CRUD app; see Danger zone — CRUD deletes.

24.2 TTL structure

```
:oat_sensor a brick:Outside_Air_Temperature_Sensor ;
    rdfs:label "OAT (°F)" ;
    ref:hasExternalReference [
        a ref:TimeseriesReference ;
        ref:hasTimeseriesId "oat_sensor" ;
        ref:storedAt "postgresql://localhost:5432/openfdd/
timeseries_readings"
    ] ;
    brick:isPointOf :ahu_7 ;
    ofdd:mapsToRuleInput "oat" .
```

- Brick classes define sensor types
- `ofdd:mapsToRuleInput` maps to FDD DataFrame columns
- `ref:hasExternalReference` links points to BACnet/timeseries external systems
- `rdfs:label` for display

24.3 Data modeling process (discover → tag → import → validate)

1. **Discover** — BACnet discovery and/or manual entry populate **sites**, **equipment**, and **points** in the DB.
2. **Export** — Use **GET /data-model/export** (or the Export card on the Data Model Protocols page) to get JSON for tagging.
3. **Tag** — Either **manual**: copy JSON → use an external LLM or human → paste back (or automate with an external agent like Open-Claw).

If your agent needs platform documentation as context, fetch it from `GET /model-context/docs`.

4. **Import** — **PUT /data-model/import** (or the Import card / auto-import from the agent) writes tagged points and optional equipment relationships into the DB and reserializes the Brick TTL.
5. **Validate** — Use the **Data Model Testing** page (SPARQL, "Summarize your HVAC") to confirm the model; treat the result as pass or fail.

24.4 Data-model API

Endpoint	Description
`GET /data-model/export`	Single export route: BACnet discovery + DB points (optional `?bacnet_only=true`, `?site_id=...`). Use for AI-assisted tagging.
`PUT /data-model/import`	Import JSON: **points** (required) and optional **equipment** (feeds/fed_by). Creates/updates points; does not accept sites or equipments.
`GET /data-model/ttl`	Generate Brick TTL from DB (and in-memory BACnet graph). Optional `?save=true` to write to file.
`POST /data-model/sparql`	Run SPARQL query against the current data model.

24.5 Validation

Use SPARQL to validate:

- All rule inputs have `ofdd:mapsToRuleInput`
- Equipment types and points are consistent
- Brick schema compliance (optional)

25 Sites

26 Sites

Sites represent buildings or facilities. Equipment and points are scoped to a site.

26.1 Structure

Field	Description
`id`	UUID primary key
`name`	Display name

26.2 Scoping

All time-series and equipment queries can be filtered by `site_id`. Grafana dashboards typically scope by site.

26.3 API

- `GET /sites` — List sites
- `GET /sites/{id}` — Get one
- `POST /sites` — Create
- `PATCH /sites/{id}` — Update
- `DELETE /sites/{id}` — Delete (cascades to equipment, points, timeseries, fault results; see Danger zone)

27 Equipment

28 Equipment

Equipment represents physical devices or systems (AHUs, VAVs, heat pumps, weather stations). Points may belong to equipment.

28.1 Structure

Field	Description
`id`	UUID primary key
`site_id`	FK to sites
`name`	Display name (e.g. “AHU-7”)
`equipment_type`	Optional type string
`external_id`	Optional raw identifier from source

28.2 Hierarchy

```
Site
 └── Equipment (AHU, VAV, etc.)
      └── Points (sensors, setpoints)
```

28.3 API

- `GET /equipment` — List equipment (filter by site)
- `GET /equipment/{id}` — Get one
- `POST /equipment` — Create
- `PATCH /equipment/{id}` — Update
- `DELETE /equipment/{id}` — Delete (cascades to points and their timeseries; see Danger zone)

29 AI-assisted data modeling

30 AI-assisted Brick tagging

Open-FDD supports **AI-assisted data modeling**: use **GET /data-model/export** to dump BACnet discovery and existing points as JSON, then have an **LLM or human** add **Brick point classes** (`brick_type`), **Brick equipment classes** (`equipment_type` — best guess the operator confirms), rule inputs, equipment relationships, and which points to poll — and send the result to **PUT /data-model/import**. The platform stays the single source of truth; tagging is the step where raw BACnet objects become a curated, rule-ready dataset.

This workflow is intended for **mechanical engineers and building operators** who need the data model tagged so that FDD rules in `stack/rules/` have the correct inputs and only the right points are logged long-term.

30.1 Workflow (export → tag → import)

1. **Discover** — Use the API: **POST /bacnet/whois_range**, then **POST /bacnet/point_discovery_to_graph** per device. The in-memory graph and `config/data_model.ttl` now contain BACnet devices and objects.

2. **Sites and equipment** — Create the building/site and equipment (AHUs, VAVs, zones) via **POST /sites** and **POST /equipment**. Keep `site_id` UUIDs from export when present. For equipment assignment on import, you can use `equipment_id` UUIDs or `equipment_name` (resolved/created under `site_id`).

3. **Export** — **GET /data-model/export** (or `?bacnet_only=true` for discovery-only). Returns a single JSON array: BACnet discovery rows plus all DB points. Unimported BACnet rows have `point_id: null`, `polling: false`, and null `equipment_id`/`brick_type`/`rule_input`. **`site_id` / `site_name` are pre-filled** when you pass **`?site_id=`** (UUID or site name), when the **Data model** UI has a site selected in the top bar, or when the database has **exactly one site** — so LLMs can assign `equipment_name` safely. With **multiple sites** and no `site_id` filter, those fields stay null until you scope the export.

4. **Tag** — Send the export JSON to an LLM or edit manually. For each point set:

    - **site_id**, **external_id** (time-series key)
    - **brick_type** (e.g. `Supply_Air_Temperature_Sensor`, `Zone_Air_Temperature_Sensor`) — Brick **point** class
    - **equipment_type** (e.g. `Air_Handling_Unit`, `Variable_Air_Volume_Box`) — Brick **1.4 equipment** class local name when you set **equipment_name**, so **Data Model Testing** preset buttons (AHUs, VAVs, chillers, ...) return meaningful counts after import. Use the **best defensible guess**; the operator corrects rare mistakes with one-click SPARQL checks. See LLM workflow canonical prompt and `frontend/src/data/brick-1.4-query-class-allowlist.ts`.
    - **rule_input** (name FDD rules use)
    - **unit** when known (e.g. `degF`, `%`, `cfm`, `0/1` for binary). Units are stored in the data model and TTL; the frontend uses them for Plots axis labels and grouping (e.g. temperatures on one axis, humidity on another). Use standard abbreviations so Plots and exports stay consistent.
    - **equipment_id** (optional) or **equipment_name** (optional, with site_id)
    - **polling: true** for every point that must be **logged long-term** for this job (sensors/setpoints that FDD rules use); **polling: false** for points not needed. The LLM should tell the operator which points will be logged so they can confirm rules in `stack/rules/` have the required inputs.
    - For **equipment relationships** (Brick feeds/isFedBy), optional **engineering**, and **equipment_type** on named equipment, use the import **equipment** array. UUID-based and name-based forms are both supported; name-based relationship rows require `site_id`.

5. **Import** — **PUT /data-model/import** with body: **points** (array) and optional **equipment** (array for feeds/fed_by, **equipment_type**, **engineering**, etc.). The API accepts **only** these two keys — no `sites`, `equipments`, or `relationships`. The backend creates/updates points and equipment relationships, then rebuilds the RDF and TTL.

6. **Scraping** — The BACnet scraper (data-model path) loads points where `bacnet_device_id`, `object_identifier`, and **polling** = **true**; it calls diy-bacnet-server **client_read_multiple** per device and writes to `timeseries_readings`. Grafana and FDD use the same data model.

30.2 API contract (import)

- **points** (required): Each row is a point to **create** (omit `point_id`; set `site_id`, `external_id`, `bacnet_device_id`, `object_identifier`) or **update** (set `point_id` and fields to change). `site_id` should remain the export UUID when present. `equipment_id` is optional if `equipment_name` is provided with `site_id`.
- **equipment** (optional): Updates equipment with Brick feeds/isFedBy, **`equipment_type`** (Brick class for `rdf:type`), **`engineering`**, **`metadata`**. Rows may use UUID fields (`equipment_id`, `feeds_equipment_id`, `fed_by_equipment_id`) or name fields (`equipment_name`, `feeds`, `fed_by`) with `site_id` for resolution.
- **No other top-level keys:** The API model uses **`extra="forbid"`** — unknown keys such as `sites` or `relationships` produce **422 Unprocessable Entity** (they are not silently ignored).

diy-bacnet ready: The list of points to poll is **GET /data-model/export** filtered to rows where **polling** === **true**; each has `bacnet_device_id` and `object_identifier`.

30.3 LLM prompt and agent guidelines

Where this is documented: the tagging rules and agent flow are described **on this page** (you are in the right place) and summarized under **LLM tagging workflow** in the Technical reference. The README points here under **AI and data modeling**.

Canonical prompt text: The **full** copy-paste system prompt (points + equipment rules, strict JSON output, **fault-first pre-flight** for polling) is on the docs site under **LLM workflow — Copy/paste prompt template**. Published hub: **Data modeling** (same content as the repo after Pages build). You can save the same text to a local file (e.g. `pdf/canonical_llm_prompt.txt`) for agents; keep it in sync when you change instructions. The running API does **not** auto-inject this into chat UIs; load it yourself or point agents at the published section. For **HTTP model context**, use **`GET /model-context/docs`**, which serves **`pdf/open-fdd-docs.txt`** (or **`OFDD_DOCS_PATH`**)—regenerate that bundle after doc changes so agents see updates. Agents that can **fetch the web** may also load the modeling hub or the template anchor directly.

What the prompt must cover: It should be generic for **any site** (single building, campus, or tenant). The only input that changes per run is the export JSON from **GET /data-model/export** (optionally ?

`site_id=YourSiteName`). The LLM must preserve all fields, add **brick_type**, **rule_input**, **unit** (when known), and **polling**, and use equipment by name and **site_id** from the export. **Best practice:** the operator should state **which faults/rules** will run and supply **YAML (or snippets) where possible** before treating **polling** as final; the canonical prompt enforces that as **pre-flight / job context** or a **conservative draft**.

For exact schema details and import body (points + equipment only), see the Technical reference. It defines the primary task (Brick tagging), the export → tag → import flow, polling semantics, equipment feeds, and the **unit** field (e.g. degrees-fahrenheit, percent) used by the frontend and stored in the RDF/TTL.

30.4 Rules for this project (what to send the LLM)

Start here for real jobs: which Open-FDD faults you will run, plus **actual YAML** for those rules when you have it — that yields the best **polling** and **rule_input** choices.

To have the LLM align **rule_input**, Brick types, **units**, and **polling** with your FDD rules, include:

- **Fault rules overview** — FDD rule types and YAML format.
- **Expression Rule Cookbook** — AHU, chiller, weather, and advanced recipes; **rule_input** examples (e.g. sat, rat, zone_temp, sf_status).
- Your project's **YAML rule files** (e.g. from `stack/rules/`) — Paste snippets so the LLM uses the same input names.

See LLM workflow (export + rules + validate → import) for the full one-shot flow and validating LLM output so it parses on the backend.

30.5 See also

- LLM workflow — One prompt + export JSON + optional rules; validate with schema/Pydantic; then import and run FDD/tests.
- Data model overview — Flow (DB → TTL → FDD)

30.6 External agent integration (Open-Claw or any OpenAI-compatible LLM)

Where it runs: Outside Open-FDD (in your external LLM/Open-Claw environment). Open-FDD stays the single source of truth; your agent orchestrates `GET /data-model/export` → LLM tagging → `PUT /data-model/import`.

How it's invoked: Your external agent calls `GET /data-model/export` (optionally with a site filter), tags the export via Open-Claw (OpenAI-compatible) using the canonical prompt, then imports validated JSON with `PUT /data-model/import`. If validation fails, include the error text and retry (prompt chaining).

What the agent does: It uses the same API endpoints as the manual process: `GET /data-model/export` (optionally filtered by site) → LLM tagging using the canonical prompt → validate that the returned JSON matches Open-FDD's import schema → `PUT /data-model/import` (or return JSON for the engineer to import).

30.6.1 External retry loop (prompt chaining)

The agent is typically **not** a long-lived process; your external loop exports the model, tags it, validates the output, and retries when needed:

1. **Export** — Call `GET /data-model/export` (optionally filtered by site) and send the export JSON to your agent.
2. **System prompt** — Use the full tagging instructions (this page + Technical reference, or your local `pdf/canonical_llm_prompt.txt` if you maintain one) as the Open-Claw/OpenAI-compatible **system** prompt.
3. **User message** — Send the export JSON as the user message. Optionally prepend any engineer description of feeds/fed_by topology and assumptions so the LLM can choose correct Brick types and relationships.
4. **LLM call** — Call your Open-Claw/OpenAI-compatible LLM once (for typical payloads). For very large exports, your agent may split into chunks and merge results. Request JSON output (for example with `response_format={"type": "json_object"}`).

5. **Validation** — The response is parsed as JSON and validated with **DataModelImportBody** (same schema as import). If validation fails:
   - **Prompt chaining:** When validation fails, append the error text to the next attempt so the model can correct the JSON.
   - **Retry** — Retry up to a configurable max attempts in your agent.
6. **Agent log** — Optionally log attempts/outcomes so the engineer can inspect what changed between retries.
7. **Optional import** — Either return JSON for review, or immediately call `PUT /data-model/import` after validation.

So the agent uses **retry** with **prompt chaining**: keep the same export/rules, and after a validation failure include the error so the LLM can correct its output.

30.7 Automating tagging (external agents)

1. Call `GET /data-model/export` to get the export JSON (optionally with `?site_id=...`).
2. Fetch Open-FDD documentation context for the LLM: `GET /model-context/docs` (use `query=...` to retrieve relevant sections).
3. Prompt/tag with Open-Claw (OpenAI-compatible) to produce import JSON that matches the Open-FDD schema.
4. Validate and import:
   - either return JSON for the engineer to review, or
   - call `PUT /data-model/import` with the validated JSON.

30.8 Troubleshooting import failures (fixing your tagged JSON)

Sometimes the LLM returns valid JSON that passes schema validation, but `PUT /data-model/import` still rejects it during referential checks.

30.8.1 `site_id` must be a UUID

If the UI shows an error like: `site_id must be a valid UUID ... Got: 'BensOffice'` it means a **UUID field** holds a human-readable value (often the site name was pasted into `points[].site_id`).

Human fix (pick the path that matches your row shape):

1. **Preferred:** Replace the bad value with the **real site UUID** from `GET /data-model/export` or `GET /sites` (same site the row belongs to). This is always safe for **updates** (`point_id` set) and for **creates**, and it keeps **name-based** `equipment[]` rows valid — those rows still **require** a proper `site_id` UUID alongside `equipment_name` / `feeds` / `fed_by`.
2. **Creates only (`point_id` omitted):** If the backend should resolve the site by name, you may set `site_id` to `null` (or remove the key) **and** set `site_name` to an existing site name (see import logic in the API). Do **not** use this as a blanket fix for every row: any row that uses **equipment by name** must keep a valid `site_id` UUID.
3. **Bad equipment or point UUIDs:** If the error is about `point_id` or `equipment_id`, remove or replace only those fields with IDs from your DB/export — don't strip `site_id` from unrelated rows.

If `PUT /data-model/import` rejects otherwise-valid JSON, re-run tagging with prompt chaining: include the import error text in your next LLM attempt so it can correct UUIDs/references, or return the JSON for a human to edit and import manually.

30.9 Possible extension: AI assist on Data Model Testing

A future **Data Model Testing** page could offer a second AI assist (same chat style as on Data Model Protocols): the engineer describes what they see (e.g. SPARQL summary, missing relationships, or test failure), and the model suggests **changes as import JSON** (points/equipment) so the engineer can apply and re-test. Under the hood this would use a **separate prompt** from the tagging prompt:

- **Input:** Current data model context (e.g. TTL snippet or SPARQL "Summarize your HVAC" result) plus the engineer's message (e.g. "Add feeds/fed-by between AHU-1 and VAV-1", "Fix the brick_type for SA-T").
- **Output:** Same schema as the tagging flow — valid **points** and **equipment** import JSON only — so the same **PUT /data-model/import** and validation path apply. The engineer reviews, applies, then re-runs SPARQL or predefined tests and passes/fails.

That way: **Setup** = tag from export (same instructions as above, optionally stored in `pdf/canonical_llm_prompt.txt`); **Testing** = revise from current model + chat (optional second file e.g. `pdf/canonical_llm_prompt_testing.txt`). Both use the same API and retry/prompt-chaining behavior; only the system prompt and user message differ.

- Appendix: API Reference — Data model export/import, CRUD
- BACnet overview — Discovery and data-model scrape
- Fault rules — Brick-driven rules in `stack/rules/`

31 Points

32 Points

Points are time-series references in the Open-FDD data model. They link raw telemetry (BACnet objects, weather sensors) to equipment and Brick semantics.

32.1 Structure

Each point has:

Field	Description
`id`	UUID primary key
`site_id`	FK to sites
`equipment_id`	FK to equipment (nullable)
`external_id`	Raw identifier from source (e.g. BACnet object name); unique per site
`fdd_input`	Column name used by FDD rules (e.g. `oat`, `sat`); data-model API uses `rule_input`
`brick_type`	Optional Brick class (e.g. `Outside_Air_Temperature_Sensor`)
`unit`	Optional unit of measure
`description`	Optional

Field	Description
BACnet addressing	
`bacnet_device_id`	Optional. BACnet device instance (e.g. `3456789`). With `object_identifier`, used by the BACnet scraper to poll this point.
`object_identifier`	Optional. BACnet object ID (e.g. `analog-input,1`).
`object_name`	Optional. BACnet object name (often same as `external_id`).

32.2 Time-series data

Readings are stored in `timeseries_readings`:

Column	Type	Description
`point_id`	UUID	FK to points
`ts`	timestamp	Timestamp (UTC)
`value`	float	Numeric value

TimescaleDB hypertable, optimized for range scans and downsampling.

32.3 Layers and mapping

- **BACnet layer:** Points that have `bacnet_device_id` and `object_identifier` are scraped by the BACnet driver (data-model path). Add them via CRUD or after **POST /bacnet/point_discovery_to_graph** and data-model export/import. `external_id` is typically the BACnet object name.
- **Weather layer:** Points with `external_id` = `temp_f`, `rh_pct`, `dewpoint_f`, etc. come from the Open-Meteo weather fetch. They are linked to a synthetic equipment **Open-Meteo** (type **Weather_Service**) per site so they appear under "Open-Meteo" (web weather) in the **Data model** and **Points** tree. In the RDF graph that equipment is tagged with `ofdd:dataSource "open_meteo"` so you can query it via SPARQL.
- **Rule layer:** `fdd_input` / `rule_input` maps to DataFrame column names used by YAML rules.

The data-model API and Brick TTL coordinate `external_id` ↔ `rule_input` ↔ `brick_type`. Open-FDD also emits Brick v1.4 external references (`ref:hasExternalReference`) so points can be resolved to BACnet and timeseries systems. See External representations.

32.3.1 BACnet addressing

Points with `bacnet_device_id` and `object_identifier` set are used by the BACnet scraper (data-model path). Add them via CRUD or after point_discovery_to_graph and data-model export/import. See BACnet overview.

32.4 API

- `GET /points` — List points (filter by site or equipment)
- `GET /points/{id}` — Get one
- `POST /points` — Create
- `PATCH /points/{id}` — Update
- `DELETE /points/{id}` — Delete (cascades to timeseries_readings; see Danger zone)

32.5 Queries

```
SELECT ts, p.external_id, tr.value
FROM timeseries_readings tr
JOIN points p ON p.id = tr.point_id
WHERE p.external_id = 'temp_f'
  AND ts > NOW() - INTERVAL '1 day'
ORDER BY ts DESC;
```

33 LLM workflow (export + rules + validate → import)

34 LLM workflow: one prompt, export JSON, rules, and validated import

This page describes a **single upload** workflow for mechanical engineers: send the **canonical prompt**, the **data-model export JSON**, and **fault/rule context** (strongly recommended) to an LLM; get back import-ready JSON; **validate** it so you know it will parse on the Open-FDD backend; then **PUT /data-model/import** and run FDD (or Sparkl/tests) as needed.

Best practice: Decide **which Open-FDD faults and rules** you will run *before* finalizing `polling`. The canonical prompt is **fault-first**: it tells the model to gather job context (faults, YAML, units, production vs bench, weather scope) or to stay conservative on polling until that context exists.

> **Automated path available:** External OpenAI-compatible agents (for example Open-Claw) can automate the same flow by calling `GET /data-model/export`, fetching platform documentation context from `GET /model-context/docs`, and then calling `PUT /data-model/import` with validated import JSON. The manual copy-paste workflow below always works too.

> **Web-connected agents:** If your LLM can fetch HTTPS documentation, point it at the published **Data modeling** hub and this page's template anchor **Copy/paste prompt template (recommended)** in addition to (or alongside) `GET /model-context/docs`, so instructions stay aligned with the live docs site.

34.1 What you upload to the LLM

1. **The canonical prompt** — Use **AI-assisted data modeling** (section *LLM prompt and agent guidelines*), the inline template **below** on this page ("Copy/paste prompt template"), or the Technical reference — LLM tagging workflow. You can copy from this page or keep an optional local mirror (e.g. `pdf/canonical_llm_prompt.txt`) for agents. The prompt must tell the LLM to return only `{"points": [...], "equipment": [...]}` with Brick types, rule_input slugs, equipment_name, feeds/fed_by, polling, and units (subject to the **pre-flight / job context** rules in the template).

2. **The export JSON** — From **GET /data-model/export** (optionally `?site_id=YourSiteName`). Each row includes point fields (`point_id`, `bacnet_device_id`, `object_identifier`, `object_name`, `external_id`, `site_id`, `site_name`, `equipment_id`, `equipment_name`, `brick_type`, `rule_input`, `unit`, `polling`, ...). Rows may also include **`engineering`** (and **`equipment_metadata`**) — a **per-equipment** mirror from the DB so the LLM sees rated/submittal context next to points. On **import**, write engineering updates under **`equipment[].engineering`**, not as ad-hoc point fields (see Data model engineering and `examples/223P_engineering/`). Unimported BACnet rows have `point_id: null` and null tagging fields until you import.

3. **Faults and rules for this job (strongly recommended)** — So the LLM can align **rule_input**, **Brick types**, **units**, and especially **polling** with what you will actually run in Open-FDD:

   - Paste **YAML** from your project's rules (e.g. from `stack/rules/` or your own rule files). **This is the best input for correct polling decisions** — the model can see exactly which point inputs each rule uses.
   - Or point the LLM at the **Fault rules overview** and **Expression Rule Cookbook** (AHU, chiller, weather, advanced recipes) and list which fault IDs or recipes apply.

> **Polling and YAML:** Without fault/rule context, many BACnet points may be tagged correctly but should still stay `polling: false`. The canonical prompt defaults to **asking for job context first** or emitting a **conservative draft** (mostly `polling: false`) until you provide which faults run and ideally the **actual rule YAML** for the job.

34.2 Copy/paste prompt template (recommended)

Use this as your LLM **system** or **developer** prompt when transforming `GET /data-model/export` into import JSON. This is the **canonical** copy-paste text for the published docs (save to a local file such as `pdf/canonical_llm_prompt.txt` if you want a path for agents or runbooks).

```
You are transforming Open-FDD export JSON into Open-FDD import
JSON.

I will paste JSON from:
GET /data-model/export?site_id=<site_id>

Your job is to return ONLY valid JSON with EXACTLY these two top-
level keys:

{
  "points": [...],
  "equipment": [...]
}

Do not return markdown.
Do not return explanations.
Do not return comments.
Do not return any extra top-level keys.

Exception for multi-turn chat: if fault/rule job context is still
missing and you must ask pre-flight questions first, you may reply
with plain-language questions only in that turn (see PRE-FLIGHT
below). When you emit import JSON, the STRICT OUTPUT REQUIREMENTS
at the end apply.

----------------------------------------------------
PRE-FLIGHT / JOB CONTEXT (required before final polling decisions)
----------------------------------------------------

Polling must be driven by the **actual faults and rules** the
operator plans to run in Open-FDD — not by “this point looks
generally useful.”

Before deciding final polling values, establish job context.

**Required pre-flight question:** What faults/rules are you going
to run in Open-FDD?

Ask or confirm the following (plain language) if not already
answered:

2. Can you provide the actual YAML rule files or rule snippets for
```

this job?
3. Are units imperial or metric for this job?
4. Is this a production/live HVAC job or a bench/demo/test bench?
5. Do you want weather-related rules or weather polling included?
6. Should polling be limited to points required by the selected faults, or also include extra plotting/trending points the operator explicitly wants?

HARD RULE — if faults/rules context is missing:

- Either (a) ask the pre-flight questions above and wait (no import JSON in that turn), OR (b) return import JSON that is an explicitly **conservative draft**: set polling=false for every point unless it is clearly essential; do **not** enable broad polling coverage by guesswork.
- Do not treat aggressive polling=true choices as “final” until the operator has described which faults run and (ideally) supplied YAML or clear rule snippets.

If YAML rules or snippets **are** provided:

- Align brick_type, rule_input, unit, and polling with those rules’ inputs.
- Set polling=true primarily for points **required** by the selected faults, plus any points the operator **explicitly** approved for plotting/trending.
- Do not turn on polling for unrelated “nice to have” points unless the operator asked for broader trending.

After faults/rules are known, you must:

- Infer which **Brick classes** and **rule_input** slugs those rules require (from YAML expressions, `column_map`, and the Expression Rule Cookbook when YAML is thin).
- Set **polling=true** for points that clearly supply those required inputs once tagged.
- Leave **polling=false** for unrelated points unless the operator explicitly asked for broader trending/plotting.

Example: If the operator runs only **sensor-bounds** and **sensor-flatline**-style temperature rules, BACnet **commands**, **setpoints**, **occupancy/schedules**, and **unrelated weather/Open-Meteo** points usually stay **polling=false** unless the operator asked to trend them or a named rule truly needs them.

POINT RULES

For each point in the input:

1. KEEP every existing field exactly as provided, including:

- point_id
- bacnet_device_id
- object_identifier
- object_name
- external_id
- site_id
- site_name
- equipment_id
- any other existing point fields present in the export

Note: the export may include **engineering** / **equipment_metadata** on each row (a mirror of **equipment**-level metadata). For **PUT /data-model/import**, rated/submittal engineering belongs under **equipment[].engineering** (see EQUIPMENT RULES). Do not add spurious extra keys on point objects if your validator rejects them; fold engineering into the **equipment** array once per **equipment_name** + **site_id**.

2. ADD or FILL these fields:
- brick_type (Brick **point** class — sensor, command, setpoint, …)
- rule_input
- polling
- unit
- equipment_name
- equipment_type (Brick **equipment** class — see §4 — powers Data Model Testing summary buttons)

3. brick_type:
Choose the best matching Brick class for the point.
Use a standard Brick class name such as:
- Supply_Air_Temperature_Sensor
- Return_Air_Temperature_Sensor
- Mixed_Air_Temperature_Sensor
- Zone_Air_Temperature_Sensor
- Damper_Position_Command
- Supply_Air_Flow_Sensor
- Static_Pressure_Sensor
- Occupancy_Command
Prefix with "brick:" only if the input already uses that style consistently. Otherwise omit the prefix.

4. equipment assignment:
Assign points to equipment by NAME ONLY.
Example:
- "equipment_name": "AHU-1"
- "equipment_name": "VAV-1"

Do NOT use equipment_id or any UUID for equipment relationships.

Conservative equipment_name inference:
- Set or change **equipment_name** only when **strongly

supported** by BACnet **device** grouping, consistent **object_name** / **external_id** patterns, and any operator brief — not from a single ambiguous point name.
- If grouping is unclear, keep the export’s **equipment_name** / **equipment_id** relationship as-is rather than inventing AHUs/VAVs.

equipment_type (Brick 1.4 equipment class — best LLM guess, human verifies with one click):
- When you set **equipment_name** on points (or list equipment in **equipment[]**), also set **equipment_type** to the **most specific defensible Brick equipment class** as a **bare local name** (no `brick:` prefix), matching what Open-FDD writes to RDF for `rdf:type`.
- Goal: the operator can open **Data Model Testing → Summarize your HVAC** and use preset buttons (**AHUs**, **VAV boxes**, **Chillers**, **Central plant**, **Meters**, …) without re-tagging. Those queries filter on Brick **1.4** classes such as:
  - **Air_Handling_Unit** — central air handler (supply/return/mixed air, fan, coils context)
  - **Variable_Air_Volume_Box** or **Variable_Air_Volume_Box_With_Reheat** — VAV / fan-powered terminal
  - **Chiller**, **Boiler**, **Cooling_Tower**, **Water_Pump**, **Heat_Exchanger**
  - **Chilled_Water_System**, **Condenser_Water_System**, **Hot_Water_System** — distribution systems when the row truly represents that system
  - **HVAC_Zone** — thermal / zoning context when points are zone-level only
  - **Building_Electrical_Meter** — building-level electrical meter equipment (not every kW sensor)
  - Generic fallback: **Equipment** (Open-FDD default) when class is unclear — better than mis-typing a VAV as a Chiller.
- **Same names as the UI presets** are maintained in the frontend allowlist **`brick-1.4-query-class-allowlist.ts`** (regression test: **`data-model-testing-queries.brick.test.ts`**).
- On **equipment[]** rows, include **equipment_type** whenever you include **equipment_name** + **site_id** so created/updated equipment gets the correct type before points attach.

5. unit:
Fill unit when known.
Use standard abbreviations consistent with the job units mode from pre-flight (imperial vs metric):
- temperature -> "degF" or "°F" (imperial) / "degC" or "°C" (metric) as appropriate
- percentage -> "percent" or "%"
- airflow -> "cfm" (imperial) or align with metric job conventions when stated
- binary / boolean -> "0/1"
- power -> "W"

- irradiance -> "W/m²"
Preserve unit strings already present in the export when they are consistent with the stated job mode; do not rewrite known-good values unless the operator asked for conversion.
If unknown, use null.

Conservative units (power, flow, energy):
- Do **not** force **unit** for ambiguous power, flow, or energy points (e.g. kW vs W, CFM vs percent of max, BTU vs MBH) without **operator confirmation** or an unambiguous BACnet engineering unit in context.
- Prefer **null** over a confident-looking guess that could mislead FDD and plots.

6. polling:
Set polling=true **primarily** for points required by the **specific faults/rules** the operator intends to run, plus any **operator-approved** plotting/trending points (see pre-flight question 6).
Set polling=false for points not needed by those rules or approved use cases.
Do **not** enable broad polling just because a point looks generally useful.
If fault/rule context has not been provided, follow the HARD RULE in PRE-FLIGHT (conservative draft or ask first).
If YAML or rule snippets were provided, align polling (and brick_type, rule_input, unit) with those rules.

7. rule_input:
Only populate rule_input when it is actually needed for disambiguation or explicit aliasing.

Use rule_input in these cases:
- two or more points in the same equipment or rule scope share the same Brick class
- a rule needs a stable explicit alias that cannot be reliably inferred from Brick class alone
- the point name clearly indicates a meaningful distinction such as pre/post, entering/leaving, heating/cooling, min/max, supply/return when same-class ambiguity exists

Example:
If one AHU has two Supply_Air_Temperature_Sensor points:
- SAT before coil -> rule_input: "sat_pre"
- SAT after coil -> rule_input: "sat_post"

If only one point of that Brick class exists in scope and it is unambiguous, set:
- "rule_input": null

Do NOT invent unnecessary rule_input values for every point.

8. weather / Open-Meteo / network weather points:
Do not invent weather equipment or weather points that are not in the export.
If the export already contains Open-Meteo or other weather/network points, keep those rows in the output. Set polling=true only when weather-related faults/rules are in scope, or the operator explicitly wants weather polling/plotting; otherwise prefer polling=false for weather rows when fault context does not require them.
Do not add new weather points unless the operator or attached rules explicitly require them.

9. fallback behavior:
If uncertain:
- brick_type = null
- rule_input = null
- unit = null
- polling = false
- equipment_type = omit or null (backend defaults to generic Equipment — use this instead of a wrong class)

REAL-JOB / CONSERVATIVE MODE (not optional for production buildings)

Bench and demo setups can be forgiving; **on a real live HVAC job** the model must not drift from discoverable truth.

Do not invent or guess:

- Extra BACnet devices, synthetic weather stations, or integration “placeholder” equipment
- Equipment rows or feeds/fed_by topology that are not in the export, the user’s brief, or an attached as-built
- Point rows that were not in the export (no hallucinated objects)
- **Engineering** numerics (HP, tons, MBH, design CFM, kW) on **equipment[].engineering** without operator-provided submittal/spec text

Preserve identity exactly (character-for-character when the export provides them):

- `bacnet_device_id`, `object_identifier`, `object_name`, `external_id`, `point_id`, `site_id`, `site_name`, `equipment_id`

When unsure, prefer the safer default:

- `null` for unknown Brick type or unit — not a best guess
- `polling: false` unless the point is clearly needed for the faults/rules or plotting/trending the operator asked for
- Omit `feeds` / `fed_by` rather than inferring ductwork

relationships
- Saying (in a side channel) “cannot determine X from export” is better than fabricating X in JSON

Demo vs live: Use pre-flight answers: test-bench convenience must not override the rules above on a production import.

Operator review before import (short checklist):

1. Compare row count and key BACnet fields to the latest `GET /data-model/export` — no mystery devices.
2. Confirm every non-null `site_id` is still a UUID from `GET /sites`.
3. `PUT dry-run` is not currently supported by the Open-FDD API for `PUT /data-model/import`; use schema/Pydantic validation first, and if you need a no-risk rehearsal, run the same `PUT` against a staging instance while comparing inputs from `GET /data-model/export` and site UUIDs from `GET /sites`.
4. After import, verify a handful of BACnet reads match the gateway for the same object ids.
5. On **Data Model Testing**, run **AHUs**, **VAV boxes**, **Class summary**, etc., to confirm **equipment_type** choices match reality (quick human QA).

EQUIPMENT RULES

Create an "equipment" array with one entry per equipment.

Each equipment item must use this shape:

```
{
  "equipment_name": "AHU-1",
  "equipment_type": "Air_Handling_Unit",
  "site_id": "<same site_id as the points>",
  "feeds": ["VAV-1"]
}
```

or

```
{
  "equipment_name": "VAV-1",
  "equipment_type": "Variable_Air_Volume_Box",
  "site_id": "<same site_id as the points>",
  "fed_by": ["AHU-1"]
}
```

Rules:
- Use equipment names only
- Do not use equipment UUIDs
- Preserve the exact site_id from the export

```
- Set **equipment_type** on each row (see §4) so Brick `rdf:type`
matches **Data Model Testing** preset queries (Brick **1.4**
vocabulary).
- Include feeds/fed_by only when supported by the provided data or
clearly specified by the user
- Do not invent mechanical relationships unless they are
explicitly given or obvious from the provided context

**Optional — engineering metadata (`equipment[].engineering`):**
- Open-FDD **GET /data-model/export** duplicates
**equipment.metadata.engineering** on each point row as
**engineering** for LLM context. On import, put updates under
**equipment[]** using the same **equipment_name** + **site_id**,
with an **engineering** object (nested sections such as
**mechanical**, **electrical**, **controls**, **topology**,
**documents** — see project examples).
- You **may** help capture **rated** data: design CFM, cooling/
heating capacity, fan or pump motor nameplate (HP/kW/FLA), coil or
heat-exchanger ratings, feeder/panel references, **when the
operator provides submittal, schedule, or as-built values** (or
asks you to transcribe pasted specs).
- Do **not** invent numeric ratings (HP, tons, MBH, CFM, kW) from
BACnet object names alone.
- Preserve existing **engineering** from the export when present
unless the operator asks to correct it.

--------------------------------------------------
STRICT OUTPUT REQUIREMENTS
--------------------------------------------------

When emitting import JSON:
- Return ONLY valid JSON
- EXACTLY two top-level keys: "points" and "equipment"
- No "sites"
- No "relationships"
- No "equipments"
- No prose
- No markdown
- No comments

If duplicate external_id appears for the same site, keep the last
occurrence.

Now process the following input JSON:
```

After the final line, paste the **export JSON** (or send it as the next user message). This template works for manual copy-paste and external agents (including Open-Claw workflows).

Again, for agents with web access: Data modeling (GitHub Pages) and this template section.

34.3 Where the rules live

What	Where
Fault rules overview	docs/rules/overview — FDD rule types, YAML format, Brick-driven inputs.
Expression Rule Cookbook	docs/expression_rule_cookbook — AHU, chiller, weather, and advanced recipes; **rule_input** examples and expression patterns.
Actual YAML rule files	`stack/rules/` in the repo (or your `rules_dir`). Paste snippets into the LLM session so it uses the same input names **and** can set polling only where rules need data.

The cookbook is **not** a fault rule file itself; it's documentation. The **rules you want to use** are the YAML files in `stack/rules/` (or your project's rules). For the LLM, prefer pasting that YAML; you can also say "use rule_input slugs from the Expression Rule Cookbook (sat, rat, zone_temp, …)" when YAML is not at hand — but polling decisions will be less certain until you tie them to concrete faults.

34.4 Engineering metadata (fan HP, pumps, coils, design CFM, …)

Yes — the **export** exposes equipment-level engineering on each row as **`engineering`** (and raw **`equipment_metadata`**). The **import** accepts the same data on **`equipment[]`** via **`engineering`**, merged into `equipment.metadata.engineering` in PostgreSQL and emitted into the knowledge graph (see **`open_fdd/platform/data_model_ttl.py`**).

Use this for **nameplate** / **submittal** style fields (design CFM, cooling/heating capacity, motor HP or FLA, feeder panel, topology sketches, source drawing references). **FDD rules still run on time-series**

columns from polled points; engineering scalars are for **context**, SPARQL, dashboards, and downstream analytics unless you extend the runner.

- **Docs:** Data model engineering (Brick + 223P MVP)
- **Example import:** `examples/223P_engineering/engineering_import_example.json`

The canonical prompt tells the LLM to **only** fill or change engineering when the operator supplies evidence — not to invent tons, HP, or CFM from BACnet names alone.

34.5 Feeds / fed_by (HVAC topology)

- **feeds** — This equipment supplies another (e.g. AHU feeds VAV).
- **fed_by** — This equipment is supplied by another (e.g. VAV fed_by AHU).

If topology is not known confidently, omit feeds/fed_by rather than guessing. You can add or refine relationships later.

34.6 Validate before import (so backend CRUD accepts it)

The Open-FDD **PUT /data-model/import** endpoint expects a body that matches the **DataModelImportBody** Pydantic model: exactly `points` (array) and optional `equipment` (array). If the LLM returns extra keys, wrong types, or invalid UUIDs, the API returns **422 Unprocessable Entity**.

To avoid that:

1. **Instruct the LLM** — In your prompt, add: "Return only valid JSON that conforms to the Open-FDD import schema: top-level keys `points` and `equipment` only; each point has the fields listed in the prompt; equipment items use `equipment_name`, `site_id`, optional `feeds` / `fed_by`, optional `engineering` / `metadata`."

2. **Use the API's JSON Schema** — The OpenAPI spec at **GET /openapi.json** (or **GET /docs** and "openapi.json") includes a **schema** for the import body. You can:
   - Export that schema (e.g. the `DataModelImportBody` and nested `PointImportRow` / `EquipmentImportRow` from the spec) and give it to the LLM: "Return JSON that validates against this schema."
   - Or run a local **validation step** before pasting into the UI: validate the LLM's JSON against the same schema (e.g. with a small script or tool that loads the schema and runs `jsonschema.validate`). If it passes, PUT /data-model/import will accept it (aside from referential issues like missing site_id).
3. **Common failure: `site_id` is not a UUID** — `PUT /data-model/import` expects `points[].site_id` to be either:
   - a UUID string (as returned by `GET /data-model/export`), or
   - `null`/omitted (so the backend can resolve it from `points[].site_name`).

   If you see an error like: `site_id must be a valid UUID from GET /sites ... Got: 'BensOffice'` then your LLM replaced the UUID with the human-readable site name. Reinforce: **never** replace `site_id` with `site_name`; if the export has no UUID, keep `site_id` null and keep `site_name`.
4. **Pydantic in the repo** — The backend defines the import shape in **open_fdd/platform/api/data_model.py**: `DataModelImportBody`, `PointImportRow`, `EquipmentImportRow`. A script or pipeline can import those models and validate the LLM output (e.g. `DataModelImportBody.model_validate(json.loads(llm_output))`) before returning it to the human. That way the human only sees JSON that is known to parse on the backend.

34.7 Mechanical engineer flow (short)

1. **Create site** (and optionally equipment) via API or UI; note **site_id**.
2. **Export** — GET /data-model/export?site_id=YourSiteName (or no filter for full dump).
3. **Upload to LLM** — Paste (a) the canonical template above (or your saved copy of the same text), (b) **fault/rule context** (which faults you run + **YAML snippets when possible**), then (c) export JSON.

Optionally include the import JSON Schema so the LLM returns a valid payload.
    - **Fault-first:** Answer the pre-flight questions (or let the model ask them) before treating polling as final.
    - **UUID reminder:** Never replace `points[].site_id` with a human-readable site name; keep the UUID from the export (see **Validate before import** below).
4. **Validate** — Run schema validation or Pydantic validation on the LLM reply so you know it will parse on the backend.
5. **Import** — PUT /data-model/import with the validated JSON.
6. **Run FDD / tests** — Trigger an FDD run or Sparkl (or other) tests as needed for the project.

34.8 See also

- AI-assisted data modeling — Export → tag → import and API contract.
- Data model engineering — `equipment.metadata.engineering`, import/export, TTL / SPARQL.
- Technical reference — PyPI vs repo, LLM tagging workflow; full prompt is above on this page.
- Fault rules overview and Expression Rule Cookbook — Rules and rule_input reference.

35 SPARQL cookbook

36 SPARQL cookbook

Open-FDD keeps one **knowledge graph** (Brick + BACnet + platform config) in `config/data_model.ttl`. All queries in this cookbook run **via the REST API only** — use **POST /data-model/sparql** with a JSON body `{"query": "..."}`. Do not query the TTL file directly; the API is the single entry point so the same flow works for UIs, scripts, and tests.

36.1 Data Model Protocols vs Data Model Testing

The frontend has two tabs:

Tab	Route	Purpose
Data Model Protocols	`/data-model`	Sites, equipment, import/export JSON, view TTL. Build or edit the data model (sites, equipment, points) and paste AI-tagged JSON to import.
Data Model Testing	`/data-model-testing`	**Summarize your HVAC** (one-click SPARQL buttons: Sites, AHUs, Zones, Building, VAV boxes, etc.) and **Custom SPARQL** (textarea, Run, upload .sparql file). Same queries as this cookbook.

Use **Data Model Testing** to run SPARQL in the browser. Use **Data Model Protocols** to manage sites, equipment, and import.

36.2 How to run SPARQL

Method	Use
Data Model Testing (UI)	Open **Data Model Testing** → click a predefined button (Sites, AHUs, Zones, …) or paste SPARQL in Custom SPARQL → Run SPARQL. Optional: check "Include BACnet device and point IDs" for `bacnet_device_id` / `object_identifier` in results.
Swagger	

Method	Use
	http://localhost:8000/docs → **POST /data-model/sparql** → body `{"query": "SELECT ..."}` → Execute.
curl	`curl -X POST http://localhost:8000/data-model/sparql -H "Content-Type: application/json" -d '{"query":"SELECT ?s ?p ?o WHERE { ?s ?p ?o } LIMIT 10"}'`
Upload .sparql file	**Data Model Testing** → Custom SPARQL → "Upload .sparql file", then Run; or **POST /data-model/sparql/upload** with a `.sparql` file.

Response: JSON with a `bindings` array; each element is a map of variable names to values (e.g. `{"site_label": {"value": "DemoSite"}}`). Use the `value` field for literals and the full object for URIs.

36.3 Prefixes and graph contents

The graph uses these namespaces. Include them in your queries:

Prefix	Namespace
`brick:`	https://brickschema.org/schema/Brick#
`rdfs:`	http://www.w3.org/2000/01/rdf-schema#
`ref:`	https://brickschema.org/schema/Brick/ref#
`ofdd:`	http://openfdd.local/ontology#
`bacnet:`	http://data.ashrae.org/bacnet/2020#
`:` (default)	http://openfdd.local/site#

Graph contents: Brick triples (sites, equipment, points from the DB), Brick external references (`ref:BACnetReference`, `ref:TimeseriesReference`), BACnet triples (devices and objects from discovery), and **ofdd:PlatformConfig** (platform config from GET/PUT /config). All are queryable together.

36.4 Recipe 1: Platform config

Platform config is stored as RDF (same graph). Use it to verify or audit settings after PUT /config or bootstrap.

All config triples:

```
PREFIX ofdd: <http://openfdd.local/ontology#>
SELECT ?p ?v WHERE {
  ?c a ofdd:PlatformConfig .
  ?c ?p ?v .
}
```

Example bindings: `ofdd:ruleIntervalHours`, `ofdd:bacnetServerUrl`, `ofdd:bacnetEnabled`, etc. Predicates are camelCase (e.g. `ruleIntervalHours`). Use this to confirm config is in the graph after seeding.

36.5 Recipe 2: Sites and counts

All sites with labels:

```
PREFIX brick: <https://brickschema.org/schema/Brick#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
SELECT ?site ?site_label WHERE {
  ?site a brick:Site .
  ?site rdfs:label ?site_label .
}
```

Site and aggregate counts (equipment count, point count per site):

```
PREFIX brick: <https://brickschema.org/schema/Brick#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
SELECT ?site_label (COUNT(DISTINCT ?equipment) AS ?
equipment_count) (COUNT(?point) AS ?point_count) WHERE {
  ?site a brick:Site .
  ?site rdfs:label ?site_label .
  ?equipment brick:isPartOf ?site .
  ?point brick:isPointOf ?equipment .
```

```
}
GROUP BY ?site ?site_label
```

Use this to validate the data model after CRUD or import (see Getting started).

36.6 Recipe 3: Equipment and points by site

Equipment labels for a given site (replace `"YourSiteLabel"` with your site name):

```
PREFIX brick: <https://brickschema.org/schema/Brick#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
SELECT ?eq_label WHERE {
  ?eq brick:isPartOf ?site .
  ?site rdfs:label "YourSiteLabel" .
  ?eq rdfs:label ?eq_label .
}
```

Point labels for a site (points under equipment under site):

```
PREFIX brick: <https://brickschema.org/schema/Brick#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
SELECT ?pt_label WHERE {
  ?pt brick:isPointOf ?eq .
  ?eq brick:isPartOf ?site .
  ?site rdfs:label "YourSiteLabel" .
  ?pt rdfs:label ?pt_label .
}
```

Equipment feeds (Brick `feeds` relationships for a site):

```
PREFIX brick: <https://brickschema.org/schema/Brick#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
SELECT ?eq_label ?feeds_label WHERE {
  ?eq brick:isPartOf ?site .
  ?site rdfs:label "YourSiteLabel" .
  ?eq rdfs:label ?eq_label .
  ?eq brick:feeds ?other .
  ?other rdfs:label ?feeds_label .
}
```

In the **Data Model Testing** UI, the **Feed topology** button runs a graph-wide query that lists both `brick:feeds` and `brick:isFedBy` edges (with optional site / equipment labels). See `frontend/src/data/data-model-testing-queries.ts` (`equipment_feeds_topology`).

36.7 Recipe 3b: External references (Brick v1.4)

List point-to-reference relationships:

```
PREFIX ref: <https://brickschema.org/schema/Brick/ref#>
SELECT ?point ?rep ?type WHERE {
  ?point ref:hasExternalReference ?rep .
  ?rep a ?type .
}
LIMIT 200
```

BACnet references from point metadata:

```
PREFIX ref: <https://brickschema.org/schema/Brick/ref#>
PREFIX bacnet: <http://data.ashrae.org/bacnet/2020#>
SELECT ?point ?oid ?device WHERE {
  ?point ref:hasExternalReference ?rep .
  ?rep a ref:BACnetReference ;
       bacnet:object-identifier ?oid ;
       bacnet:objectOf ?device .
}
```

Timeseries references:

```
PREFIX ref: <https://brickschema.org/schema/Brick/ref#>
SELECT ?point ?tsid ?store WHERE {
  ?point ref:hasExternalReference ?rep .
  ?rep a ref:TimeseriesReference ;
       ref:hasTimeseriesId ?tsid ;
       ref:storedAt ?store .
}
```

36.8 Recipe 4: BACnet devices and objects

After **POST /bacnet/point_discovery_to_graph**, the graph contains BACnet devices and their objects.

All BACnet devices:

```
PREFIX bacnet: <http://data.ashrae.org/bacnet/2020#>
SELECT ?dev WHERE { ?dev a bacnet:Device }
```

Object count per device:

```
PREFIX bacnet: <http://data.ashrae.org/bacnet/2020#>
SELECT ?dev (COUNT(?obj) AS ?n) WHERE {
  ?dev a bacnet:Device .
  ?dev bacnet:contains ?obj .
```

```
}
GROUP BY ?dev
```

Object names for a device (e.g. device instance 3456789):

```
PREFIX bacnet: <http://data.ashrae.org/bacnet/2020#>
SELECT ?name WHERE {
  ?dev a bacnet:Device ;
       bacnet:device-instance 3456789 ;
       bacnet:contains ?obj .
  ?obj bacnet:object-name ?name .
}
```

Use these to confirm discovery results are in the graph and to cross-check with GET /data-model/export.

BACnet device and object addresses for points with polling = true (join Brick points to BACnet via matching label and object-name):

```
PREFIX bacnet: <http://data.ashrae.org/bacnet/2020#>
PREFIX ofdd: <http://openfdd.local/ontology#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?device_instance ?device_address ?object_identifier ?
point_label
WHERE {
  ?point ofdd:polling true ;
         rdfs:label ?point_label .
  ?bacnet_obj bacnet:object-name ?point_label ;
              bacnet:object-identifier ?object_identifier .
  ?device bacnet:contains ?bacnet_obj ;
          bacnet:device-instance ?device_instance ;
          bacnet:device-address ?device_address .
}
ORDER BY ?device_instance ?object_identifier
```

Returns one row per polling point that has a matching BACnet object: `device_instance`, `device_address`, `object_identifier` (e.g. `analog-input, 1`), and `point_label`. Use this for scraper config or to list "which BACnet addresses are in the data model and polled."

Why "No bindings (empty result)"? The query above joins **Brick** points to **BACnet** objects. If your graph has no BACnet triples (no discovery has been run, or you only have Brick from a TTL file), there are no `bacnet:Device`, `bacnet:contains`, or `bacnet:object-name` triples, so the join returns nothing. Use the query below when you only have Brick data.

Polling points with a Brick type (no BACnet) — use this when the graph has no BACnet data; it returns points that have `ofdd:polling true` and a Brick class (e.g. `Return_Air_Temperature_Sensor`):

```
PREFIX brick: <https://brickschema.org/schema/Brick#>
PREFIX ofdd: <http://openfdd.local/ontology#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?brick_class ?point_label ?unit
WHERE {
  ?point ofdd:polling true ;
         rdfs:label ?point_label ;
         a ?brick_type .
  FILTER(STRSTARTS(STR(?brick_type), STR(brick:)))
  BIND(REPLACE(STR(?brick_type), "https://brickschema.org/schema/
Brick#", "") AS ?brick_class)
  OPTIONAL { ?point ofdd:unit ?unit . }
}
ORDER BY ?point_label
```

Data Model Protocols: Equipment table vs "View full data model (TTL)" — The **Equipment** table on **Data Model Protocols** comes from the **database** (REST API: sites, equipment, points). **Data Model Testing** (SPARQL) and **"View full data model (TTL)"** on Protocols use the **in-memory graph**, which is synced from the DB (Brick) plus BACnet discovery. If you see a full TTL with equipment but "No equipment configured" in the table, the DB may be empty or out of sync: the graph can be loaded from `config/data_model.ttl` on startup, but the UI list is always from the API/DB. Use the site selector in the top bar ("All sites" vs a specific site); ensure sites and equipment exist in the DB (e.g. create via the UI or import).

36.9 Recipe 5: FDD — points and rule mapping

FDD rules resolve Brick classes to DataFrame columns via the TTL. These queries help you validate that points have the right Brick types and rule inputs for your rules (e.g. sensor_bounds or expression rules).

Brick class → label → rule input (timeseries reference and FDD input name):

```
PREFIX brick: <https://brickschema.org/schema/Brick#>
PREFIX ofdd: <http://openfdd.local/ontology#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
```

```
SELECT ?brick_class ?label ?rule_input WHERE {
  ?point a ?brick_type .
  FILTER(STRSTARTS(STR(?brick_type), STR(brick:)))
  BIND(REPLACE(STR(?brick_type), "https://brickschema.org/schema/
Brick#", "") AS ?brick_class)
  ?point rdfs:label ?label .
  OPTIONAL { ?point ofdd:mapsToRuleInput ?rule_input . }
}
ORDER BY ?brick_class ?rule_input
LIMIT 50
```

- **`rdfs:label`** = `external_id` in the DB = timeseries key (column name in the FDD DataFrame).
- **`ofdd:mapsToRuleInput`** = `fdd_input` in the DB = name the rule uses when multiple points share a Brick class.

Brick classes used in the model (count per class):

```
PREFIX brick: <https://brickschema.org/schema/Brick#>
SELECT ?brick_class (COUNT(?point) AS ?count) WHERE {
  ?point a ?brick_type .
  FILTER(STRSTARTS(STR(?brick_type), STR(brick:)))
  BIND(REPLACE(STR(?brick_type), "https://brickschema.org/schema/
Brick#", "") AS ?brick_class)
}
GROUP BY ?brick_class
ORDER BY DESC(?count)
```

Use this to ensure every Brick class your rules reference (e.g. `Supply_Air_Temperature_Sensor`, `Zone_Temperature_Sensor`) has at least one point in the graph.

36.10 Recipe 6: FDD — time-series references for rules

Rules in `stack/rules/` (e.g. `sensor_bounds.yaml`, expression rules) declare **inputs** by Brick class only. The runner uses the Brick TTL (this graph) to resolve:

- **Brick class** → points of that type → **rdfs:label** (external_id) → column in the timeseries DataFrame.

So the **timeseries reference** for FDD is the point's **rdfs:label** (which equals `external_id` in the DB and is the key in `timeseries_readings` when joined via `point_id`). To list "which columns will this rule see for site X?":

Points that map to rule inputs for a site (equipment hierarchy + label = timeseries key):

```
PREFIX brick: <https://brickschema.org/schema/Brick#>
PREFIX ofdd: <http://openfdd.local/ontology#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?equipment_label ?point_class ?label ?rule_input WHERE {
  ?site a brick:Site .
  ?site rdfs:label "YourSiteLabel" .
  ?equipment brick:isPartOf ?site .
  ?equipment rdfs:label ?equipment_label .
  ?point brick:isPointOf ?equipment .
  ?point a ?brick_type .
  FILTER(STRSTARTS(STR(?brick_type), STR(brick:)))
  BIND(REPLACE(STR(?brick_type), "https://brickschema.org/schema/
Brick#", "") AS ?point_class)
  ?point rdfs:label ?label .
  OPTIONAL { ?point ofdd:mapsToRuleInput ?rule_input . }
}
ORDER BY ?equipment_label ?point_class
```

Interpretation: For each row, **`label`** is the timeseries column name (external_id). The rule runner loads timeseries by (site, external_id) and builds the DataFrame; then Brick class (and optional rule_input) select which column is used for each rule input. So this query answers "what timeseries refs (columns) does the FDD pipeline have for this site?"

Polling points only (points that the BACnet scraper writes to; exclude non-polling):

```
PREFIX brick: <https://brickschema.org/schema/Brick#>
PREFIX ofdd: <http://openfdd.local/ontology#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
SELECT ?label ?brick_class WHERE {
  ?pt a ?brick_type .
  FILTER(STRSTARTS(STR(?brick_type), STR(brick:)))
  BIND(REPLACE(STR(?brick_type), "https://brickschema.org/schema/
Brick#", "") AS ?brick_class)
  ?pt rdfs:label ?label .
  ?pt ofdd:polling true .
}
ORDER BY ?brick_class ?label
```

These queries mirror the logic in `scripts/automated_testing/sparql/` (e.g. 05_brick_rule_mapping.sparql); run them via the API for validation or for building a small "FDD readiness" report.

36.11 Recipe 7: Integrity and completeness

Graph check — Use **GET /data-model/check** for triple counts, sites count, BACnet device count, and orphan warnings. For custom checks, use SPARQL:

Count triples:

```
SELECT (COUNT(*) AS ?n) WHERE { ?s ?p ?o }
```

Equipment hierarchy snapshot (site → equipment type → point class → rule input):

```
PREFIX brick: <https://brickschema.org/schema/Brick#>
PREFIX ofdd: <http://openfdd.local/ontology#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
SELECT ?site_label ?equipment_label ?equipment_type ?point_class ?
rule_input WHERE {
  ?site a brick:Site .
  ?site rdfs:label ?site_label .
  ?equipment brick:isPartOf ?site .
  ?equipment rdfs:label ?equipment_label .
  ?equipment ofdd:equipmentType ?equipment_type .
  ?point brick:isPointOf ?equipment .
  ?point a ?brick_type .
  FILTER(STRSTARTS(STR(?brick_type), STR(brick:)))
  BIND(REPLACE(STR(?brick_type), "https://brickschema.org/schema/
Brick#", "") AS ?point_class)
  OPTIONAL { ?point ofdd:mapsToRuleInput ?rule_input . }
}
ORDER BY ?site_label ?equipment_label ?point_class
LIMIT 100
```

36.12 Predefined buttons (Data Model Testing)

The **Data Model Testing** tab has one-click buttons under **Summarize your HVAC** that run the same SPARQL as this cookbook.

Button	Recipe
Sites	Recipe 2 (sites with labels)
AHUs, Zones, Building, VAV boxes, VAVs per AHU	Count or list by Brick type
	Count or list equipment types

Button	Recipe
Chillers, Cooling towers, Boilers, Central plant	
HVAC equipment, Meters, Points, Class summary	Counts and Brick class summary

Checking **Include BACnet device and point IDs** on Data Model Testing runs a variant of each query that adds `bacnet_device_id` and `object_identifier` (for telemetry and algorithms).

36.13 SPARQL smoke suite (repo files)

The same queries used by **Data Model Testing** predefined buttons and by optional bench automation are checked in under **`openclaw/bench/sparql/`** in the open-fdd repository. Run them via **POST /data-model/sparql** (body `{"query": "..."}`), **POST /data-model/sparql/upload**, or **Data Model Testing** → Custom SPARQL (paste or upload). See also the directory README on GitHub for the mapping to UI buttons and cookbook recipes.

File	Role
`01_platform_config.sparql`	Recipe 1: platform config
`02_sites_labels.sparql`	Recipe 2 / **Sites** button
`02_site_counts.sparql`	Site aggregate counts
`03_equipment_labels_testbench.sparql`	Equipment for test-bench site
`03_point_labels_testbench.sparql`	Points for test-bench site
`04_bacnet_devices.sparql`	Recipe 4: BACnet devices
`05_brick_rule_mapping.sparql`	Recipe 5: Brick → rule input
`06_polling_points_brick_type.sparql`	Polling points + Brick type (no BACnet join)
`07_count_triples.sparql`	Triple count
`08_bacnet_telemetry_points.sparql`	BACnet telemetry / addressing
`09_graph_db_sync_counts.sparql`	Graph vs DB sync counts
`10_ahus.sparql`	**AHUs** button
`11_zones.sparql`	**Zones** button
`12_building.sparql`	**Building** button

File	Role
`13_vav_boxes.sparql`	**VAV boxes** button
`14_vavs_per_ahu.sparql`	**VAVs per AHU** button
`15_chillers.sparql`	**Chillers** button
`16_cooling_towers.sparql`	**Cooling towers** button
`17_boilers.sparql`	**Boilers** button
`18_central_plant.sparql`	**Central plant** button
`19_hvac_equipment.sparql`	**HVAC equipment** button
`20_meters.sparql`	**Meters** button
`21_points.sparql`	**Points** button
`22_class_summary.sparql`	**Class summary** button
`23_orphan_external_references.sparql`	Orphan `ref:` hygiene
`24_operator_site_context.sparql`	Operator site context discovery

36.14 Related docs

Topic	Page
Data model flow	Overview
AI-assisted tagging	AI-assisted tagging
Rule inputs and Brick	Expression rule cookbook · Fault rules overview · Test bench rule catalog
Grafana (SQL, not SPARQL)	Grafana SQL cookbook
Dev reference	Technical reference

37 External representations (Brick v1.4)

38 External representations (Brick v1.4)

Open-FDD models Brick points with Brick's **ref schema** so software can discover how each point maps to external systems (BACnet and timeseries storage) using:

- `ref:hasExternalReference`
- `ref:BACnetReference`
- `ref:TimeseriesReference`

Open-FDD keeps legacy `ofdd:*` BACnet fields for compatibility, but `ref:*` is the preferred representation.

38.1 When ref triples appear in the TTL

{: #when-ref-appears-in-the-ttl }

Brick external references (`ref:hasExternalReference`, etc.) are **materialized when the Brick section of the graph is rebuilt from Postgres** and the graph is serialized (same path as `sync_brick_from_db` / TTL generation). They are **not** invented solely by BACnet discovery in memory: discovery updates **BACnet** RDF in the graph, while `ref:BACnetReference` / `ref:TimeseriesReference` under each point come from **current point rows** in the DB when that Brick sync runs.

You will see `ref:` in TTL (or in **View full data model**) after any of these:

- **CRUD or import** on sites, equipment, or points (the API reserializes Brick to disk on those paths).
- **GET /data-model/ttl** with `save=true` (default in many clients), **POST /data-model/serialize**, or **Data model → Serialize to TTL** in the UI.
- The **background graph sync** on a timer: interval **`graph_sync_interval_min`** (platform config). In the React app this is

edited on **OpenFDD Config**, not the Overview page. See Configuration → Platform keys.

So if you only ran BACnet discovery and are watching **`config/data_model.ttl` on disk**, `ref:` may lag until the next serialize (timer or explicit action above). Discovery may also request a TTL write when **write to file** is enabled on that flow—otherwise rely on CRUD/import or an explicit serialize.

38.2 What Open-FDD emits

For each point in `config/data_model.ttl`:

- A **timeseries reference**:
  - `a ref:TimeseriesReference`
  - `ref:hasTimeseriesId "<external_id>"`
  - `ref:storedAt "<postgresql://host:port/db/timeseries_readings>"`
- A **BACnet reference** when BACnet addressing is known (`bacnet_device_id` + `object_identifier`):
  - `a ref:BACnetReference`
  - `bacnet:object-identifier "<object-type,instance>"`
  - optional `bacnet:object-name "<name>"`
  - `bacnet:objectOf <bacnet://<device_instance>>`
  - `brick:BACnetURI "bacnet://<device>/<object>/present-value"`

38.3 Example

```
:pt_123 a brick:Zone_Air_Temperature_Sensor ;
    rdfs:label "ZoneTemp" ;
    ref:hasExternalReference [
        a ref:BACnetReference ;
        bacnet:object-identifier "analog-input,3" ;
        bacnet:object-name "BLDG-Z410-ZATS" ;
        brick:BACnetURI "bacnet://123/analog-input,3/present-
value" ;
        bacnet:objectOf <bacnet://123>
    ] ;
    ref:hasExternalReference [
        a ref:TimeseriesReference ;
        ref:hasTimeseriesId "ZoneTemp" ;
        ref:storedAt "postgresql://localhost:5432/openfdd/
timeseries_readings"
```

```
    ] ;
    brick:isPointOf :eq_ahu_1 .
```

38.4 SPARQL discovery pattern

Use this to enumerate all external references and their types:

```
PREFIX ref: <https://brickschema.org/schema/Brick/ref#>
SELECT ?point ?rep ?type WHERE {
  ?point ref:hasExternalReference ?rep .
  ?rep a ?type .
}
```

BACnet-only references:

```
PREFIX ref: <https://brickschema.org/schema/Brick/ref#>
PREFIX bacnet: <http://data.ashrae.org/bacnet/2020#>
SELECT ?point ?oid ?dev WHERE {
  ?point ref:hasExternalReference ?rep .
  ?rep a ref:BACnetReference ;
       bacnet:object-identifier ?oid ;
       bacnet:objectOf ?dev .
}
```

Timeseries-only references:

```
PREFIX ref: <https://brickschema.org/schema/Brick/ref#>
SELECT ?point ?tsid ?store WHERE {
  ?point ref:hasExternalReference ?rep .
  ?rep a ref:TimeseriesReference ;
       ref:hasTimeseriesId ?tsid ;
       ref:storedAt ?store .
}
```

39 Data modeling

40 Data modeling

The Open-FDD data model is built around **sites**, **equipment**, and **points**: the same entities you create, read, update, and delete via the API (**CRUD** = Create, Read, Update, Delete). The REST API (Swagger at `/docs`) and the data-model endpoints (`/data-model/export`, `/data-model/`

`import`, `/data-model/ttl`) are the main way to manage this data. The database is the single source of truth; the Brick TTL file is regenerated on every change so FDD rules and Grafana stay in sync.

Concepts (entities):

- **Sites** — Buildings or facilities. All equipment and points are scoped to a site.
- **Equipment** — Physical devices (AHUs, VAVs, heat pumps). Belong to a site; have points.
- **Points** — Time-series references (sensors, setpoints). Link to equipment and site; store `external_id`, optional `brick_type` and `rule_input` for FDD.
- **External representations (Brick v1.4)** — `ref:hasExternalReference` mappings from points to BACnet and timeseries systems.
- **Engineering metadata + 223P topology** — equipment-level engineering metadata is stored as JSON and emitted to RDF using `ofdd:*` extension predicates plus `s223:*` topology patterns (`hasConnectionPoint`, connection conduits, mediums). Same graph as Brick and timeseries refs, so you can join **rated capacity** and **topology** to **FDD results** for optimization context or rough energy-penalty sketches (Data model engineering).
- **SPARQL cookbook** — Run SPARQL via POST /data-model/sparql only: config, data model, BACnet, FDD rule mapping, time-series references. Copy-paste queries for validation and UIs.
- **AI-assisted data modeling** — Export → LLM or human tagging → import (Brick types, rule_input, polling, equipment feeds). External agents (e.g. Open-Claw) can use `GET /model-context/docs` as platform documentation context and `GET /mcp/manifest` for HTTP discovery; the canonical import schema is the same.
- **LLM workflow (export + rules + validate → import)** — Canonical prompt (fault-first pre-flight for polling), export JSON, faults + rules (prefer YAML); validate with schema or Pydantic; then PUT /data-model/import and run FDD/tests. Published hub.

Framework: CRUD is provided by the FastAPI REST API. The data-model API adds bulk export/import and Brick TTL generation; see Overview for the flow (DB → TTL → FDD column_map) and Appendix: API Reference for endpoints.

41 Overview

42 Fault rules

Fault rules are YAML-defined checks run against time-series DataFrames. Each rule produces a boolean fault flag. **Open-FDD is 100% Brick-model driven:** rule inputs refer only to Brick classes; column mapping comes from the Brick TTL via SPARQL and Brick external timeseries references (timeseries storage), which Open-FDD embraces at the heart of FDD.

42.1 Where rules live: config path and how to manage them

Single directory: All FDD rule YAML files live in one directory configured as **`rules_dir`** in platform config (RDF: `ofdd:rulesDir`, e.g. `"stack/rules"`). The FDD loop loads from this path every run; the rules API (list, upload, download, delete) uses the same path. **You still need `rules_dir` in config** — it defines where files are stored. The frontend does not replace it; it lets you manage the files *in* that directory.

Two ways to manage rules:

Method	Use case
React frontend (Faults page)	Upload new YAML, download existing files, delete files, and **Sync definitions** so the fault_definitions table updates without waiting for the next FDD run. Preferred when you have UI access.
Files on disk	Edit or add files directly under the configured path (e.g. `stack/rules/` on the host or in the container). Same outcome: next FDD run (or **Sync definitions** from the UI) picks them up.

Config: `rules_dir: "stack/rules"` (GET/PUT `/config` or `OFDD_RULES_DIR` at bootstrap). If that path does not exist, the loop falls back to `stack/rules`. Default rules ship in `stack/rules/` (e.g. `sensor_bounds.yaml`, `sensor_flatline.yaml`). See the Expression Rule Cookbook to add or adapt rules. For how YAML becomes pandas operations and how telemetry is pivoted into a DataFrame, see YAML rules → Pandas (under the hood).

Reference rule library (not loaded by default): Additional AHU, chiller, heat-pump, and weather YAML examples used for lab automation and docs live under **`openclaw/bench/rules_reference/`** in the repo. See the Test bench rule catalog for a table of every file with GitHub links.

42.2 Hot reload (edit → run → view)

Open-FDD is **AFDD** (Automated Fault Detection and Diagnostics). The project **supports hot reloading of YAML rule files** so the AFDD maintainer can tune faults without restarting. Rule YAML contains fault logic and **params** (thresholds, tolerances, windows); the FDD loop loads from `rules_dir` every run — no restart.

Fault definitions: Each FDD run syncs the loaded rules into the `fault_definitions` table (fault_id, name, category, equipment_types). When you add or edit a rule (via frontend upload or by editing a file in `rules_dir`), the next run updates the DB and the Faults UI reflects the change. From the frontend you can also click **Sync definitions** to update the definitions table immediately.

1. **Add or edit** rules: use the Faults page (upload/paste YAML, or choose file) or edit files in `stack/rules/*.yaml`. Change `params` (e.g. `tolerance`, `rolling_window`) to tune sensitivity.
2. **Run** FDD: wait for the next scheduled run (per `rule_interval_hours` and `lookback_days` in platform config), or trigger with `touch config/.run_fdd_now` or `POST /run-fdd` (see Appendix: API Reference). Or use **Sync definitions** in the UI to only refresh the definitions table.
3. **View** fault results in the React Faults/Plots views or in Grafana (see Grafana SQL cookbook). Every run reloads all rules from disk — hot reload.

42.3 Rule types

Type	Purpose
`bounds`	Value outside `[low, high]`
`flatline`	Rolling spread < tolerance (stuck sensor)
`hunting`	Excessive state changes (PID hunting)
`expression`	Custom pandas/numpy expression
`oa_fraction`	OA fraction vs design airflow error
`erv_efficiency`	ERV effectiveness out of range

42.4 YAML structure

Inputs use **Brick class names** only; the Brick TTL (SPARQL) resolves each to a DataFrame column via external timeseries references. No `column` in YAML.

```yaml
name: sensor_bounds
type: bounds
flag: bad_sensor
inputs:
  Outside_Air_Temperature_Sensor:
    brick: Outside_Air_Temperature_Sensor
  Supply_Air_Temperature_Sensor:
    brick: Supply_Air_Temperature_Sensor
params:
  low: 40
  high: 90
  rolling_window: 6   # optional: require N consecutive True samples before flagging
```

Per-rule rolling window: Set `params.rolling_window` (e.g. `6`) in a rule to require that many consecutive True samples before the fault is flagged. Omit for "flag on any True." See `sensor_flatline.yaml`, `weather_temp_stuck.yaml`.

42.5 Input resolution (100% Brick)

- **Brick TTL:** Each rule input declares a Brick class (e.g. `brick: Supply_Air_Temperature_Sensor`). The engine resolves Brick points via SPARQL to their external timeseries reference

(`ref:TimeseriesReference` / `ref:hasTimeseriesId`), yielding the DataFrame column name. See Brick timeseries storage.
- **Column map:** Built from the Brick model (e.g. `rdfs:label` or external_id) → column names; no `column` in rule YAML.

42.6 Running rules

- **Platform:** FDD loop loads rules from `rules_dir` (default `stack/rules`) each run; edit YAML and trigger a run to see changes in Grafana.
- **Standalone:** `RuleRunner(rules_path=...)` or `RuleRunner(rules=[...])`; call `run(df, ...)`.

42.7 Engineering metadata (223P-style) and energy-style analytics

FDD rules consume **time-series columns** built from the Brick graph (`ref:TimeseriesReference` + `ofdd:mapsToRuleInput`). **Engineering** data (design CFM, cooling tons, heating MBH, `s223:*` connection topology) lives on **equipment** in the DB and in the **same TTL** after import.

That split is intentional: the RuleRunner stays fast and pandas-centric. **Downstream**, you can combine:

- **Fault results** (which equipment, which fault, when), and
- **SPARQL** (or export TTL) for **rated capacity** and **topology** on that equipment, and
- **SQL** on `timeseries_readings` for duty estimates over the fault window

…to approximate **energy penalties** or rank impact (see §5 in `examples/223P_engineering/README.md` and Data model engineering (Brick + 223P MVP)).

42.8 Cookbook

See Expression Rule Cookbook for AHU, chiller, weather, and advanced recipes.

43 Expression Rule Cookbook

44 Expression Rule Cookbook

A reference for building fault detection rules in open-fdd. Rules use **YAML** with **expression** type: when the expression evaluates to **True**, a fault is flagged. open-fdd injects **NumPy** as `np` into expression evaluation, so you can use `np.maximum`, `np.abs`, `np.sqrt`, etc. for vectorized math. **Tuning:** Change `params` in YAML and trigger an FDD run (or wait for the schedule); the platform hot-reloads rules each run, so no restart is needed. See Fault rules overview and Configuration (`rules_dir` is RDF-driven via GET `/config`).

> **`column_map` — platform vs library:** On the **full AFDD stack**, **`fdd-loop`** runs **`run_fdd_loop`**, which builds **`column_map`** from **`config/data_model.ttl`** using **`BrickTtlColumnMapResolver`** (same mapping as **`resolve_from_ttl`** in **`open_fdd.engine.brick_resolver`**). Recipes below stay **Brick-oriented** and assume that path. When you run **`RuleRunner`** yourself (**`pip install open-fdd`**, notebooks, external IoT pipelines), **you** supply **`column_map`** as a **dict** (logical / Brick-class keys → your DataFrame column names), or plug a custom **`ColumnMapResolver`** if you extend the platform loop. The rule YAML shape does not change; only **who** builds the map changes. See Engine-only deployment and external IoT pipelines and The optional openfdd-engine package.

44.1 100% Brick-model driven (no column in rules)

Rule inputs use **Brick class names** only (e.g. `Supply_Air_Temperature_Sensor`). The **Brick TTL** is the source of truth: SPARQL resolves each Brick point to its external timeseries reference (`ref:TimeseriesReference` / `ref:hasTimeseriesId`), which yields the DataFrame column. Open-FDD embraces Brick timeseries storage at the heart of FDD — **rules never reference `column`**; the data

model provides the mapping. See the SPARQL cookbook (Recipe 5) and run similar queries via Data Model Testing or `scripts/automated_testing/sparql/`.

- **Platform (DB):** Points have `brick_type` and `external_id`; TTL is built from DB. PATCH points or use data-model import to set `brick_type` (e.g. `Supply_Air_Temperature_Sensor` for SA-T).
- **Disambiguation:** When multiple points share a Brick class, use `ofdd:mapsToRuleInput` in TTL; the runner resolves `BrickClass|rule_input`.

44.2 How to define expressions

1. **Inputs** — Declare Brick classes only (`brick: Supply_Air_Temperature_Sensor`). The runner resolves these via the Brick TTL (SPARQL) and Brick external timeseries references to DataFrame columns. Do not add `column` in YAML.
2. **Params** — Thresholds and constants go in `params`. Reference by name (e.g. `err_thresh`, `vfd_max`).
3. **Expression** — Must evaluate to a boolean Series (True = fault). Use `&` (AND), `|` (OR), `~` (NOT). Use `.diff()`, `.rolling()`, `.notna()` for time-series logic.

Minimal example (TTL-driven; no column):

```yaml
name: high_temp_check
type: expression
flag: high_temp_flag

inputs:
  Supply_Air_Temperature_Sensor:
    brick: Supply_Air_Temperature_Sensor

params:
  max_temp: 90.0

expression: |
  Supply_Air_Temperature_Sensor > max_temp
```

The Brick TTL (and SPARQL) maps each Brick class to the actual column via external timeseries references. Set `points.brick_type` in the DB and sync TTL so the engine can resolve columns from the Brick model.

44.3 AHU rules (reference-style)

The following rules follow common industry practice for air-handling fault detection. Rules A through M are adapted from ASHRAE Guideline 36 (GL36) AFDD guidance. Thresholds and logic are tunable; adjust params for your site.

44.3.1 Rule A — Duct static below setpoint at full fan speed

Static pressure under setpoint while supply fan runs near maximum. May indicate duct leakage, undersized fan, or terminal damper issues. *Adapted from GL36 AFDD guidance.*

```
        name: duct_static_low_at_full_speed
        description: Static pressure below setpoint when fan at full speed
(GL36-inspired)
        type: expression
        flag: rule_a_flag
        equipment_type: [VAV_AHU]

        inputs:
          Supply_Air_Static_Pressure_Sensor:
            brick: Supply_Air_Static_Pressure_Sensor
          Supply_Air_Static_Pressure_Setpoint:
            brick: Supply_Air_Static_Pressure_Setpoint
          Supply_Fan_Speed_Command:
            brick: Supply_Fan_Speed_Command

        params:
          sp_margin: 0.12
          drv_hi_frac: 0.93
          drv_near_hi: 0.06

        expression: |
          (Supply_Air_Static_Pressure_Sensor <
Supply_Air_Static_Pressure_Setpoint - sp_margin) &
(Supply_Fan_Speed_Command >= drv_hi_frac - drv_near_hi)
```

44.3.2 Rule B — Blended air temp below expected band

Blended air temp should lie between outdoor and return. If below both (minus tolerance), suspect sensor or mixing fault. *Adapted from GL36 AFDD guidance.*

```
        name: blend_temp_below_band
        description: Blended air temp below expected range (OAT/RAT)
(GL36-inspired)
```

```
        type: expression
        flag: rule_b_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Mixed_Air_Temperature_Sensor:
            brick: Mixed_Air_Temperature_Sensor
          Return_Air_Temperature_Sensor:
            brick: Return_Air_Temperature_Sensor
          Outside_Air_Temperature_Sensor:
            brick: Outside_Air_Temperature_Sensor
          Supply_Fan_Speed_Command:
            brick: Supply_Fan_Speed_Command

        params:
          blend_tol: 1.15
          rat_tol: 1.15
          oat_tol: 1.15

        expression: |
          (Mixed_Air_Temperature_Sensor - blend_tol <
np.minimum(Return_Air_Temperature_Sensor - rat_tol,
Outside_Air_Temperature_Sensor - oat_tol)) &
(Supply_Fan_Speed_Command > 0.01)
```

44.3.3 Rule C — Blended air temp above expected band

Blended air temp above the higher of OAT and RAT (plus tolerance) indicates mixing or sensor fault. *Adapted from GL36 AFDD guidance.*

```
        name: blend_temp_above_band
        description: Blended air temp above expected range (OAT/RAT)
(GL36-inspired)
        type: expression
        flag: rule_c_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Mixed_Air_Temperature_Sensor:
            brick: Mixed_Air_Temperature_Sensor
          Return_Air_Temperature_Sensor:
            brick: Return_Air_Temperature_Sensor
          Outside_Air_Temperature_Sensor:
            brick: Outside_Air_Temperature_Sensor
          Supply_Fan_Speed_Command:
            brick: Supply_Fan_Speed_Command

        params:
          blend_tol: 1.15
          rat_tol: 1.15
          oat_tol: 1.15
```

```
        expression: |
          (Mixed_Air_Temperature_Sensor - blend_tol >
np.maximum(Return_Air_Temperature_Sensor + rat_tol,
Outside_Air_Temperature_Sensor + oat_tol)) &
(Supply_Fan_Speed_Command > 0.01)
```

Hunting/oscillation — see rule types (hunting).

44.3.4 Rule D — Discharge air cold when heating commanded

Discharge air temp below blended air when heating valve is open. Indicates heating coil or valve failure. *Adapted from GL36 AFDD guidance.*

```
        name: discharge_cold_when_heating
        description: Discharge air below blended air when heating active
(GL36-inspired)
        type: expression
        flag: rule_d_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Mixed_Air_Temperature_Sensor:
            brick: Mixed_Air_Temperature_Sensor
          Supply_Air_Temperature_Sensor:
            brick: Supply_Air_Temperature_Sensor
          Valve_Command:
            brick: Valve_Command
          Supply_Fan_Speed_Command:
            brick: Supply_Fan_Speed_Command

        params:
          blend_tol: 1.15
          sat_tol: 1.15
          fan_delta_t: 0.55

        expression: |
          (Supply_Air_Temperature_Sensor + sat_tol <=
Mixed_Air_Temperature_Sensor - blend_tol + fan_delta_t) &
(Valve_Command > 0.01) & (Supply_Fan_Speed_Command > 0.01)
```

OA fraction — see rule types (oa_fraction).

44.3.5 Rule E — SAT too low with full heating

Heating valve fully open but SAT remains below setpoint. Indicates undersized coil or valve failure. *Adapted from GL36 AFDD guidance.*

```
        name: sat_too_low_full_heating
        description: SAT below setpoint with heating valve fully open
(GL36-inspired)
        type: expression
        flag: rule_e_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Supply_Air_Temperature_Sensor:
            brick: Supply_Air_Temperature_Sensor
          Supply_Air_Temperature_Setpoint:
            brick: Supply_Air_Temperature_Setpoint
          Valve_Command:
            brick: Valve_Command
          Supply_Fan_Speed_Command:
            brick: Supply_Fan_Speed_Command

        params:
          supply_err_thres: 1.0

        expression: |
          (Supply_Air_Temperature_Sensor < Supply_Air_Temperature_Setpoint
- supply_err_thres) & (Valve_Command > 0.9) &
(Supply_Fan_Speed_Command > 0)
```

44.3.6 Rule F — SAT/MAT mismatch in economizer mode

In economizer mode (min mechanical cooling), SAT should approximate MAT. Large deviation suggests coil bypass or sensor error. *Adapted from GL36 AFDD guidance.*

```
        name: discharge_blend_mismatch_econ
        description: Discharge and blended air diverge in economizer mode
(GL36-inspired)
        type: expression
        flag: rule_f_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Mixed_Air_Temperature_Sensor:
            brick: Mixed_Air_Temperature_Sensor
          Supply_Air_Temperature_Sensor:
            brick: Supply_Air_Temperature_Sensor
          Damper_Position_Command:
            brick: Damper_Position_Command
```

```
          Valve_Command:
            brick: Valve_Command

        params:
          fan_delta_t: 0.55
          blend_tol: 1.15
          sat_tol: 1.15
          econ_min_open: 0.12

        expression: |
          (np.abs(Supply_Air_Temperature_Sensor - fan_delta_t -
Mixed_Air_Temperature_Sensor) > np.sqrt(sat_tol**2 +
blend_tol**2)) & (Damper_Position_Command > econ_min_open) &
(Valve_Command < 0.1)
```

44.3.7 Rule G — Ambient too warm for free cooling

Outside air temperature exceeds SAT setpoint while economizer is active and mechanical cooling is off. Economizer should not be providing "free" cooling under these conditions. *Adapted from GL36 AFDD guidance.*

```
        name: ambient_warm_free_cool
        description: Outdoor air above setpoint in free cooling mode
(GL36-inspired)
        type: expression
        flag: rule_g_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Outside_Air_Temperature_Sensor:
            brick: Outside_Air_Temperature_Sensor
          Supply_Air_Temperature_Setpoint:
            brick: Supply_Air_Temperature_Setpoint
          Damper_Position_Command:
            brick: Damper_Position_Command
          Valve_Command:
            brick: Valve_Command

        params:
          oat_tol: 1.15
          fan_delta_t: 0.55
          sat_tol: 1.15
          econ_min_open: 0.12

        expression: |
          (Outside_Air_Temperature_Sensor - oat_tol >
Supply_Air_Temperature_Setpoint - fan_delta_t + sat_tol) &
(Damper_Position_Command > econ_min_open) & (Valve_Command < 0.1)
```

44.3.8 Rule H — Ambient vs blended mismatch (econ + mech cooling)

When both economizer and mechanical cooling are active, MAT should approach OAT. Large deviation suggests inadequate mixing or damper fault. *Adapted from GL36 AFDD guidance.*

```
        name: ambient_blend_mismatch_econ_mech
        description: Outdoor and blended air diverge in econ+mech cooling
(GL36-inspired)
        type: expression
        flag: rule_h_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Outside_Air_Temperature_Sensor:
            brick: Outside_Air_Temperature_Sensor
          Mixed_Air_Temperature_Sensor:
            brick: Mixed_Air_Temperature_Sensor
          Valve_Command:
            brick: Valve_Command
          Damper_Position_Command:
            brick: Damper_Position_Command

        params:
          oat_tol: 1.15
          blend_tol: 1.15

        expression: |
          (np.abs(Mixed_Air_Temperature_Sensor -
Outside_Air_Temperature_Sensor) > np.sqrt(blend_tol**2 +
oat_tol**2)) & (Valve_Command > 0.01) & (Damper_Position_Command >
0.9)
```

44.3.9 Rule I — Ambient vs blended mismatch (econ-only)

In economizer-only mode, MAT should match OAT. Deviation indicates damper or mixing fault. *Adapted from GL36 AFDD guidance.*

```
        name: ambient_blend_mismatch_econ
        description: Outdoor and blended air diverge in economizer-only
mode (GL36-inspired)
        type: expression
        flag: rule_i_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Outside_Air_Temperature_Sensor:
            brick: Outside_Air_Temperature_Sensor
          Mixed_Air_Temperature_Sensor:
```

```
            brick: Mixed_Air_Temperature_Sensor
          Damper_Position_Command:
            brick: Damper_Position_Command

        params:
          oat_tol: 1.15
          blend_tol: 1.15

        expression: |
          (np.abs(Mixed_Air_Temperature_Sensor -
Outside_Air_Temperature_Sensor) > np.sqrt(blend_tol**2 +
oat_tol**2)) & (Damper_Position_Command > 0.9)
```

44.3.10 Rule J — Discharge above blended in cooling

SAT exceeds MAT when cooling (econ+mech or mech-only) is active. Indicates underperforming cooling coil or valve. *Adapted from GL36 AFDD guidance.*

```
        name: discharge_above_blend_cooling
        description: Discharge air above blended air in cooling modes
(GL36-inspired)
        type: expression
        flag: rule_j_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Supply_Air_Temperature_Sensor:
            brick: Supply_Air_Temperature_Sensor
          Mixed_Air_Temperature_Sensor:
            brick: Mixed_Air_Temperature_Sensor
          Valve_Command:
            brick: Valve_Command
          Damper_Position_Command:
            brick: Damper_Position_Command

        params:
          fan_delta_t: 0.55
          blend_tol: 1.15
          sat_tol: 1.15
          econ_min_open: 0.12

        expression: |
          (Supply_Air_Temperature_Sensor > Mixed_Air_Temperature_Sensor +
np.sqrt(sat_tol**2 + blend_tol**2) + fan_delta_t) &
(((Damper_Position_Command > 0.9) & (Valve_Command > 0)) |
((Damper_Position_Command <= econ_min_open) & (Valve_Command >
0.9)))
```

44.3.11 Rule K — Discharge above setpoint in full cooling

SAT above setpoint with cooling at full capacity. Suggests undersized coil or plant limits. *Adapted from GL36 AFDD guidance.*

```
        name: discharge_above_sp_full_cool
        description: Discharge air above setpoint in full cooling mode
(GL36-inspired)
        type: expression
        flag: rule_k_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Supply_Air_Temperature_Sensor:
            brick: Supply_Air_Temperature_Sensor
          Supply_Air_Temperature_Setpoint:
            brick: Supply_Air_Temperature_Setpoint
          Valve_Command:
            brick: Valve_Command
          Damper_Position_Command:
            brick: Damper_Position_Command

        params:
          sat_tol: 1.15
          econ_min_open: 0.12

        expression: |
          (Supply_Air_Temperature_Sensor > Supply_Air_Temperature_Setpoint
+ sat_tol) & (((Damper_Position_Command > 0.9) & (Valve_Command >
0.9)) | ((Damper_Position_Command <= econ_min_open) &
(Valve_Command > 0.9)))
```

44.3.12 Rule L — Cooling coil delta-T when inactive

Temperature drop across cooling coil when it should be off. Indicates leaking CHW valve or coil bypass. *Adapted from GL36 AFDD guidance.*

```
        name: clg_coil_drop_when_off
        description: Temperature drop across cooling coil when it should
be off (GL36-inspired)
        type: expression
        flag: rule_l_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Cooling_Coil_Entering_Air_Temperature_Sensor:
            brick: Cooling_Coil_Entering_Air_Temperature_Sensor
          Cooling_Coil_Leaving_Air_Temperature_Sensor:
            brick: Cooling_Coil_Leaving_Air_Temperature_Sensor
          Heating_Valve_Command:
```

```
            brick: Valve_Command
          Cooling_Valve_Command:
            brick: Valve_Command
          Damper_Position_Command:
            brick: Damper_Position_Command

        params:
          enter_tol: 1.15
          leave_tol: 1.15
          econ_min_open: 0.12

        expression: |
          ((Cooling_Coil_Entering_Air_Temperature_Sensor -
Cooling_Coil_Leaving_Air_Temperature_Sensor) >
np.sqrt(enter_tol**2 + leave_tol**2)) & (((Heating_Valve_Command >
0) & (Cooling_Valve_Command == 0) & (Damper_Position_Command <=
econ_min_open)) | ((Heating_Valve_Command == 0) &
(Cooling_Valve_Command == 0) & (Damper_Position_Command >
econ_min_open)))
```

44.3.13 Rule M — Heating coil delta-T when inactive

Temperature rise across heating coil when it should be off. Indicates leaking HW valve. *Adapted from GL36 AFDD guidance.*

```
        name: htg_coil_rise_when_off
        description: Temperature rise across heating coil when it should
be off (GL36-inspired)
        type: expression
        flag: rule_m_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Heating_Coil_Entering_Air_Temperature_Sensor:
            brick: Heating_Coil_Entering_Air_Temperature_Sensor
          Heating_Coil_Leaving_Air_Temperature_Sensor:
            brick: Heating_Coil_Leaving_Air_Temperature_Sensor
          Heating_Valve_Command:
            brick: Valve_Command
          Cooling_Valve_Command:
            brick: Valve_Command
          Damper_Position_Command:
            brick: Damper_Position_Command

        params:
          enter_tol: 1.15
          leave_tol: 1.15
          fan_delta_t: 0.55
          econ_min_open: 0.12

        expression: |
```

```
          ((Heating_Coil_Leaving_Air_Temperature_Sensor -
Heating_Coil_Entering_Air_Temperature_Sensor) >
np.sqrt(enter_tol**2 + leave_tol**2) + fan_delta_t) &
(((Heating_Valve_Command == 0) & (Cooling_Valve_Command == 0) &
(Damper_Position_Command > econ_min_open)) |
((Heating_Valve_Command == 0) & (Cooling_Valve_Command > 0) &
(Damper_Position_Command > 0.9)) | ((Heating_Valve_Command == 0) &
(Cooling_Valve_Command > 0) & (Damper_Position_Command <=
econ_min_open)))
```

Heat exchanger effectiveness — see rule types (erv_efficiency).

44.4 Central plant

44.4.1 Differential pressure at max pump speed

Variable-speed pump cannot meet differential pressure setpoint at full speed. Indicates piping issues, undersized pump, or blocked strainers.

```yaml
        name: dp_below_sp_pump_max
        description: Differential pressure below setpoint with pump at
full speed
        type: expression
        flag: dp_pump_flag

        inputs:
          Differential_Pressure_Sensor:
            brick: Differential_Pressure_Sensor
          Differential_Pressure_Setpoint:
            brick: Differential_Pressure_Setpoint
          Pump_Speed_Command:
            brick: Pump_Speed_Command

        params:
          dp_margin: 2.2
          pmp_hi_frac: 0.93
          pmp_near_hi: 0.06

        expression: |
          (Differential_Pressure_Sensor < Differential_Pressure_Setpoint -
dp_margin) & (Pump_Speed_Command >= pmp_hi_frac - pmp_near_hi)
```

44.4.2 Plant flow high at max pump

Flow unusually high with pump at high speed. Suggests short circuit or flow meter error.

```
        name: flow_high_pump_max
        description: Water flow unusually high with pump at full speed
        type: expression
        flag: flow_high_flag

        inputs:
          Water_Flow_Sensor:
            brick: Water_Flow_Sensor
          Pump_Speed_Command:
            brick: Pump_Speed_Command

        params:
          flow_hi_limit: 1100.0
          pmp_hi_frac: 0.93
          pmp_near_hi: 0.06

        expression: |
          (Water_Flow_Sensor > flow_hi_limit) & (Pump_Speed_Command >=
pmp_hi_frac - pmp_near_hi)
```

44.4.3 Plant supply temp outside deadband

CHW supply temperature outside deadband while pump runs.
Requires coil entering/leaving or plant supply temp sensors.

```
        name: plant_supply_temp_deadband
        description: Supply water temp outside deadband during pump
operation
        type: expression
        flag: chw_temp_fault

        inputs:
          Chilled_Water_Supply_Temperature_Sensor:
            brick: Chilled_Water_Supply_Temperature_Sensor
          Chilled_Water_Supply_Temperature_Setpoint:
            brick: Chilled_Water_Supply_Temperature_Setpoint
          Pump_Speed_Command:
            brick: Pump_Speed_Command

        params:
          sp_band: 2.2

        expression: |
          (Pump_Speed_Command > 0.01) &
((Chilled_Water_Supply_Temperature_Sensor <
Chilled_Water_Supply_Temperature_Setpoint - sp_band) |
(Chilled_Water_Supply_Temperature_Sensor >
Chilled_Water_Supply_Temperature_Setpoint + sp_band))
```

44.4.4 Chiller runtime over daily limit

Chiller running beyond a daily threshold (e.g. 23 hours). Often indicates over-cooling or schedules that bypass lockout. Use a rolling window sized for your data interval (e.g. 5-min data → 276 samples ≈ 23 h).

```yaml
        name: chiller_excessive_runtime
        description: Chiller runtime exceeds daily threshold (rolling
window)
        type: expression
        flag: chiller_runtime_fault

        inputs:
          Chiller_Status:
            brick: Chiller_Status

        params:
          # For 5-min data: 23 hours ≈ 276 samples; max_runtime = count of
"on" samples in window
          rolling_samples: 276
          max_runtime_samples: 264

        expression: |
          Chiller_Status.rolling(window=rolling_samples).sum() >
max_runtime_samples
```

Note: Adjust `rolling_samples` and `max_runtime_samples` for your data interval. For 5-min data, 264 samples ≈ 22 hours.

44.5 Heat pumps

44.5.1 Discharge cold when heating

Flags when discharge air temperature is below a minimum (e.g. 80°F) while the supply fan is running. Indicates the heat pump is not heating effectively—possible issues with compressor, refrigerant, or reversing valve. Tunable via `min_discharge_temp`. Logic: if zone temp < 69°F (heating mode), the discharge should be warm; a cold discharge with the fan on and a cold zone indicates the heat pump is failing to heat.

```yaml
        name: hp_discharge_cold_when_heating
        description: Discharge air below minimum when fan on and zone is
cold (heating mode)
        type: expression
```

```
        flag: hp_discharge_cold_flag
        equipment_type: [Heat_Pump]

        inputs:
          Supply_Air_Temperature_Sensor:
            brick: Supply_Air_Temperature_Sensor
          Zone_Temperature_Sensor:
            brick: Zone_Temperature_Sensor
          Supply_Fan_Status:
            brick: Supply_Fan_Status

        params:
          min_discharge_temp: 85
          zone_cold_threshold: 69.0
          fan_on_threshold: 0.01

        expression: |
          (Supply_Fan_Status > fan_on_threshold) &
(Zone_Temperature_Sensor < zone_cold_threshold) &
(Supply_Air_Temperature_Sensor < min_discharge_temp)
```

44.6 VAV zones

44.6.1 Excessive heating during warm weather

Reheat valve open when outdoor air is warm. Suggests over-cooling or setpoint issues.

```
        name: zone_reheat_warm_ambient
        description: Heating valve open when OAT is high
        type: expression
        flag: excessive_heating_flag
        equipment_type: [VAV]

        inputs:
          Outside_Air_Temperature_Sensor:
            brick: Outside_Air_Temperature_Sensor
          Reheat_Valve_Command:
            brick: Valve_Command

        params:
          t_amb_cutoff: 78.0
          reheat_open_min: 0.52

        expression: |
          (Outside_Air_Temperature_Sensor > t_amb_cutoff) &
(Reheat_Valve_Command > reheat_open_min)
```

44.6.2 Damper or valve at full open

Damper or reheat valve consistently at full open. Indicates override, undersized equipment, or control fault.

```
        name: zone_damper_valve_full_open
        description: Damper or valve at full open for extended period
        type: expression
        flag: damper_100_flag
        equipment_type: [VAV]

        inputs:
          Damper_Position_Command:
            brick: Damper_Position_Command

        params:
          full_open_pct: 97.5
          roll_samples: 105

        expression: |
          (Damper_Position_Command > full_open_pct) &
(Damper_Position_Command.rolling(roll_samples).min() >
full_open_pct)
```

44.6.3 Zone and IAQ bounds

For CO2 and zone temperature out-of-range checks, use the bounds rule type — `co2_bounds` and `zone_temp_bounds` examples.

44.7 Opportunistic rules (economizer & ventilation)

44.7.1 Economizing when outdoor conditions are unfavorable

Economizer active when outdoor air is too warm or humid. Use OAT (or enthalpy if available) vs. threshold.

```
        name: econ_active_warm_ambient
        description: OA damper open when outdoor conditions do not favor
economizing
        type: expression
        flag: econ_when_shouldnt_flag
        equipment_type: [AHU, VAV_AHU]
```

```
        inputs:
          Outside_Air_Temperature_Sensor:
            brick: Outside_Air_Temperature_Sensor
          Damper_Position_Command:
            brick: Damper_Position_Command

        params:
          t_amb_econ_cutoff: 63.0
          dpr_econ_min: 0.42

        expression: |
          (Outside_Air_Temperature_Sensor > t_amb_econ_cutoff) &
(Damper_Position_Command > dpr_econ_min)
```

44.7.2 Mechanical cooling when econ could suffice

Cooling valve open when outdoor air is cool enough for economizer. Opportunity to reduce mechanical cooling.

```
        name: mech_cool_when_econ_available
        description: Mechanical cooling active when economizing could
suffice
        type: expression
        flag: cooling_when_econ_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Outside_Air_Temperature_Sensor:
            brick: Outside_Air_Temperature_Sensor
          Damper_Position_Command:
            brick: Damper_Position_Command
          Valve_Command:
            brick: Valve_Command

        params:
          t_amb_econ_cutoff: 63.0
          dpr_not_econ_max: 0.32

        expression: |
          (Outside_Air_Temperature_Sensor < t_amb_econ_cutoff) &
(Damper_Position_Command < dpr_not_econ_max) & (Valve_Command >
0.01)
```

44.7.3 Low ventilation (estimated OA fraction)

For units without airflow meters, estimate OA fraction from OAT, MAT, RAT. Flag when below minimum design OA.

```
        name: low_oa_fraction_estimated
        description: Estimated OA fraction below minimum (OAT, MAT, RAT
```

```
method)
        type: expression
        flag: low_vent_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Mixed_Air_Temperature_Sensor:
            brick: Mixed_Air_Temperature_Sensor
          Return_Air_Temperature_Sensor:
            brick: Return_Air_Temperature_Sensor
          Outside_Air_Temperature_Sensor:
            brick: Outside_Air_Temperature_Sensor
          Supply_Fan_Speed_Command:
            brick: Supply_Fan_Speed_Command

        params:
          oa_min_pct: 21.0
          t_rat_oat_min_gap: 2.2

        expression: |
          (Supply_Fan_Speed_Command > 0.01) &
(np.abs(Return_Air_Temperature_Sensor -
Outside_Air_Temperature_Sensor) > t_rat_oat_min_gap) &
(((Mixed_Air_Temperature_Sensor - Return_Air_Temperature_Sensor) /
(Outside_Air_Temperature_Sensor - Return_Air_Temperature_Sensor) *
100) < oa_min_pct)
```

Note: Guard against division-by-zero when outdoor and return temps are close.

44.7.4 Preheat over-conditioning

Preheat coil leaving temp higher than needed (e.g. above OAT when OAT > SAT SP, or above SAT SP when OAT < SAT SP). Indicates wasted heating energy.

```
        name: preheat_excess_temp
        description: Preheat coil leaving temp above required level
        type: expression
        flag: preheat_waste_flag
        equipment_type: [AHU, VAV_AHU]

        inputs:
          Preheat_Coil_Leaving_Air_Temperature_Sensor:
            brick: Preheat_Coil_Leaving_Air_Temperature_Sensor
          Supply_Air_Temperature_Setpoint:
            brick: Supply_Air_Temperature_Setpoint
          Outside_Air_Temperature_Sensor:
            brick: Outside_Air_Temperature_Sensor
          Valve_Command:
```

```
      brick: Valve_Command

    params:
      excess_tol: 2.2

    expression: |
      (Valve_Command > 0.01) & (((Outside_Air_Temperature_Sensor >
Supply_Air_Temperature_Setpoint) &
(Preheat_Coil_Leaving_Air_Temperature_Sensor -
Outside_Air_Temperature_Sensor > excess_tol)) |
((Outside_Air_Temperature_Sensor <
Supply_Air_Temperature_Setpoint) &
(Preheat_Coil_Leaving_Air_Temperature_Sensor -
Supply_Air_Temperature_Setpoint > excess_tol)))
```

44.7.5 Blended air damper deviation

Expected MAT (from OAT, RAT, damper positions) differs from measured MAT. Indicates damper leakage or faulty mixing.

Requires airflow or damper position data; logic is more involved. A simplified version compares MAT to a weighted blend of OAT and RAT when dampers suggest significant OA.

44.8 Weather station

weather_temp_stuck (flatline) — see flatline rule type.

44.8.1 Unrealistic temperature spike

Temperature change between consecutive readings exceeds physical limit.

```
    name: weather_temp_spike
    description: Unrealistic temperature change between readings
    type: expression
    flag: fault_temp_spike

    inputs:
      Outside_Air_Temperature_Sensor:
        brick: Outside_Air_Temperature_Sensor

    params:
      spike_limit: 16.0
```

```
        expression: |
          Outside_Air_Temperature_Sensor.diff().abs() > spike_limit
```

44.8.2 RH bounds

For relative humidity out-of-range, use the bounds rule type — `rh_bounds` example.

44.8.3 Wind gust vs sustained

```
        name: weather_gust_lt_wind
        description: Wind gust reported lower than sustained wind
        type: expression
        flag: fault_gust_lt_wind

        inputs:
          Wind_Gust_Speed_Sensor:
            brick: Wind_Gust_Speed_Sensor
          Wind_Speed_Sensor:
            brick: Wind_Speed_Sensor

        expression: |
          Wind_Gust_Speed_Sensor.notna() & Wind_Speed_Sensor.notna() &
(Wind_Gust_Speed_Sensor < Wind_Speed_Sensor)
```

44.9 Sensor validation (bounds & flatline)

Use the bounds and flatline rule types for generic sensor checks. Typical bounds:

Sensor type	Min	Max
Zone temp	40	100
Supply air temp	40	150
Air pressure (inH2O)	-5	10
RH	0	100
Chilled water temp	35	100
Hot water temp	50	212
Condenser water	50	110
CO2 (ppm)	400	2000

44.10 Follow-up (RFCs): multi-ontology selectors & cookbook matrix

Multi-ontology rule YAML (e.g. optional `selectors.brick` / `.haystack` / `.dbo` / `.s223` under each input) is a **separate RFC** — it would feed the same **`column_map`** / resolver layer while the **full AFDD stack stays Brick-only by default** for v1. Track design discussion on GitHub **#122** and follow-on issues.

Cookbook matrix / generator: When a selector schema is stable, we can add a **generated appendix** (logical input × Brick × Haystack × DBO × 223P notes) from a single CSV/YAML source so the long cookbook does not drift. Until then, see **`examples/column_map_resolver_workshop/`** for manifest-based illustrations.

Next: Fake fault schedule / flatline monitoring

45 Test bench rule catalog

46 Test bench rule catalog

Open-FDD ships **two** default rules under `stack/rules/` (`sensor_bounds.yaml`, `sensor_flatline.yaml`). For **extended HVAC, chiller, heat-pump, and weather expression rules** used in lab automation and cookbook examples, the repository also keeps a **reference set** under **`openclaw/bench/rules_reference/`** (not loaded until you copy or upload them).

Each file below links to the copy on **GitHub** (`master` branch) so the published docs site stays in sync with the repo.

46.1 Reference YAML files

File	GitHub
`ahu_fc1.yaml`	link
`ahu_fc2.yaml`	link
`ahu_fc3.yaml`	link

File	GitHub
`ahu_fc4.yaml`	link
`ahu_fc5.yaml`	link
`ahu_fc6.yaml`	link
`ahu_fc7.yaml`	link
`ahu_fc8.yaml`	link
`ahu_fc9.yaml`	link
`ahu_fc10.yaml`	link
`ahu_fc11.yaml`	link
`ahu_fc12.yaml`	link
`ahu_fc13.yaml`	link
`ahu_fc14.yaml`	link
`ahu_fc15.yaml`	link
`ahu_fc16.yaml`	link
`ahu_rule_a.yaml`	link
`chiller_flow_fc2.yaml`	link
`chiller_pump_fc1.yaml`	link
`hp_discharge_cold_when_heating.yaml`	link
`sensor_bounds.yaml`	link
`sensor_flatline.yaml`	link
`weather_gust_lt_wind.yaml`	link
`weather_rh_out_of_range.yaml`	link
`weather_temp_spike.yaml`	link
`weather_temp_stuck.yaml`	link

46.2 Related docs

- Fault rules overview — where live rules live (`stack/rules`) and how to upload or sync.
- Expression rule cookbook — AHU, chiller, weather patterns that match many of these files.
- OpenClaw lab README — bench layout including `rules_reference/` and E2E automation.

The former **open-fdd-automated-testing** repo is superseded by the **`openclaw/`** tree in this repository; do not link automation to a separate rules tree.

47 YAML rules → Pandas (under the hood)

48 YAML rules → Pandas DataFrames (under the hood)

This note is for engineers who want to see **exactly** how Open-FDD turns **rule YAML on disk** into **pandas operations** on **time-series DataFrames**—and where Brick TTL fits in. It complements Fault rules overview and standalone CSV / pandas.

48.1 1. YAML never becomes "one giant DataFrame of rules"

Rule files are **configuration**, not tabular data. Open-FDD:

1. Reads each `*.yaml` in `rules_dir` with PyYAML (`yaml.safe_load`) into a **Python `dict`** per file (`open_fdd.engine.runner.load_rule` / `load_rules_from_dir`).
2. Keeps those dicts in a **`list[dict]`** inside `RuleRunner` (`RuleRunner(rules=...)` or `RuleRunner(rules_path=...)`).
3. At evaluation time, walks that list and, for each rule, computes a **boolean mask** (`pandas.Series`) aligned to the **sensor DataFrame's index**, then writes a **new column** for the fault flag (e.g. `my_rule_flag`).

So: **rules** = **list of dicts in memory**; **data** = **one wide-ish DataFrame** of timestamps and point columns.

48.2 2. From Postgres rows to a site/equipment DataFrame

The continuous loop (`open_fdd.platform.loop.run_fdd_loop`) loads telemetry with SQL, then reshapes it for pandas:

1. **Query** `timeseries_readings` joined to `points` for the time window (`load_timeseries_for_site` / `load_timeseries_for_equipment`).
2. **Build a long table**: `pd.DataFrame(rows)` with columns like `ts`, `external_id`, `value`.
3. **Pivot to wide**: `df.pivot_table(index="ts", columns="external_id", values="value")` so each point is a column (one column per `external_id`).
4. **Rename columns** using a **column map** derived from Brick TTL (`resolve_from_ttl`): external refs or labels become the names rules expect (Brick-class-driven mapping).
5. **Add** `timestamp = pd.to_datetime(df["ts"])` for time-based checks.

Under the hood, pandas is doing **grouped aggregation in the pivot** (duplicate `(ts, external_id)` would aggregate), then **aligning** all series to a common `DatetimeIndex` (implicit via the pivot index).

48.3 3. What `RuleRunner.run` does to that DataFrame

`RuleRunner.run` (`open_fdd.engine.runner`):

1. **`result = df.copy()`** — rules never mutate the caller's frame in place (callers can still hold a reference to the original).
2. For **each** rule dict:
   - Derives **`flag_name`** from `flag` or `{name}_flag`.
   - Calls **`_evaluate_rule`** → returns a **boolean `Series`** (fault mask) aligned to `result.index`.
   - Optionally applies a **rolling window** on the mask: `mask.astype(int).rolling(window=rw).sum() >= rw` so a fault must persist for N consecutive samples.
   - Assigns **`result[flag_name] = ...`** as integer 0/1.

So each rule adds **one column** of flags; the frame grows **width-wise**, not row-wise.

48.4 4. How a YAML rule dict becomes pandas expressions

Inside `_evaluate_rule`, Open-FDD branches on `rule["type"]` (e.g. `bounds`, `flatline`, `expression`, …):

- **`column_map` resolution**: For each logical input key, the runner picks a **DataFrame column name**. If the YAML input has a **`brick`** class, the global `column_map` from TTL is consulted first (`brick` → column label), so the **same YAML** can run against different exports as long as TTL maps Brick classes to the right columns.
- **Bounds / thresholds**: Typical pattern is `(series < low) | (series > high)` using **vectorized** comparisons — these are **numpy ufuncs** under pandas, no Python `for` over rows.
- **Expressions**: String expressions may be evaluated in a **restricted eval** context (`open_fdd.engine.checks.check_expression`) with named series bound to `result[col]` — still vectorized.

If a required column is missing, the runner either **raises** or **skips** the rule (`skip_missing_columns=True`), depending on the call site.

48.5 5. Hot reload: YAML vs DataFrame lifetime

`run_fdd_loop` calls **`load_rules_from_dir(rules_path)` every run**, then filters by equipment types from TTL, then **`RuleRunner(rules=rules)`**. There is **no** long-lived compiled rule object on disk—editing YAML affects the **next** scheduled run (or the next manual `POST /run-fdd`). The **DataFrame** exists only for the duration of that run's Python call stack (load → run → persist results).

48.6 6. Mental model checklist

Artifact	In-memory shape	pandas role
Rule YAML files	N/A (on disk)	None until loaded
Loaded rules	`list[dict]`	None
	`DataFrame` (time × points)	

Artifact	In-memory shape	pandas role
Telemetry window		pivot, datetime index, column rename
Rule output	Same index, +flag columns	vectorized masks, optional `rolling`
`fault_results` / DB	Written from row-wise `FDDResult`	After flags are computed

For a **minimal** example of rules + CSV (no DB), see standalone CSV / pandas and unit tests in `open_fdd/tests/engine/test_runner.py`.

49 Fault rules for HVAC

50 Fault rules for HVAC

FDD fault rule types, YAML format, and expression cookbook for HVAC and building systems. Rules are **100% Brick-model driven**: inputs reference Brick classes only; column mapping comes from the Brick TTL and Brick external timeseries references (timeseries storage), which Open-FDD embraces at the heart of FDD.

Page	Description
Overview	Rule types, `stack/rules`, hot reload, Brick inputs
Test bench rule catalog	Links to reference YAML under `openclaw/bench/rules_reference/` (AHU FC, chillers, weather, etc.)

51 Cloud export example

52 Cloud export example

Open-FDD runs **inside the building, behind the firewall**. It does not push data to the cloud itself. Cloud-based FDD providers, MSI (Monitoring, Savings, and Intelligence) vendors, and commissioning or IoT contractors run their **own** export process: a small script or edge gateway on the building or OT network that **pulls** from the Open-FDD API over the LAN and then sends that data to their cloud or analytics platform. See Behind the firewall; cloud export is vendor-led on the docs home.

This page describes the **cloud export example** in the repo: a minimal Python script that demonstrates how vendor X, Y, or Z could use the Open-FDD API to get fault and timeseries data and use it as a starting point for their cloud pipeline.

52.1 What the example does

The script `examples/cloud_export.py` calls standard Open-FDD download and analytics endpoints and prints the results. In a real integration you would replace the print steps with your own logic: POST to your REST API, write to S3, push to Azure IoT Hub or AWS IoT, send to SkySpark or another analytics platform, etc.

Step	Endpoint	Purpose
1	`GET /download/faults?format=json`	Fault results for programmatic ingestion (JSON)
2	`GET /download/faults?format=csv`	Fault CSV (Excel-friendly)
3	`GET /analytics/motor-runtime`	Motor runtime (data-model driven)
4	`GET /download/csv`	Timeseries wide-format CSV
5		Fault counts by fault_id

Step	Endpoint	Purpose
	`GET /analytics/fault-summary`	

All requests use date range and optional `site_id`; the API docs at `/docs` list full parameters.

52.2 Run the example

Prerequisites: Open-FDD API running (e.g. `http://localhost:8000`), `pip install httpx`.

```
# From repo root
python examples/cloud_export.py
python examples/cloud_export.py --site default --days 14
API_BASE=http://your-openfdd-host:8000 python examples/cloud_export.py
```

Docker: See `examples/cloud_export/README.md` for building and running the example in a container (e.g. on a vendor edge device that reaches the Open-FDD API over the LAN).

52.3 Use it as your starting point

1. **Clone or copy** `examples/cloud_export.py` into your integration codebase.
2. **Keep** the HTTP calls to `/download/faults`, `/download/csv`, and the analytics endpoints; adjust `API_BASE` (or env) to point at the customer's Open-FDD instance.
3. **Replace** the `print()` / local handling with your cloud send: REST POST, message queue, blob storage, or IoT hub. Add auth (e.g. API key or TLS client cert) if you expose the puller to a wider network.
4. **Schedule** the script (cron, systemd timer, or container restart policy) so you poll the Open-FDD API at the interval that fits your product (e.g. hourly or daily fault sync).

Open-FDD does not initiate outbound cloud connections or manage your data transmission; the example shows how your process can pull data from Open-FDD and then send it to your cloud for deeper insights.

53 Context and recordkeeping

54 Context and recordkeeping

Open-FDD context should be durable, visible, and reusable by both humans and agents.

54.1 Rule

If context affects repeatable operations, store it in versioned docs instead of leaving it in chat-only memory.

54.2 Good context to commit

- Verification strategy and sweep logic.
- BACnet/model assumptions that affect diagnostics.
- Operator playbook updates and recurring workflow improvements.
- Cross-check guidance for frontend/API parity and model-based telemetry.

54.3 Context anti-patterns

- One-machine-only tribal notes with no repo trace.
- Secrets in Markdown.
- Raw transcript dumps without distillation.

55 Operational states

56 Operational states

Open-FDD operations should explicitly identify environment mode before asserting correctness.

56.1 States

- `TEST_BENCH`: deterministic fake-device conditions or lab-only setup.
- `LIVE_HVAC`: production building telemetry and real-world operating behavior.

56.2 Why this matters

The same observed value can be expected in bench mode and suspicious in live mode. Mode-aware interpretation avoids false positives and noisy operations.

57 Big picture gaps and product suggestions

58 Big picture gaps and product suggestions

This is a running list of higher-level product, workflow, and documentation gaps discovered while operating Open-FDD and its automated testing stack.

The goal is to capture the things that matter beyond the immediate bug or test run.

58.1 Current themes

58.1.1 1. AI-assisted data modeling needs a stronger review-before-import flow

The current workflow makes it easy to jump from discovered BACnet points to imported semantic changes.

Suggested improvement:

- stage proposed site/equipment/point modeling changes

- let the user review them clearly before import
- make the before/after delta obvious

58.1.2 2. Graph reset behavior is easy to misunderstand

`POST /data-model/reset` does **not** mean “wipe everything.” It means:

- clear BACnet graph/orphans
- repopulate Brick from DB-only

Suggested improvement:

- clarify this distinction more aggressively in UI/docs
- keep repeating that full empty-state requires deleting all sites via CRUD first

58.1.3 3. Rule-aware minimal polling is a high-impact lever

The testing/modeling flow should emphasize that only points needed by active rules should be polled.

This reduces unnecessary BACnet traffic and makes Open-FDD deployments more realistic for live HVAC systems.

Suggested improvement:

- add rule-aware polling recommendations in product/docs
- highlight minimal polling sets for active rules

58.1.4 4. The gap between discovery and FDD-ready modeling is still wide

Discovery gets BACnet objects into the graph, but that is still far from:

- Brick semantics
- equipment relationships
- rule applicability
- optimization readiness

Suggested improvement:

- describe maturity stages more explicitly:
  - discovered
  - modeled

    - FDD-ready
    - operations-ready

58.1.5 5. Overnight testing still needs stronger generated evidence

The current overnight setup is much better than before, but it should mature into a real evidence pipeline:

- BACnet source evidence
- SPARQL/model evidence
- YAML rule evidence
- fault-result evidence
- docs/link review evidence
- container-log correlation

Suggested improvement:

- generate durable overnight reports by default (see `openclaw/reports/` templates in the repo)

58.1.6 6. Container log viewing should become part of the testing story

The container-log feature is not just an admin convenience. It should support:

- frontend/backend mismatch diagnosis
- scraper troubleshooting
- FDD loop troubleshooting
- overnight evidence correlation

Suggested improvement:

- make container-log review part of normal dev-testing and docs

58.1.7 7. Open-FDD needs a stronger live-HVAC operations mental model

The platform already contains pieces for:

- discovery
- modeling
- rules
- faults

- charts
- logs
- weather

But the product/docs should frame those around the real user contexts:

- controls engineer
- building operator / facilities manager
- AI agent assisting with diagnostics and optimization

58.2 Security / hardening context to remember

Relevant existing issue on the main Open-FDD repo:

- **#73** — Stack security hardening (DB, Caddy, secrets) — https://github.com/bbartling/open-fdd/issues/73

This should remain in the background context for future deployment and production-readiness work.

Also relevant:

- **#44** — `Test Caddy. Integrate and Define Secure Coding Best Practices` — https://github.com/bbartling/open-fdd/issues/44

58.3 Why this doc exists

A lot of the most important insight is not just “what failed,” but “what is still missing from the big picture.”

This file is meant to preserve those bigger observations so future clones, future agents, and future humans can build from them instead of rediscovering them from scratch.

59 Concepts

60 Concepts

Conceptual guides and worked examples for how Open-FDD fits into larger workflows.

Page	Description
Cloud export example	How a cloud vendor or MSI can pull fault and timeseries data from the Open-FDD API and send it to their cloud for deeper insights. Uses the `examples/cloud_export` script as a starting point.
Context and recordkeeping	Durable context policy: what belongs in versioned docs vs private local memory.
Operational states	Mode-aware interpretation between test bench and live HVAC environments.
Big picture gaps and product suggestions	Higher-level product and workflow gaps observed in the field.

See also Behind the firewall; cloud export is vendor-led on the docs home.

61 MQTT integration (optional)

62 MQTT integration (optional) — Open-FDD + diy-bacnet-server

Open-FDD does **not** require MQTT for core FDD, BACnet scraping, or the web UI. MQTT is **optional** and aimed at **future** edge/automation patterns and **generic** brokers (typically **Mosquitto**), not a specific cloud vendor.

62.1 What the stack can do today

1. **Mosquitto (compose profile)**
   Run `./scripts/bootstrap.sh --with-mqtt-bridge` to start a broker on **`localhost:1883`** (see Getting started and Quick reference).

2. **BACnet2MQTT (diy-bacnet-server)**
   When **`BACNET2MQTT_ENABLED=true`** and **`MQTT_BROKER_URL`** point at that broker, **diy-bacnet-server** publishes per-point state under **`MQTT_BASE_TOPIC`** (default `bacnet2mqtt`) and can publish Home Assistant discovery under **`HA_DISCOVERY_TOPIC`**. This is documented in the **diy-bacnet-server repo** (README and `HOME_ASSISTANT_MQTT_CHEATSHEET.md`).

3. **MQTT RPC gateway (experimental, diy-bacnet-server)**
   When **`MQTT_RPC_GATEWAY_ENABLED=true`**, the same gateway process can subscribe to **`{MQTT_RPC_TOPIC_PREFIX}/cmd`** and publish structured acks on **`.../ack`**, using the **same method names** as HTTP JSON-RPC (`server_hello`, `client_whois_range`, `client_read_property`, etc.). Optional **telemetry** topics advertise supported methods and periodic metadata.
   Configure via **`stack/.env`** (variables are passed through **`stack/docker-compose.yml`** to the **bacnet-server** service). See the upstream **MQTT RPC gateway** section for topic layout and security notes (TLS, ACLs).

62.2 Open-FDD product scope

- Core data path remains **HTTP JSON-RPC** from the **BACnet scraper** and API to **diy-bacnet-server**.
- MQTT features are **additive**: monitoring via BACnet2MQTT, command/ack experimentation via the RPC gateway, without replacing the database-backed scraper or auth model (**`OFDD_BACNET_SERVER_API_KEY`** → **`BACNET_RPC_API_KEY`** on the gateway).

62.3 Related docs

- Getting started — `--with-mqtt-bridge`
- Quick reference — broker port and status checks
- BACnet overview — gateway and scraper roles

63 Open-Claw Integration

64 Open-Claw Integration (external worker)

Open-FDD does not embed or run an LLM. Instead, you run Open-Claw (or any OpenAI-compatible agent) externally, and use Open-FDD's REST API as the "data + tool" layer. Open-Claw can be local (same host) or hosted remotely; your agent just needs network access to the Open-FDD API.

This page shows how to:

1. Fetch platform documentation as model context (`GET /model-context/docs`)
2. Discover HTTP mappings programmatically (`GET /mcp/manifest`)
3. Automate Brick tagging (`GET /data-model/export` → LLM → `PUT /data-model/import`)
4. Tune FDD faults by updating rule YAML files (`/rules/*`) and syncing definitions

Security note: when **auth is required** (`OFDD_API_KEY` and/or app-user login), API routes (including `GET /model-context/docs`) accept **`Authorization: Bearer`** with either the **API key** or a **JWT access token** issued from **`POST /auth/login`**. External HTTP agents normally use the **API key**; browsers use login + JWT. `/auth/*` is exempt from the middleware so login/refresh/logout work.

64.1 1) Model context endpoint (`GET /model-context/docs`)

External agents should fetch Open-FDD documentation context from:

- `GET /model-context/docs`

Control the output size and retrieval:

- `mode=excerpt` (default): truncated excerpt of `pdf/open-fdd-docs.txt`
- `mode=full`: entire doc file

- `mode=slice&offset=...`: a substring (character offset + `max_chars`)
- `query=...`: keyword-retrieved relevant doc sections (simple lexical matching)

If your agent's context window is running out, re-fetch context using `query=...` (or `mode=slice`) as needed.

64.2 1b) MCP-style discovery (`GET /mcp/manifest`)

Open-FDD exposes a **small JSON manifest** (not a full MCP stdio/SSE server) at:

- `GET /mcp/manifest`

Use it to wire external workers or a **separate** MCP server you maintain: the manifest lists **resource URIs** (e.g. `openfdd://docs`) and **tool-shaped** entries with `http.method` + `http.path` for:

- Fetching docs (`/model-context/docs`)
- Data-model export/import (`/data-model/export`, `/data-model/import`)
- Capability discovery (`/capabilities`)

Your agent implementation should perform the described HTTP requests with the same **Bearer** token as the rest of the API when auth is enabled.

64.3 1c) Optional MCP RAG sidecar (`--with-mcp-rag`)

Open-FDD can also run an optional MCP-style RAG sidecar built from canonical docs:

- start with `./scripts/bootstrap.sh --with-mcp-rag`
- service: `http://localhost:8090`
- manifest: `GET /manifest`

This sidecar provides retrieval-focused tools over derived index artifacts from `docs/` and `pdf/open-fdd-docs.txt`. By default it is read-oriented; guarded API action tools exist but are disabled unless explicitly enabled with `OFDD_MCP_ENABLE_ACTION_TOOLS=true`.

64.4 1d) Mode-aware orchestration with bootstrap

OpenClaw can orchestrate partial deployments through bootstrap modes:

- `./scripts/bootstrap.sh --mode full` (default)
- `./scripts/bootstrap.sh --mode collector`
- `./scripts/bootstrap.sh --mode model`
- `./scripts/bootstrap.sh --mode engine`

This enables targeted runs (for example model-only SPARQL work or engine-only rule validation) without always launching the full stack.

64.5 1e) OpenClaw on a different machine than Open-FDD (split setup)

OpenClaw does **not** have to run on the same host as Docker / the git clone. A common pattern is **Open-FDD on a Linux server** and **OpenClaw on a Windows (or other) workstation** on the same **test bench LAN**.

How OpenClaw-style tests talk to Open-FDD (HTTP)

Bench and helper scripts treat Open-FDD like any other REST client: they read **`OFDD_API_KEY`** from the environment (often after loading a copy of **`stack/.env`** from the Linux host) and send **`Authorization: Bearer <OFDD_API_KEY>`** on each request. They do **not** open a browser, call **`/auth/login`**, or manage HttpOnly cookies — that flow is for the React dashboard only. If the stack has **app-user** auth enabled **without** an **`OFDD_API_KEY`**, those scripts would need a JWT instead (cookie + login); **for automation, keep `OFDD_API_KEY` in `stack/.env` and mirror it into the Windows environment** (or a small `.env` you load before running Python). Product details of the bench live under **`openclaw/`** in the repo; this page only describes how that pattern fits Open-FDD.

What the external agent needs

1. **Shell access to the Linux host** (SSH) if it will run `./scripts/bootstrap.sh`, `pytest`, or edit files in the clone — point OpenClaw's workspace at that path or mount it. If the agent only drives **HTTP tools**, a full clone on Windows is optional.
2. **Reachable base URL** — from the Windows machine, **`http://localhost:8000` does not reach the Linux server.** Use the host's **LAN IP or DNS name**:
   - **Direct API:** `http://<open-fdd-host>:8000/...` (paths as in Swagger, e.g. `/sites`, `/data-model/sparql`).
   - **Through Caddy (port 80):** `http://<open-fdd-host>/api/...` for REST paths the UI uses (Caddy strips the `/api` prefix before proxying). Auth endpoints: `http://<open-fdd-host>/auth/...`. Bench scripts are usually simplest against **`:8000`** so paths match `/sites` with no prefix. Use **port-forward** or **SSH tunnel** only if you deliberately map remote 8000 to local `localhost`.
3. **Bearer token for REST** — set **`OFDD_API_KEY`** on the bench to the same value as in **`stack/.env` on the Open-FDD host** (see Security — authentication). Do not commit that file; copy the key out of band. DB passwords and other secrets can stay on the server.
4. **Browser UI from Windows (optional)** — open **`http://<open-fdd-host>/`** (Caddy) or **`http://<open-fdd-host>:5173`** (direct frontend). Sign in at **`/login`** when dashboard user auth is configured; the app uses **`VITE_API_BASE=/api`** behind Caddy so API calls stay same-origin. This is **independent** of the Bearer header OpenClaw uses for REST.

File handoff (`openclaw/issues_log.md`) — `HANDOFF_PROTOCOL.md` assumes both Cursor and OpenClaw can read the **same repo tree**. With a split setup, use **git push/pull** (or SSH to one canonical clone) so the lab trail stays in sync.

64.6 2) Automated Brick tagging loop

For automated tagging, use the same underlying workflow as manual export/import:

1. Export the current model:
   - `GET /data-model/export` (optionally `?site_id=...`)
2. Get documentation context for the LLM:
   - `GET /model-context/docs?query=data-model+import` (or a broader query)

3. Ask Open-Claw to return **import JSON**:
    - Output must match the Open-FDD import schema: top-level keys `points` and optional `equipment`
4. Validate and import:
    - `PUT /data-model/import` with the validated JSON

Practical prompt pattern for retries:

- If `PUT /data-model/import` returns an error, include the error text in the next LLM prompt and retry (prompt chaining).

64.7 3) "Tune faults" worker loop (rule YAML)

Open-FDD's fault definitions are driven by YAML rule files stored under your configured `rules_dir`.

An Open-Claw worker can update those rules via:

- `GET /rules` (list YAML files and the resolved `rules_dir` path)
- `POST /rules` (upload/overwrite a rule YAML file)
- `POST /rules/sync-definitions` (immediately refresh `fault_definitions` from `rules_dir`)
- `DELETE /rules/{filename}` (delete a YAML file)

Then run FDD so the new rules take effect:

- `POST /run-fdd` (trigger "run now")
- or `POST /jobs/fdd/run` (async job)

If you need to validate improvements before running the loop, use:

- `POST /data-model/sparql` for SPARQL-based checks
- and/or `GET /faults/definitions` / `GET /faults/active` for runtime state.

64.8 4) Example "worker" prompt for Open-Claw

Use this as your agent's system prompt (or as the first instruction in your developer prompt). Replace bracketed placeholders as needed.

```
You are an Open-FDD worker for building operators.

Tools you can call:
- Open-FDD documentation context: GET {{OFDD_BASE_URL}}/model-
context/docs
- Data model export: GET {{OFDD_BASE_URL}}/data-model/export
- Data model import: PUT {{OFDD_BASE_URL}}/data-model/import
- Rule YAML management:
  - GET {{OFDD_BASE_URL}}/rules
  - POST {{OFDD_BASE_URL}}/rules
  - POST {{OFDD_BASE_URL}}/rules/sync-definitions
  - DELETE {{OFDD_BASE_URL}}/rules/{filename}
- Validation helpers:
  - POST {{OFDD_BASE_URL}}/data-model/sparql

Context strategy:
- If you need more Open-FDD details and your context is running
out, re-fetch docs context:
  - GET /model-context/docs?query={{SEARCH_TERMS}}
&mode=excerpt&max_chars=28000

Brick tagging task:
1) Call GET /data-model/export.
2) Call GET /model-context/docs?query=data-model+import.
3) Produce ONLY valid import JSON: top-level keys `points` and
optional `equipment`.
4) Call PUT /data-model/import with the validated JSON.
5) If PUT returns an error, include the error message in your next
attempt prompt and retry (prompt chaining).

Fault tuning task:
1) Read current rules: GET /rules and (optionally) GET /rules/
{filename}.
2) Propose minimal YAML changes to improve the target fault
behavior.
3) Upload updated YAML: POST /rules (overwrite).
4) Sync definitions: POST /rules/sync-definitions.
5) Trigger evaluation: POST /run-fdd (or POST /jobs/fdd/run).
6) Summarize what changed and why.
```

64.9 5) Recommended “context window” behavior

For larger exports and/or multi-step workers:

- Start with `mode=excerpt`
- When you hit a knowledge gap, fetch a smaller targeted subset with `query=...`

- If you still need more, use `mode=slice&offset=...` to walk through the full doc file.

65 How-to Guides

66 How-to Guides

- MQTT integration (optional) — Mosquitto profile, BACnet2MQTT vs experimental MQTT RPC gateway on diy-bacnet-server; links to upstream docs.
- Grafana dashboards (optional) — Optional Grafana; React frontend provides equivalent timeseries and fault views. Datasource and dashboard JSON.
- Grafana SQL cookbook — SQL recipes for BACnet, faults, weather, system resources (variables, panels, sparklines).
- PyPI releases (open-fdd) — Tags, trusted publishing, local `twine check`.
- The optional openfdd-engine package — `openfdd_engine` vs `open_fdd.engine` vs Docker `fdd-loop`; when to use which install.
- Engine-only deployment and external IoT pipelines — `--mode engine` vs pandas `RuleRunner`; same YAML on DataFrames.
- Data model engineering (Brick + 223P MVP) — Engineering UI, JSON round-trip, `s223`/`ofdd` RDF, SPARQL examples, and how that ties to FDD + DB for impact-style analytics.
- Cloning and porting — Same tools, any building; checklist for OpenClaw clones on a bench.
- OpenClaw subscription setup (Codex OAuth) — ChatGPT subscription path vs API key; stale `openai/...` cleanup.
- Monitor the fake fault schedule — Interpret 180°F spikes on the fake BACnet bench.

67 Data model engineering (Brick + 223P MVP)

68 Data Model Engineering (MVP)

Open-FDD keeps **Brick** as the operational model and adds an **Engineering** layer: schedule-style metadata and **223-style topology** in the database, emitted into the same **knowledge graph** (TTL) you already query for FDD and BACnet.

68.1 What this adds

- Frontend page: **Data Model Engineering**
- Equipment engineering metadata persisted in `equipment.metadata.engineering` (PostgreSQL JSON)
- Round-trip via `PUT /data-model/import` and `GET /data-model/export`
- RDF emission on the unified model (same TTL build as Brick points —see `open_fdd/platform/data_model_ttl.py`, `_append_equipment_engineering`):
  - `ofdd:*` extension predicates for practical fields (design CFM, cooling tons, heating MBH, electrical, feeder panel, source documents, etc.)
  - `s223:*` topology patterns for connection points and conduits (inlet/outlet, duct segments, mediums)
- **Data Model Testing** — SPARQL presets and copy-paste examples for engineering + topology

68.2 223P and what Open-FDD implements

ASHRAE Standard 223 defines a semantic way to describe how building equipment and media connect. In practice, **"223P"** often means tooling and workflows that capture that connectivity and related engineering data alongside operations.

Open-FDD's MVP is intentionally narrow:

- It **stores** structured engineering and topology in the DB and **emits** them into RDF using `s223:`-namespaced types (e.g. connection points, ducts) plus `ofdd:` for non-223 schedule fields.

- Connection endpoint linkage is currently represented with **`ofdd:connectsFromRef`** / **`ofdd:connectsToRef`** string references (MVP), not full RDF object links via `s223:connectsFrom` / `s223:connectsTo`.
- It does **not** claim full Standard 223 validation, certification, or a complete 223 product data model. The value here is **one graph**: Brick + BACnet + timeseries refs + engineering + topology, all queryable with **SPARQL** and aligned with how integrators already think about **connections** and **submittal data**.

68.3 How this connects to FDD, PostgreSQL, and the knowledge graph

The platform already treats the **database as source of truth** and **regenerates** the Brick TTL on CRUD and data-model import. Engineering metadata follows the same pattern.

Layer	Role
PostgreSQL	Sites, equipment (including `metadata.engineering`), points, **`timeseries_readings`**, fault definitions and fault **results** (per run / equipment / time).
TTL (`config/data_model.ttl` or export)	**Unified RDF**: Brick entities, `ref:BACnetReference` / `ref:TimeseriesReference`, `ofdd:mapsToRuleInput`, BACnet device graph, plus **`ofdd:`** and **`s223:`** from engineering import.
FDD loop	SPARQL over that graph builds the **column map** (timeseries `external_id` → rule input). Rules are still **pandas over pivoted telemetry**; they do not automatically pull engineering scalars into the DataFrame unless you extend the runner or join externally.

Layer	Role
SPARQL / integrations	Query **design cooling tons** on an AHU, **`brick:feeds`** downstream VAVs, and **BACnet-backed** valve points in **one** graph—then join to SQL on site/equipment/point keys or time windows.

So: **yes**, the **data model, knowledge graph, and DB** are wired so you can **reason across** "what the equipment is rated for" (engineering), "what it is doing" (time series), and "what the rules said" (faults). That is the foundation for **optimization playbooks**, **GL36-style context**, and **energy impact sketches**.

68.4 Energy penalty and impact calculations (scope)

Today, core Open-FDD focuses on detection and diagnostics (fault flags, trends, Grafana/React views). It does **not** ship a calibrated **M&V** or automatic **utility-dollar** engine in the main loop.

Architecturally, energy penalty workflows are supported because you can:

1. **SPARQL** equipment for **`ofdd:coolingCapacityTons`**, **`ofdd:heatingCapacityMBH`**, **`ofdd:designCFM`**, and topology (**`brick:feeds`** / **`brick:isFedBy`**).
2. **Resolve points** on that equipment via **`brick:isPointOf`** and **`ref:TimeseriesReference`** → `external_id` / DB table.
3. **Correlate** with fault episodes from the **fault results** tables (or exported CSV) over a time window.
4. Apply a **simple model** (e.g. design capacity × estimated duty fraction × hours × crude kW/ton)—see §5 in `examples/223P_engineering/README.md`.

That path is appropriate for **notebooks**, **downstream analytics services**, or future product features—not a promise that the FDD binary computes kWh for every fault out of the box.

68.5 Backward compatibility

- Existing Brick / BACnet / time-series / `rule_input` workflows are unchanged
- Unknown metadata keys under `engineering` are preserved on import
- Import/export shape remains backward compatible; engineering fields are **optional**

68.6 JSON pattern

Use `equipment[].engineering` in import payloads. Example: `examples/223P_engineering/engineering_import_example.json`.

68.7 RDF pattern

Examples (offline or diff against live export):

- `examples/223P_engineering/engineering_topology_example.ttl` — `s223` connection points + `ofdd` engineering
- `examples/223P_engineering/engineering_graph_mini.ttl` — adds **`brick:feeds`** / **`brick:isFedBy`**, plus a point with **`ref:BACnetReference`** + **`ref:TimeseriesReference`**

68.8 Query examples

- `examples/223P_engineering/README.md` — §4 SPARQL (copy-paste)
- Broader SPARQL patterns: SPARQL cookbook, External representations

68.9 Related docs

- Data model flow (DB → TTL → FDD)
- Fault rules overview — Brick-driven column map; engineering is complementary metadata for analytics

69 PyPI releases (open-fdd)

70 PyPI releases (`open-fdd`)

Canonical public install for the 2.x rules engine:

```
pip install open-fdd
```

That distribution ships **`open_fdd.engine`** (`RuleRunner`, YAML rules, same stack as the platform FDD loop). **Why does PyPI "Release history" mix `0.1.x` and `2.x`?** That is still **one** project, **`open-fdd`**: older **0.1.x** uploads were the legacy, hard-coded pandas era; **2.x** is the current config-driven engine + platform-capable tree. Pip resolves the latest **2.x** unless you pin an old version.

70.1 `openfdd-engine` (optional second PyPI name)

The repo also contains **`packages/openfdd-engine/`** (import name **`openfdd_engine`**): a thin re-export that **depends on `open-fdd`**. Its **`version` is kept the same as `open-fdd` 2.x** (e.g. both `2.0.13`) so release numbers stay easy to reason about, even though it is a **separate** PyPI project name.

Publishing uses **`.github/workflows/publish-openfdd-engine.yml`**: push tag **`openfdd-engine-vX.Y.Z`** after the **`openfdd-engine`** PyPI project has a **trusted publisher** for that workflow (same OIDC idea as `open-fdd`). **`workflow_dispatch`** builds only; upload runs on matching tags.

70.2 Release baseline (PyPI)

- Check live version:

```
curl -s https://pypi.org/pypi/open-fdd/json | python3 -c "import sys, json; print(json.load(sys.stdin)['info']['version'])"
```

70.3 1) Before a release of `open-fdd`

1. **Version** — Root **`pyproject.toml`** → `[project] version` (e.g. `2.0.11`).
2. **`openfdd-engine`** — Bump **`packages/openfdd-engine/pyproject.toml`** `version` to the **same `X.Y.Z`** as `open-fdd`, and set **`open-fdd>=X.Y.Z`** in dependencies. Tag **`openfdd-engine-vX.Y.Z`** if you publish that project.

70.3.1 Transition checklist (legacy 0.1.x → 2.x on PyPI)

1. Confirm root `pyproject.toml` sets **`open-fdd`** to the intended **`2.x`** version.
2. Merge/push release commits to **`master`** (or your release branch).
3. Tag **`open-fdd-vX.Y.Z`** and push the tag.
4. Confirm open-fdd on PyPI shows **`X.Y.Z`**.

70.3.2 PyPI upload auth (`open-fdd` only)

CI uses **PyPI Trusted Publishing** (OIDC from GitHub). Configure **one** trusted publisher on the **`open-fdd`** PyPI project:

1. pypi.org → project **`open-fdd`** → **Manage project** → **Publishing** (not Settings).
2. Add a **GitHub** trusted publisher:
   - **Owner:** `bbartling`
   - **Repository:** `open-fdd`
   - **Workflow name:** **`publish-open-fdd.yml`**
3. Save.

Workflow file: **`.github/workflows/publish-open-fdd.yml`**. Tags **`open-fdd-v*`** trigger upload. **`workflow_dispatch`** is build/check only (no upload) if configured that way.

Official guide: Publishing with GitHub Actions OIDC.

70.3.3 Fallback: API token

If you cannot use OIDC, pass a project-scoped token, for example:

```yaml
uses: pypa/gh-action-pypi-publish@release/v1
with:
  packages-dir: dist/
  password: ${{ secrets.PYPI_OPENFDD_TOKEN }}
```

70.3.4 If CI shows `invalid-publisher` or `403`

- Trusted publisher on PyPI doesn't match the **exact** workflow filename or repository.
- Tag points to a commit **before** the OIDC workflow existed — merge fix, retag.
- **Version already on PyPI** — bump version and use a **new** tag.

70.4 2) Publish `open-fdd` (commands)

70.4.1 Local dry run

```
cd /path/to/open-fdd   # repo root
python -m pip install --upgrade pip build twine
python -m build
twine check dist/*
```

70.4.2 GitHub Actions

```
git checkout master   # or your release branch
# bump version in pyproject.toml, commit, push
git tag open-fdd-v2.0.11
git push origin open-fdd-v2.0.11
```

70.5 3) Publish `openfdd-engine` (second PyPI project)

1. Create project **`openfdd-engine`** on PyPI (if it does not exist).
2. Add a **trusted publisher** pointing at workflow **`publish-openfdd-engine.yml`** (same repository as `open-fdd`).
3. Tag **`openfdd-engine-vX.Y.Z`** (same numbers as **`open-fdd-vX.Y.Z`**) and push — only after PyPI is configured, or OIDC fails with `invalid-publisher`.

70.6 Scope policy

- **PyPI** ships the **Python packages** (**`open-fdd`** is the supported public name for 2.x).

- **Full edge stack** remains **repo** + **Docker** (`./scripts/bootstrap.sh`).

See also: Engine-only deployment and external IoT pipelines.

71 Engine-only deployment and external IoT pipelines

72 Engine-only deployment and external IoT pipelines

Some integrators already operate **data collection** (historians, MQTT, proprietary BAS exports) and **modeling / semantics** (warehouse schemas, optional Brick elsewhere). Open-FDD's **`--mode engine`** and the **pandas YAML engine** let you add **FDD** without adopting the full stack.

> **Package names:** The rules code lives under **`open_fdd.engine`**. The repo's optional **`openfdd-engine`** package (**`openfdd_engine`**) is a thin re-export around the same API — not a different engine. See The optional openfdd-engine package for a comparison table and Docker vs library paths.

72.1 What `--mode engine` starts (Docker)

From the repo root:

```
./scripts/bootstrap.sh --mode engine
```

This brings up **TimescaleDB**, the **`fdd-loop`** service, and **`weather-scraper`** — see Modular architecture for the matrix. There is **no** API, React, or BACnet scraper in this slice by default.

When it fits

- You will **load aligned time-series into Postgres** (same DB the platform uses) and let the existing loop consume **rules** + **data model** from the normal Open-FDD deployment pattern.
- You want **weather-augmented** or **scheduled** FDD in the same containerized runtime as production Open-FDD.

When something else fits better

- Your data lives in **Snowflake** / **BigQuery** / **a lake** and you only need **rule evaluation** on batches or streams: use the **Python library** path below (no Docker).

72.2 Library path — same YAML, any DataFrame

The rule runner is **`open_fdd.engine.runner.RuleRunner`**. It loads the **same** `.yaml` rule files as the platform (`type: bounds|flatline|expression|hunting|oa_fraction|erv_efficiency`, `inputs`, `params`, etc.). Authoring references:

- Expression rule cookbook
- Examples under `examples/my_rules/` in the repo

Minimal integration pattern

1. Build a **pandas** `DataFrame` whose columns are your sensor traces (and optional `timestamp`).
2. Point **`RuleRunner`** at a directory of `.yaml` files **or** pass `rules=[...]` dicts.
3. Call **`run(df, timestamp_col=..., skip_missing_columns=True, column_map={...})`**.
4. Read boolean **`*_flag`** columns (and optional rolling persistence — same parameters as in-platform).

`column_map` — when your modeling layer uses Brick class URIs or tags but dataframe columns are different (e.g. `temp_sa` vs `Supply_Air_Temperature_Sensor`), pass the same **`column_map`** concept the platform uses after SPARQL resolution. See `RuleRunner.run` docstring in `open_fdd/engine/runner.py`.

72.2.1 Bring your own `column_map` or resolver

Integrators own the bridge from **their** naming (warehouse columns, Haystack refs, another graph) into what **`RuleRunner`** expects:

1. **Plain dict (most common)** — Build `column_map: dict[str, str]` (rule input / Brick-class key → **actual DataFrame column name**) and pass it to **`RuleRunner.run(..., column_map=column_map)`**. No TTL required on your side if you already know the columns.

2. **Brick TTL in library code** — If you have a Brick `.ttl` file (or the same shape as Open-FDD's model), reuse the platform logic:

```
        from pathlib import Path
        from open_fdd.engine.column_map_resolver import
BrickTtlColumnMapResolver

        column_map =
BrickTtlColumnMapResolver().build_column_map(ttl_path=Path("path/
to/model.ttl"))
```

   Same behavior as `resolve_from_ttl` when the file exists; requires `open-fdd[brick]` / rdflib.

3. **Custom `ColumnMapResolver`** — Implement the `ColumnMapResolver` protocol (`build_column_map(*, ttl_path: Path) -> dict[str, str]`) with your own lookup (REST, SQL, manifest file, etc.). You may ignore `ttl_path` if your source is elsewhere. For **forked** platform code, `run_fdd_loop(..., column_map_resolver=your_resolver)` swaps mapping for the DB loop; the **stock Docker** `fdd-loop` does **not** set this — it keeps `BrickTtlColumnMapResolver`.

Priority / policy: There is no automatic "ontology priority" in the engine — you supply **one** `column_map` per run (or one resolver that returns it). Ambiguity (e.g. multiple Haystack matches) should be resolved **before** calling `RuleRunner` with a strict dict.

Types live in `open_fdd.engine.column_map_resolver` and are re-exported from `openfdd_engine` (shim only). More context: The optional openfdd-engine package, GitHub **#122** (resolver RFC).

72.2.2 Manifest file + composite priority (workshop / gap-fill)

- `load_column_map_manifest(path)` — reads `.json` or `.yaml` / `.yml`. Accepts either a flat `str → str` object or `column_map:` nested mapping.
- `ManifestColumnMapResolver(path)` — same map via the `ColumnMapResolver` protocol; `build_column_map` ignores `ttl_path` (manifest is the source of truth for that resolver).
- `FirstWinsCompositeResolver(r1, r2, ...)` — runs each resolver in order; **the first resolver to define a key wins** (e.g. `BrickTtlColumnMapResolver()` then `ManifestColumnMapResolver("extras.yaml")` so TTL fills most keys and the manifest only adds missing ones). This is the supported

pattern for **ontology-style priority** without unsafe dynamic imports.

Config-driven resolver class names (e.g. loading a Python import path from env) are **not** supported on purpose — easy to turn into **import injection**. Compose resolvers in code or a thin startup script.

Examples: `examples/column_map_resolver_workshop/` — **`demo_one_shot.py`** (manifest + **`RuleRunner`**), **`demo_multi_ontology_illustration.py`** (illustrative Brick / Haystack / DBO / 223P key shapes).

Install

```
        pip install -e ".[dev]"   # from open-fdd clone, or
        pip install open-fdd      # when using a published version that
includes the engine
```

The repo also contains an optional **`openfdd-engine`** tree (`packages/openfdd-engine/`) that re-exports the same API; **`pip install open-fdd`** is the supported public install — use **`open_fdd.engine`** after install. More detail: The optional openfdd-engine package.

72.3 Standalone playground (optional)

An **in-repo** example folder is included for workshops and quick starts:

- `examples/engine_iot_playground/` with `README.md`, `rules/*.yaml`, `data/RTU11.csv`, `run_demo.py`, and `RTU11_engine_tutorial.ipynb`.

That pattern is **not** a second engine — it is the **same** code path as production rules, without Docker.

72.4 Summary

Approach	You bring	Open-FDD provides
`--mode engine`	Postgres feed + ops for containers	`fdd-loop`, weather worker, DB
`RuleRunner` in Python	DataFrame + YAML directory	Identical rule YAML semantics on pandas
Full stack		

Approach	You bring	Open-FDD provides
	BACnet / UI needs	Collector + model + engine together

For mode overview and service list, start at Modular architecture.

73 The optional openfdd-engine package

74 The optional `openfdd-engine` package (`openfdd_engine`)

Open-FDD has **one real rules implementation**: YAML files evaluated on **pandas** via `RuleRunner` in the `open_fdd.engine` package. Everything else is either that same code in a different wrapper, or documentation about how to deploy it.

This page explains how the **in-repo** package `packages/openfdd-engine/` (PyPI name `openfdd-engine`, import `openfdd_engine`) fits next to `open_fdd.engine` and the **Docker** `fdd-loop` service.

74.1 Re-exports only — no duplicated types

`openfdd_engine` does not define its own rule engine, Pydantic models, or resolver types. The shim `__init__.py` imports from `open_fdd.engine` (e.g. `RuleRunner`, `resolve_from_ttl`, `BrickTtlColumnMapResolver`, `ColumnMapResolver`) and lists them in `__all__`. Any new public API should be added **once** under `open_fdd.engine`; the shim only gains new names via **import** + **re-export**.

- **Why:** One source of truth for PyPI `open-fdd` and optional `openfdd-engine`; no drift between packages.
- **Versioning:** `open-fdd` on PyPI is authoritative; `openfdd-engine` pins `open-fdd>=…` in its `pyproject.toml` when published.

The same policy applies to the repo-root **`openfdd_engine/`** namespace used in some dev installs.

74.2 Mental model (three layers)

Layer	What it is	Typical import / entrypoint
Core engine	Loads YAML rules, runs checks on a `DataFrame`, produces fault columns	`from open_fdd.engine.runner import RuleRunner`
Optional shim	Same classes, different package name (depends on **`open-fdd`**)	`from openfdd_engine import RuleRunner`
Platform loop	Pulls Timescale data + TTL, builds frames per site, calls **`RuleRunner`**, writes **`fault_results`**	Docker: `python -m open_fdd.platform.drivers.run_rule_loop --loop` → `open_fdd.platform.loop.run_fdd_loop`

There is **no second rule engine** inside `openfdd_engine`: its modules **re-export** symbols from **`open_fdd.engine`** (see `packages/openfdd-engine/src/openfdd_engine/runner.py`).

74.3 When to use `pip install open-fdd` vs `openfdd-engine`

Default for everyone (PyPI, notebooks, CI, integrators):

```
pip install open-fdd
```

```
from open_fdd.engine.runner import RuleRunner, load_rule, load_rules_from_dir
```

```
# brick resolution in-platform / with extras:
from open_fdd.engine.brick_resolver import ...
```

Optional `openfdd-engine` distribution — use only if you **intentionally** want:

- A **separate PyPI project name** (`openfdd-engine`) for procurement or a minimal dependency line that still pulls **`open-fdd`** underneath, or
- An **editable install** of only the shim while developing the monorepo:

```
cd packages/openfdd-engine
pip install -e .
# optional TTL helpers:
pip install -e ".[brick]"

from openfdd_engine import (
    RuleRunner,
    load_rule,
    bounds_map_from_rule,
    resolve_from_ttl,
    BrickTtlColumnMapResolver,
    ColumnMapResolver,
)
```

Public story: `pip install open-fdd` is the supported install; **`openfdd-engine`** may or may not be published to PyPI as a second project (maintainer setup). See PyPI releases (`open-fdd`).

74.4 How the full stack uses the engine (so IoT-only users know what they are skipping)

In Docker, **`fdd-loop`** runs:

```
python -m open_fdd.platform.drivers.run_rule_loop --loop
```

That driver:

1. Optionally refreshes Open-Meteo for the lookback window.
2. Calls **`run_fdd_loop()`** in **`open_fdd.platform.loop`**, which loads YAML from **`rules_dir`**, loads timeseries from Postgres, builds **`column_map`** via **`BrickTtlColumnMapResolver`** (Brick TTL — same as historical behavior) from **`config/data_model.ttl`**, and constructs **`RuleRunner`** from **`open_fdd.engine.runner`**. Advanced use: pass **`column_map_resolver=...`** into **`run_fdd_loop`** to swap mapping logic;

the default stack does not. Library helpers in **`open_fdd.engine.column_map_resolver`**: **`ManifestColumnMapResolver`**, **`FirstWinsCompositeResolver`**, **`load_column_map_manifest`** (also re-exported from **`openfdd_engine`**).
3. Writes results to **`fault_results`** (and run log for Grafana).

So: **same YAML and same `RuleRunner` semantics** as a standalone script; the platform adds **DB, schedule, TTL, and weather**.

Operational triggers (touch file, one-shot exec): Operations — FDD loop.

74.5 Library-only / external data (no Docker engine mode)

If your data is already in a warehouse or a CSV pipeline, you do **not** need `openfdd_engine` — use **`open_fdd.engine`** on a **`DataFrame`** and the same rule files as the platform. Step-by-step and **`column_map`**: Engine-only deployment and external IoT pipelines.

Worked example under the repo: `examples/engine_iot_playground/`.

74.6 Summary

Goal	Install	Import / run
Normal development & PyPI users	`pip install open-fdd`	`open_fdd.engine.*`
Optional second package name	`pip install -e packages/openfdd-engine` (or published `openfdd-engine`)	`openfdd_engine.*`
Scheduled FDD against platform DB	Docker **`--mode engine`** / full stack	`fdd-loop` → `run_rule_loop` → `run_fdd_loop` → `RuleRunner`

74.7 See also

- Engine-only deployment and external IoT pipelines — `--mode engine` vs pandas `RuleRunner`
- PyPI releases (`open-fdd`) — tags, publishing, `openfdd-engine` scope
- Modular architecture — which services run in **engine** mode
- Expression rule cookbook — YAML rule types and patterns
- Package README: `packages/openfdd-engine/README.md`

75 Grafana SQL cookbook

76 Grafana SQL cookbook —

76.1 BACnet

- Dashboard Variables (Required for Dropdowns)

Before creating the panels below, you must set up the dashboard variables so the drop-down menus work.

In Grafana, go to **Dashboard Settings (gear icon)** > **Variables** > **Add variable**. Set the **Type** to `Query`, select your TimescaleDB datasource, and use the following queries:

76.1.1 1. Site Variable

- **Name:** `site`
- **Query:**

```
SELECT s.name AS __text, s.id::text AS __value
FROM sites s
ORDER BY s.name;
```

76.1.2 2. Device Variable

- **Name:** `device`
- **Query:**

```
SELECT DISTINCT p.bacnet_device_id::text AS __text,
p.bacnet_device_id::text AS __value
```

```sql
FROM points p
WHERE p.site_id::text = '$site'
  AND p.bacnet_device_id IS NOT NULL
ORDER BY 1;
```

76.1.3 3. Point Variable

- **Name:** `point`
- **Query:**

```sql
SELECT p.external_id AS __text, p.external_id AS __value
FROM points p
WHERE p.site_id::text = '$site'
  AND p.bacnet_device_id::text = '$device'
ORDER BY 1;
```

76.1.4 BACnet only data

Insert Snip here

- This dashboard uses `point`, `device`, and `site` variables.

```json
{
  "id": 1,
  "type": "timeseries",
  "title": "BACnet",
  "gridPos": {
    "x": 0,
    "y": 0,
    "h": 16,
    "w": 24
  },
  "fieldConfig": {
    "defaults": {
      "custom": {
        "drawStyle": "line",
        "lineInterpolation": "linear",
        "barAlignment": 0,
        "barWidthFactor": 0.6,
        "lineWidth": 1,
        "fillOpacity": 0,
        "gradientMode": "none",
        "spanNulls": false,
        "insertNulls": false,
        "showPoints": "auto",
        "showValues": false,
        "pointSize": 5,
        "stacking": {
          "mode": "none",
```

```
                "group": "A"
              },
              "axisPlacement": "auto",
              "axisLabel": "",
              "axisColorMode": "text",
              "axisBorderShow": false,
              "scaleDistribution": {
                "type": "linear"
              },
              "axisCenteredZero": false,
              "hideFrom": {
                "tooltip": false,
                "viz": false,
                "legend": false
              },
              "thresholdsStyle": {
                "mode": "off"
              }
            },
            "color": {
              "mode": "palette-classic"
            },
            "mappings": [],
            "thresholds": {
              "mode": "absolute",
              "steps": [
                {
                  "color": "green",
                  "value": null
                },
                {
                  "color": "red",
                  "value": 80
                }
              ]
            }
          },
          "overrides": []
        },
        "pluginVersion": "12.4.0",
        "targets": [
          {
            "editorMode": "code",
            "format": "time_series",
            "rawQuery": true,
            "rawSql": "SELECT\n
time_bucket(\n    make_interval(secs => ($__interval_ms /
1000)::int),\n    tr.ts\n  ) AS \"time\",\n  avg(tr.value) AS
\"value\",\n  p.external_id AS \"metric\"\nFROM
timeseries_readings tr\nJOIN points p ON p.id =
tr.point_id\nWHERE $__timeFilter(tr.ts)\n  AND p.site_id::text IN
(${site:sqlstring})\n  AND p.bacnet_device_id::text IN ($
```

```
{device:sqlstring})\n  AND p.external_id IN (${point:sqlstring})
\nGROUP BY 1, 3\nORDER BY 1, 3;",
              "refId": "A",
              "sql": {
                "columns": [
                  {
                    "parameters": [],
                    "type": "function"
                  }
                ],
                "groupBy": [
                  {
                    "property": {
                      "type": "string"
                    },
                    "type": "groupBy"
                  }
                ],
                "limit": 50
              }
            }
          ],
          "datasource": {
            "uid": "openfdd_timescale"
          },
          "options": {
            "tooltip": {
              "mode": "single",
              "sort": "none",
              "hideZeros": false
            },
            "legend": {
              "showLegend": true,
              "displayMode": "list",
              "placement": "bottom",
              "calcs": []
            }
          }
        }
```

76.2 BACnet Plus Fault Data

```
        {
          "id": null,
          "type": "timeseries",
          "title": "BACnet Telemetry w/ Fault Overlays",
          "gridPos": {
            "x": 0,
            "y": 16,
            "h": 12,
```

```
        "w": 24
      },
      "fieldConfig": {
        "defaults": {
          "custom": {
            "drawStyle": "line",
            "lineInterpolation": "linear",
            "barAlignment": 0,
            "barWidthFactor": 0.6,
            "lineWidth": 2,
            "fillOpacity": 10,
            "gradientMode": "none",
            "spanNulls": false,
            "insertNulls": false,
            "showPoints": "never",
            "showValues": false,
            "pointSize": 5,
            "stacking": {
              "mode": "none",
              "group": "A"
            },
            "axisPlacement": "left",
            "axisLabel": "",
            "axisColorMode": "text",
            "axisBorderShow": false,
            "scaleDistribution": {
              "type": "linear"
            },
            "axisCenteredZero": false,
            "hideFrom": {
              "tooltip": false,
              "viz": false,
              "legend": false
            },
            "thresholdsStyle": {
              "mode": "off"
            }
          },
          "color": {
            "mode": "palette-classic"
          },
          "mappings": [],
          "thresholds": {
            "mode": "absolute",
            "steps": [
              {
                "value": null,
                "color": "green"
              },
              {
                "value": 80,
                "color": "red"
```

```
                }
              ]
            }
          },
          "overrides": [
            {
              "matcher": {
                "id": "byFrameRefID",
                "options": "B"
              },
              "properties": [
                {
                  "id": "custom.axisPlacement",
                  "value": "right"
                },
                {
                  "id": "custom.drawStyle",
                  "value": "bars"
                },
                {
                  "id": "custom.fillOpacity",
                  "value": 30
                },
                {
                  "id": "min",
                  "value": 0
                },
                {
                  "id": "max",
                  "value": 1.2
                },
                {
                  "id": "custom.axisLabel",
                  "value": "Fault Active (1 = Yes)"
                }
              ]
            }
          ]
        },
        "pluginVersion": "12.4.0",
        "targets": [
          {
            "refId": "A",
            "format": "time_series",
            "rawQuery": true,
            "editorMode": "code",
            "rawSql": "SELECT\n  time_bucket(make_interval(secs =>
($__interval_ms / 1000)::int), tr.ts) AS \"time\",\n
avg(tr.value) AS \"value\",\n  p.external_id AS \"metric\"\nFROM
timeseries_readings tr\nJOIN points p ON p.id =
tr.point_id\nWHERE $__timeFilter(tr.ts)\n  AND p.site_id::text IN
(${site:sqlstring})\n  AND p.bacnet_device_id::text IN ($
```

```
{device:sqlstring})\n  AND p.external_id IN (${point:sqlstring})
\nGROUP BY 1, 3\nORDER BY 1, 3;"
            },
            {
              "refId": "B",
              "format": "time_series",
              "rawQuery": true,
              "editorMode": "code",
              "rawSql": "SELECT\n  time_bucket(make_interval(secs =>
($__interval_ms / 1000)::int), fr.ts) AS \"time\",\n
max(fr.flag_value) AS \"value\",\n  fd.name AS \"metric\"\nFROM
fault_results fr\nJOIN fault_definitions fd ON fd.fault_id =
fr.fault_id\nWHERE $__timeFilter(fr.ts)\n  AND (fr.site_id IN ($
{site:sqlstring}) OR fr.site_id IN (SELECT name FROM sites WHERE
id::text IN (${site:sqlstring})))\nGROUP BY 1, 3\nORDER BY 1, 3;"
            }
          ],
          "datasource": {
            "uid": "openfdd_timescale"
          },
          "options": {
            "tooltip": {
              "mode": "multi",
              "sort": "none",
              "hideZeros": false
            },
            "legend": {
              "showLegend": true,
              "displayMode": "list",
              "placement": "bottom",
              "calcs": []
            }
          }
        }
```

76.3 Host stats (stack-host-stats)

The **stack-host-stats** container scrapes host memory, load, swap, disk usage, and Docker container stats into `host_metrics`, `disk_metrics`, and `container_metrics`. Use the same TimescaleDB datasource (`openfdd_timescale`).

76.3.1 Dashboard variable (Host)

- **Name:** `hostname`
- **Type:** Query
- **Datasource:** openfdd_timescale

- **Query:**

```sql
SELECT DISTINCT hostname AS __text, hostname AS __value
FROM host_metrics
WHERE $__timeFilter(ts)
ORDER BY 1;
```

76.3.2 Host — Memory (used / available / total)

```
{
  "id": null,
  "type": "timeseries",
  "title": "Host — Memory (used / available / total)",
  "gridPos": { "x": 0, "y": 0, "h": 10, "w": 24 },
  "fieldConfig": {
    "defaults": {
      "custom": {
        "drawStyle": "line",
        "lineInterpolation": "linear",
        "lineWidth": 2,
        "fillOpacity": 12,
        "axisPlacement": "left",
        "scaleDistribution": { "type": "linear" }
      },
      "unit": "bytes",
      "color": { "mode": "palette-classic" },
      "thresholds": { "mode": "absolute", "steps": [{"color":
"green", "value": null}] }
    },
    "overrides": [
      {
        "matcher": { "id": "byName", "options":
"mem_used_bytes" },
        "properties": [{ "id": "displayName", "value": "Used" }]
      },
      {
        "matcher": { "id": "byName", "options":
"mem_available_bytes" },
        "properties": [{ "id": "displayName", "value":
"Available" }]
      },
      {
        "matcher": { "id": "byName", "options":
"mem_total_bytes" },
        "properties": [{ "id": "displayName", "value": "Total" }]
      }
    ]
  },
  "targets": [
    {
      "refId": "A",
```

```
              "format": "time_series",
              "rawQuery": true,
              "editorMode": "code",
              "rawSql": "SELECT ts AS \"time\", mem_used_bytes AS
\"mem_used_bytes\", mem_available_bytes AS
\"mem_available_bytes\", mem_total_bytes AS \"mem_total_bytes\"
FROM host_metrics WHERE $__timeFilter(ts) AND hostname IN ($
{hostname:sqlstring}) ORDER BY ts;"
            }
          ],
          "datasource": { "uid": "openfdd_timescale" },
          "options": { "legend": { "showLegend": true, "placement":
"bottom" } }
        }
```

76.3.3 Host — Load (1 / 5 / 15 min)

```
        {
          "id": null,
          "type": "timeseries",
          "title": "Host — Load (1 / 5 / 15 min)",
          "gridPos": { "x": 0, "y": 10, "h": 8, "w": 24 },
          "fieldConfig": {
            "defaults": {
              "custom": { "drawStyle": "line", "lineWidth": 2,
"fillOpacity": 8, "axisPlacement": "left" },
              "color": { "mode": "palette-classic" },
              "thresholds": { "mode": "absolute", "steps": [{"color":
"green", "value": null}] }
            },
            "overrides": [
              { "matcher": { "id": "byName", "options": "load_1" },
"properties": [{ "id": "displayName", "value": "Load 1m" }] },
              { "matcher": { "id": "byName", "options": "load_5" },
"properties": [{ "id": "displayName", "value": "Load 5m" }] },
              { "matcher": { "id": "byName", "options": "load_15" },
"properties": [{ "id": "displayName", "value": "Load 15m" }] }
            ]
          },
          "targets": [
            {
              "refId": "A",
              "format": "time_series",
              "rawQuery": true,
              "editorMode": "code",
              "rawSql": "SELECT ts AS \"time\", load_1 AS \"load_1\",
load_5 AS \"load_5\", load_15 AS \"load_15\" FROM host_metrics
WHERE $__timeFilter(ts) AND hostname IN (${hostname:sqlstring})
ORDER BY ts;"
            }
          ],
```

```
          "datasource": { "uid": "openfdd_timescale" },
          "options": { "legend": { "showLegend": true, "placement":
"bottom" } }
        }
```

76.3.4 Host — Swap (used / total)

```
        {
          "id": null,
          "type": "timeseries",
          "title": "Host — Swap (used / total)",
          "gridPos": { "x": 0, "y": 18, "h": 6, "w": 24 },
          "fieldConfig": {
            "defaults": {
              "custom": { "drawStyle": "line", "lineWidth": 2,
"fillOpacity": 10, "axisPlacement": "left" },
              "unit": "bytes",
              "color": { "mode": "palette-classic" },
              "thresholds": { "mode": "absolute", "steps": [{"color":
"green", "value": null}] }
            },
            "overrides": [
              { "matcher": { "id": "byName", "options":
"swap_used_bytes" }, "properties": [{ "id": "displayName",
"value": "Swap used" }] },
              { "matcher": { "id": "byName", "options":
"swap_total_bytes" }, "properties": [{ "id": "displayName",
"value": "Swap total" }] }
            ]
          },
          "targets": [
            {
              "refId": "A",
              "format": "time_series",
              "rawQuery": true,
              "editorMode": "code",
              "rawSql": "SELECT ts AS \"time\", swap_used_bytes AS
\"swap_used_bytes\", swap_total_bytes AS
\"swap_total_bytes\" FROM host_metrics WHERE $__timeFilter(ts) AND
hostname IN (${hostname:sqlstring}) ORDER BY ts;"
            }
          ],
          "datasource": { "uid": "openfdd_timescale" },
          "options": { "legend": { "showLegend": true, "placement":
"bottom" } }
        }
```

76.3.5 Host — Disk space (free / used / total)

Disk metrics come from `disk_metrics` (mount path from env `OFDD_DISK_MOUNT_PATHS`, default `/`). The table is created by the same migration as `host_metrics` (`stack/sql/006_host_metrics.sql`); re-run migrations if you added host-stats after an older deploy.

```
        {
          "id": null,
          "type": "timeseries",
          "title": "Host — Disk space (free / used / total)",
          "gridPos": { "x": 0, "y": 24, "h": 10, "w": 24 },
          "fieldConfig": {
            "defaults": {
              "custom": { "drawStyle": "line", "lineWidth": 2,
"fillOpacity": 12, "axisPlacement": "left" },
              "unit": "bytes",
              "color": { "mode": "palette-classic" },
              "thresholds": { "mode": "absolute", "steps": [{"color":
"green", "value": null}] }
            },
            "overrides": [
              { "matcher": { "id": "byName", "options": "free_bytes" },
"properties": [{ "id": "displayName", "value": "Free" }] },
              { "matcher": { "id": "byName", "options": "used_bytes" },
"properties": [{ "id": "displayName", "value": "Used" }] },
              { "matcher": { "id": "byName", "options": "total_bytes" },
"properties": [{ "id": "displayName", "value": "Total" }] }
            ]
          },
          "targets": [
            {
              "refId": "A",
              "format": "time_series",
              "rawQuery": true,
              "editorMode": "code",
              "rawSql": "SELECT ts AS \"time\", free_bytes AS
\"free_bytes\", used_bytes AS \"used_bytes\", total_bytes AS
\"total_bytes\" FROM disk_metrics WHERE $__timeFilter(ts) AND
hostname IN (${hostname:sqlstring}) AND mount_path = '/' ORDER BY
ts;"
            }
          ],
          "datasource": { "uid": "openfdd_timescale" },
          "options": { "legend": { "showLegend": true, "placement":
"bottom" } }
        }
```

76.3.6 Host — Disk available (gauge, latest)

```
        {
          "id": null,
          "type": "gauge",
          "title": "Host — Disk space available (latest)",
          "gridPos": { "x": 0, "y": 34, "h": 6, "w": 8 },
          "fieldConfig": {
            "defaults": {
              "unit": "bytes",
              "min": 0,
              "color": { "mode": "thresholds" },
              "thresholds": { "mode": "absolute", "steps": [{"color":
"red", "value": null}, {"color": "yellow", "value": 5368709120},
{"color": "green", "value": 10737418240}] }
            }
          },
          "targets": [
            {
              "refId": "A",
              "format": "time_series",
              "rawQuery": true,
              "editorMode": "code",
              "rawSql": "SELECT ts AS \"time\", free_bytes AS \"value\"
FROM disk_metrics WHERE hostname IN (${hostname:sqlstring}) AND
mount_path = '/' ORDER BY ts DESC LIMIT 1;"
            }
          ],
          "datasource": { "uid": "openfdd_timescale" },
          "options": { "showThresholdLabels": false }
        }
```

76.3.7 Containers — CPU %

```
        {
          "id": null,
          "type": "timeseries",
          "title": "Containers — CPU %",
          "gridPos": { "x": 0, "y": 40, "h": 8, "w": 24 },
          "fieldConfig": {
            "defaults": {
              "custom": { "drawStyle": "line", "lineWidth": 1,
"fillOpacity": 5, "axisPlacement": "left" },
              "unit": "percent",
              "min": 0,
              "color": { "mode": "palette-classic" },
              "thresholds": { "mode": "absolute", "steps": [{"color":
"green", "value": null}] }
            }
          },
          "targets": [
```

```
            {
              "refId": "A",
              "format": "time_series",
              "rawQuery": true,
              "editorMode": "code",
              "rawSql": "SELECT ts AS \"time\", cpu_pct AS \"value\",
container_name AS \"metric\" FROM container_metrics WHERE
$__timeFilter(ts) ORDER BY ts, container_name;"
            }
          ],
          "datasource": { "uid": "openfdd_timescale" },
          "options": { "legend": { "showLegend": true, "placement":
"bottom" } }
        }
```

76.3.8 Containers — Memory %

```
        {
          "id": null,
          "type": "timeseries",
          "title": "Containers — Memory %",
          "gridPos": { "x": 0, "y": 48, "h": 8, "w": 24 },
          "fieldConfig": {
            "defaults": {
              "custom": { "drawStyle": "line", "lineWidth": 1,
"fillOpacity": 5, "axisPlacement": "left" },
              "unit": "percent",
              "min": 0,
              "max": 100,
              "color": { "mode": "palette-classic" },
              "thresholds": { "mode": "absolute", "steps": [{"color":
"green", "value": null}] }
            }
          },
          "targets": [
            {
              "refId": "A",
              "format": "time_series",
              "rawQuery": true,
              "editorMode": "code",
              "rawSql": "SELECT ts AS \"time\", COALESCE(mem_pct, 0) AS
\"value\", container_name AS \"metric\" FROM container_metrics
WHERE $__timeFilter(ts) ORDER BY ts, container_name;"
            }
          ],
          "datasource": { "uid": "openfdd_timescale" },
          "options": { "legend": { "showLegend": true, "placement":
"bottom" } }
        }
```

76.3.9 Containers — Table (latest)

```
        {
          "id": null,
          "type": "table",
          "title": "Containers — Latest stats",
          "gridPos": { "x": 0, "y": 56, "h": 12, "w": 24 },
          "fieldConfig": {
            "defaults": { "custom": { "align": "auto", "displayMode":
"auto" }, "mappings": [], "thresholds": { "mode": "absolute",
"steps": [{"color": "green", "value": null}] } },
            "overrides": [
              { "matcher": { "id": "byName", "options": "cpu_pct" },
"properties": [{ "id": "unit", "value": "percent" }] },
              { "matcher": { "id": "byName", "options": "mem_pct" },
"properties": [{ "id": "unit", "value": "percent" }] },
              { "matcher": { "id": "byName", "options":
"mem_usage_bytes" }, "properties": [{ "id": "unit", "value":
"bytes" }] }
            ]
          },
          "targets": [
            {
              "refId": "A",
              "format": "table",
              "rawQuery": true,
              "editorMode": "code",
              "rawSql": "SELECT container_name, ts, cpu_pct, mem_pct,
mem_usage_bytes, pids FROM (SELECT *, row_number() OVER (PARTITION
BY container_name ORDER BY ts DESC) AS rn FROM container_metrics
WHERE $__timeFilter(ts)) sub WHERE rn = 1 ORDER BY
container_name;"
            }
          ],
          "datasource": { "uid": "openfdd_timescale" },
          "options": { "showHeader": true }
        }
```

76.4 Weather

76.4.1 Temp RH Dewpoint

```
        {
          "id": 21,
          "type": "timeseries",
          "title": "Weather — Temp / RH / Dewpoint",
          "gridPos": {
            "x": 0,
            "y": 0,
```

```
        "h": 10,
        "w": 24
      },
      "fieldConfig": {
        "defaults": {
          "custom": {
            "drawStyle": "line",
            "lineInterpolation": "linear",
            "barAlignment": 0,
            "barWidthFactor": 0.6,
            "lineWidth": 2,
            "fillOpacity": 12,
            "gradientMode": "none",
            "spanNulls": false,
            "insertNulls": false,
            "showPoints": "never",
            "showValues": false,
            "pointSize": 4,
            "stacking": {
              "mode": "none",
              "group": "A"
            },
            "axisPlacement": "left",
            "axisLabel": "°F",
            "axisColorMode": "text",
            "axisBorderShow": false,
            "scaleDistribution": {
              "type": "linear"
            },
            "axisCenteredZero": false,
            "hideFrom": {
              "tooltip": false,
              "viz": false,
              "legend": false
            },
            "thresholdsStyle": {
              "mode": "off"
            }
          },
          "color": {
            "mode": "palette-classic"
          },
          "mappings": [],
          "thresholds": {
            "mode": "absolute",
            "steps": [
              {
                "color": "green",
                "value": null
              },
              {
                "color": "red",
```

```
            "value": 80
          }
        ]
      }
    },
    "overrides": [
      {
        "matcher": {
          "id": "byName",
          "options": "temp_f"
        },
        "properties": [
          {
            "id": "unit",
            "value": "fahrenheit"
          },
          {
            "id": "custom.axisPlacement",
            "value": "left"
          },
          {
            "id": "custom.axisLabel",
            "value": "°F"
          }
        ]
      },
      {
        "matcher": {
          "id": "byName",
          "options": "dewpoint_f"
        },
        "properties": [
          {
            "id": "unit",
            "value": "fahrenheit"
          },
          {
            "id": "custom.axisPlacement",
            "value": "left"
          }
        ]
      },
      {
        "matcher": {
          "id": "byName",
          "options": "rh_pct"
        },
        "properties": [
          {
            "id": "unit",
            "value": "percent"
          },
```

```
                    {
                      "id": "custom.axisPlacement",
                      "value": "right"
                    },
                    {
                      "id": "custom.axisLabel",
                      "value": "RH (%)"
                    },
                    {
                      "id": "min",
                      "value": 0
                    },
                    {
                      "id": "max",
                      "value": 100
                    }
                  ]
                }
              ]
            },
            "pluginVersion": "12.4.0",
            "targets": [
              {
                "refId": "A",
                "editorMode": "code",
                "rawQuery": true,
                "format": "time_series",
                "rawSql": "SELECT\n  time_bucket(make_interval(secs =>
($__interval_ms / 1000)::int), tr.ts) AS \"time\",\n
avg(tr.value) AS \"value\",\n  p.external_id AS \"metric\"\nFROM
timeseries_readings tr\nJOIN points p ON p.id =
tr.point_id\nWHERE $__timeFilter(tr.ts)\n  AND p.site_id::text IN
(${site:sqlstring})\n  AND p.external_id IN
('temp_f','rh_pct','dewpoint_f')\nGROUP BY 1, 3\nORDER BY 1, 3;"
              }
            ],
            "datasource": {
              "uid": "openfdd_timescale"
            },
            "options": {
              "tooltip": {
                "mode": "single",
                "sort": "none",
                "hideZeros": false
              },
              "legend": {
                "showLegend": true,
                "displayMode": "list",
                "placement": "bottom",
                "calcs": []
              }
```

```
    }
  }
```

76.4.2 Wind Speed Gusts Direction

```
  {
    "id": 22,
    "type": "timeseries",
    "title": "Weather — Wind (Speed / Gust / Direction)",
    "gridPos": {
      "x": 0,
      "y": 10,
      "h": 9,
      "w": 24
    },
    "fieldConfig": {
      "defaults": {
        "custom": {
          "drawStyle": "line",
          "lineInterpolation": "linear",
          "barAlignment": 0,
          "barWidthFactor": 0.6,
          "lineWidth": 2,
          "fillOpacity": 10,
          "gradientMode": "none",
          "spanNulls": false,
          "insertNulls": false,
          "showPoints": "never",
          "showValues": false,
          "pointSize": 4,
          "stacking": {
            "mode": "none",
            "group": "A"
          },
          "axisPlacement": "left",
          "axisLabel": "mph",
          "axisColorMode": "text",
          "axisBorderShow": false,
          "scaleDistribution": {
            "type": "linear"
          },
          "axisCenteredZero": false,
          "hideFrom": {
            "tooltip": false,
            "viz": false,
            "legend": false
          },
          "thresholdsStyle": {
            "mode": "off"
          }
```

```
        },
        "color": {
          "mode": "palette-classic"
        },
        "mappings": [],
        "thresholds": {
          "mode": "absolute",
          "steps": [
            {
              "color": "green",
              "value": null
            },
            {
              "color": "red",
              "value": 80
            }
          ]
        }
      },
      "overrides": [
        {
          "matcher": {
            "id": "byName",
            "options": "wind_mph"
          },
          "properties": [
            {
              "id": "unit",
              "value": "mph"
            },
            {
              "id": "custom.axisPlacement",
              "value": "left"
            },
            {
              "id": "custom.axisLabel",
              "value": "mph"
            },
            {
              "id": "min",
              "value": 0
            }
          ]
        },
        {
          "matcher": {
            "id": "byName",
            "options": "gust_mph"
          },
          "properties": [
            {
              "id": "unit",
```

```
                "value": "mph"
              },
              {
                "id": "custom.axisPlacement",
                "value": "left"
              },
              {
                "id": "min",
                "value": 0
              }
            ]
          },
          {
            "matcher": {
              "id": "byName",
              "options": "wind_dir_deg"
            },
            "properties": [
              {
                "id": "unit",
                "value": "degrees"
              },
              {
                "id": "custom.axisPlacement",
                "value": "right"
              },
              {
                "id": "custom.axisLabel",
                "value": "°"
              },
              {
                "id": "min",
                "value": 0
              },
              {
                "id": "max",
                "value": 360
              }
            ]
          }
        ]
      },
      "pluginVersion": "12.4.0",
      "targets": [
        {
          "refId": "A",
          "editorMode": "code",
          "rawQuery": true,
          "format": "time_series",
          "rawSql": "SELECT\n  time_bucket(make_interval(secs =>
($__interval_ms / 1000)::int), tr.ts) AS \"time\",\n
avg(tr.value) AS \"value\",\n  p.external_id AS \"metric\"\nFROM
```

```
timeseries_readings tr\nJOIN points p ON p.id =
tr.point_id\nWHERE $__timeFilter(tr.ts)\n  AND p.site_id::text IN
(${site:sqlstring})\n  AND p.external_id IN
('wind_mph','gust_mph','wind_dir_deg')\nGROUP BY 1,
3\nORDER BY 1, 3;"
            }
          ],
          "datasource": {
            "uid": "openfdd_timescale"
          },
          "options": {
            "tooltip": {
              "mode": "single",
              "sort": "none",
              "hideZeros": false
            },
            "legend": {
              "showLegend": true,
              "displayMode": "list",
              "placement": "bottom",
              "calcs": []
            }
          }
        }
```

76.4.3 Wind Speed Gust Direction

```
        {
          "id": 22,
          "type": "timeseries",
          "title": "Weather — Wind (Speed / Gust / Direction)",
          "gridPos": {
            "x": 0,
            "y": 10,
            "h": 9,
            "w": 24
          },
          "fieldConfig": {
            "defaults": {
              "custom": {
                "drawStyle": "line",
                "lineInterpolation": "linear",
                "barAlignment": 0,
                "barWidthFactor": 0.6,
                "lineWidth": 2,
                "fillOpacity": 10,
                "gradientMode": "none",
                "spanNulls": false,
                "insertNulls": false,
                "showPoints": "never",
                "showValues": false,
```

```
            "pointSize": 4,
            "stacking": {
              "mode": "none",
              "group": "A"
            },
            "axisPlacement": "left",
            "axisLabel": "mph",
            "axisColorMode": "text",
            "axisBorderShow": false,
            "scaleDistribution": {
              "type": "linear"
            },
            "axisCenteredZero": false,
            "hideFrom": {
              "tooltip": false,
              "viz": false,
              "legend": false
            },
            "thresholdsStyle": {
              "mode": "off"
            }
          },
          "color": {
            "mode": "palette-classic"
          },
          "mappings": [],
          "thresholds": {
            "mode": "absolute",
            "steps": [
              {
                "color": "green",
                "value": null
              },
              {
                "color": "red",
                "value": 80
              }
            ]
          }
        },
        "overrides": [
          {
            "matcher": {
              "id": "byName",
              "options": "wind_mph"
            },
            "properties": [
              {
                "id": "unit",
                "value": "mph"
              },
              {
```

```
            "id": "custom.axisPlacement",
            "value": "left"
          },
          {
            "id": "custom.axisLabel",
            "value": "mph"
          },
          {
            "id": "min",
            "value": 0
          }
        ]
      },
      {
        "matcher": {
          "id": "byName",
          "options": "gust_mph"
        },
        "properties": [
          {
            "id": "unit",
            "value": "mph"
          },
          {
            "id": "custom.axisPlacement",
            "value": "left"
          },
          {
            "id": "min",
            "value": 0
          }
        ]
      },
      {
        "matcher": {
          "id": "byName",
          "options": "wind_dir_deg"
        },
        "properties": [
          {
            "id": "unit",
            "value": "degrees"
          },
          {
            "id": "custom.axisPlacement",
            "value": "right"
          },
          {
            "id": "custom.axisLabel",
            "value": "°"
          },
          {
```

```
                    "id": "min",
                    "value": 0
                  },
                  {
                    "id": "max",
                    "value": 360
                  }
                ]
              }
            ]
          },
          "pluginVersion": "12.4.0",
          "targets": [
            {
              "refId": "A",
              "editorMode": "code",
              "rawQuery": true,
              "format": "time_series",
              "rawSql": "SELECT\n  time_bucket(make_interval(secs =>
($__interval_ms / 1000)::int), tr.ts) AS \"time\",\n
avg(tr.value) AS \"value\",\n  p.external_id AS \"metric\"\nFROM
timeseries_readings tr\nJOIN points p ON p.id =
tr.point_id\nWHERE $__timeFilter(tr.ts)\n  AND p.site_id::text IN
(${site:sqlstring})\n  AND p.external_id IN
('wind_mph','gust_mph','wind_dir_deg')\nGROUP BY 1,
3\nORDER BY 1, 3;"
            }
          ],
          "datasource": {
            "uid": "openfdd_timescale"
          },
          "options": {
            "tooltip": {
              "mode": "single",
              "sort": "none",
              "hideZeros": false
            },
            "legend": {
              "showLegend": true,
              "displayMode": "list",
              "placement": "bottom",
              "calcs": []
            }
          }
        }
```


76.4.4 Solar Radiation


```
        {
          "id": 23,
          "type": "timeseries",
```

```
"title": "Weather – Solar / Radiation (W/m²)",
"gridPos": {
  "x": 0,
  "y": 19,
  "h": 9,
  "w": 24
},
"fieldConfig": {
  "defaults": {
    "custom": {
      "drawStyle": "line",
      "lineInterpolation": "linear",
      "barAlignment": 0,
      "barWidthFactor": 0.6,
      "lineWidth": 2,
      "fillOpacity": 10,
      "gradientMode": "none",
      "spanNulls": false,
      "insertNulls": false,
      "showPoints": "never",
      "showValues": false,
      "pointSize": 4,
      "stacking": {
        "mode": "none",
        "group": "A"
      },
      "axisPlacement": "left",
      "axisLabel": "W/m²",
      "axisColorMode": "text",
      "axisBorderShow": false,
      "scaleDistribution": {
        "type": "linear"
      },
      "axisCenteredZero": true,
      "hideFrom": {
        "tooltip": false,
        "viz": false,
        "legend": false
      },
      "thresholdsStyle": {
        "mode": "off"
      }
    },
    "color": {
      "mode": "palette-classic"
    },
    "mappings": [],
    "thresholds": {
      "mode": "absolute",
      "steps": [
        {
          "color": "green",
```

```
            "value": null
          },
          {
            "color": "red",
            "value": 80
          }
        ]
      },
      "unit": "wattperm2",
      "min": 0
    },
    "overrides": [
      {
        "matcher": {
          "id": "byName",
          "options": "shortwave_wm2"
        },
        "properties": [
          {
            "id": "min",
            "value": 0
          }
        ]
      },
      {
        "matcher": {
          "id": "byName",
          "options": "direct_wm2"
        },
        "properties": [
          {
            "id": "min",
            "value": 0
          }
        ]
      },
      {
        "matcher": {
          "id": "byName",
          "options": "diffuse_wm2"
        },
        "properties": [
          {
            "id": "min",
            "value": 0
          }
        ]
      },
      {
        "matcher": {
          "id": "byName",
          "options": "gti_wm2"
```

```
                },
                "properties": [
                  {
                    "id": "min",
                    "value": 0
                  }
                ]
              }
            ]
          },
          "pluginVersion": "12.4.0",
          "targets": [
            {
              "refId": "A",
              "editorMode": "code",
              "rawQuery": true,
              "format": "time_series",
              "rawSql": "SELECT\n  time_bucket(make_interval(secs =>
($__interval_ms / 1000)::int), tr.ts) AS \"time\",\n
avg(tr.value) AS \"value\",\n  p.external_id AS \"metric\"\nFROM
timeseries_readings tr\nJOIN points p ON p.id =
tr.point_id\nWHERE $__timeFilter(tr.ts)\n  AND p.site_id::text IN
(${site:sqlstring})\n  AND p.external_id IN
('shortwave_wm2','direct_wm2','diffuse_wm2','gti_wm2')\nGROUP BY
1, 3\nORDER BY 1, 3;"
            }
          ],
          "datasource": {
            "uid": "openfdd_timescale"
          },
          "options": {
            "tooltip": {
              "mode": "single",
              "sort": "none",
              "hideZeros": false
            },
            "legend": {
              "showLegend": true,
              "displayMode": "list",
              "placement": "bottom",
              "calcs": []
            }
          }
        }
```


76.4.5 Cloud Cover


```
        {
          "id": 24,
          "type": "timeseries",
```

```
"title": "Weather — Cloud Cover (%)",
"gridPos": {
  "x": 0,
  "y": 28,
  "h": 7,
  "w": 24
},
"fieldConfig": {
  "defaults": {
    "custom": {
      "drawStyle": "line",
      "lineInterpolation": "linear",
      "barAlignment": 0,
      "barWidthFactor": 0.6,
      "lineWidth": 2,
      "fillOpacity": 35,
      "gradientMode": "opacity",
      "spanNulls": false,
      "insertNulls": false,
      "showPoints": "never",
      "showValues": false,
      "pointSize": 4,
      "stacking": {
        "mode": "none",
        "group": "A"
      },
      "axisPlacement": "left",
      "axisLabel": "Cloud (%)",
      "axisColorMode": "text",
      "axisBorderShow": false,
      "scaleDistribution": {
        "type": "linear"
      },
      "axisCenteredZero": false,
      "hideFrom": {
        "tooltip": false,
        "viz": false,
        "legend": false
      },
      "thresholdsStyle": {
        "mode": "off"
      }
    },
    "color": {
      "mode": "palette-classic"
    },
    "mappings": [],
    "thresholds": {
      "mode": "absolute",
      "steps": [
        {
          "color": "green",
```

```
                "value": null
              },
              {
                "color": "red",
                "value": 80
              }
            ]
          },
          "unit": "percent",
          "min": 0,
          "max": 100
        },
        "overrides": []
      },
      "pluginVersion": "12.4.0",
      "targets": [
        {
          "refId": "A",
          "editorMode": "code",
          "rawQuery": true,
          "format": "time_series",
          "rawSql": "SELECT\n  time_bucket(make_interval(secs =>
($__interval_ms / 1000)::int), tr.ts) AS \"time\",\n
avg(tr.value) AS \"value\",\n  p.external_id AS \"metric\"\nFROM
timeseries_readings tr\nJOIN points p ON p.id =
tr.point_id\nWHERE $__timeFilter(tr.ts)\n  AND p.site_id::text IN
(${site:sqlstring})\n  AND p.external_id IN ('cloud_pct')\nGROUP
BY 1, 3\nORDER BY 1, 3;"
        }
      ],
      "datasource": {
        "uid": "openfdd_timescale"
      },
      "options": {
        "tooltip": {
          "mode": "single",
          "sort": "none",
          "hideZeros": false
        },
        "legend": {
          "showLegend": true,
          "displayMode": "list",
          "placement": "bottom",
          "calcs": []
        }
      }
    }
```

76.5 Faults

76.5.1 Fault Definitions

```
{
  "id": 1,
  "type": "table",
  "title": "Fault Definitions",
  "gridPos": {
    "x": 0,
    "y": 0,
    "h": 10,
    "w": 14
  },
  "fieldConfig": {
    "defaults": {
      "custom": {
        "align": "auto",
        "footer": {
          "reducers": []
        },
        "cellOptions": {
          "type": "auto"
        },
        "inspect": false,
        "hideFrom": {
          "viz": false
        }
      },
      "mappings": [],
      "thresholds": {
        "mode": "absolute",
        "steps": [
          {
            "color": "green",
            "value": null
          },
          {
            "color": "red",
            "value": 80
          }
        ]
      },
      "color": {
        "mode": "thresholds"
      }
    },
    "overrides": []
  },
  "pluginVersion": "12.4.0",
```

```
          "targets": [
            {
              "dataset": "openfdd",
              "datasource": {
                "type": "grafana-postgresql-datasource",
                "uid": "openfdd_timescale"
              },
              "editorMode": "code",
              "format": "table",
              "rawQuery": true,
              "rawSql": "SELECT \r\n  fault_id, \r\n  name, \r\n
category, \r\n  severity,
\r\n  -- Shows what Brick equipment this rule binds to (e.g., AHU,
VAV)\r\n  array_to_string(equipment_types, ', ') AS
target_equipment,
\r\n  -- Shows the raw JSON of the specific Brick points it needs
(e.g., sat, mat)\r\n  inputs AS required_points,\r\n
updated_at\r\nFROM fault_definitions\r\nWHERE
$__timeFilter(updated_at)\r\nORDER BY updated_at DESC;",
              "refId": "A",
              "sql": {
                "columns": [
                  {
                    "parameters": [],
                    "type": "function"
                  }
                ],
                "groupBy": [
                  {
                    "property": {
                      "type": "string"
                    },
                    "type": "groupBy"
                  }
                ],
                "limit": 50
              }
            }
          ],
          "datasource": {
            "type": "grafana-postgresql-datasource",
            "uid": "openfdd_timescale"
          },
          "options": {
            "showHeader": true,
            "cellHeight": "sm"
          }
        }
```

76.5.2 Fault Definition Count


```
        {
          "id": 2,
          "type": "stat",
          "title": "Fault Definition Count",
          "gridPos": {
            "x": 0,
            "y": 15,
            "h": 15,
            "w": 24
          },
          "fieldConfig": {
            "defaults": {
              "mappings": [],
              "thresholds": {
                "mode": "absolute",
                "steps": [
                  {
                    "color": "green",
                    "value": null
                  },
                  {
                    "color": "red",
                    "value": 80
                  }
                ]
              },
              "color": {
                "mode": "thresholds"
              }
            },
            "overrides": []
          },
          "pluginVersion": "12.4.0",
          "targets": [
            {
              "dataset": "openfdd",
              "datasource": {
                "type": "grafana-postgresql-datasource",
                "uid": "openfdd_timescale"
              },
              "editorMode": "code",
              "format": "table",
              "rawQuery": true,
              "rawSql": "SELECT count(*)::int AS \"Fault
definitions\"\r\nFROM fault_definitions;",
              "refId": "A",
              "sql": {
                "columns": [
```

```
            {
              "parameters": [],
              "type": "function"
            }
          ],
          "groupBy": [
            {
              "property": {
                "type": "string"
              },
              "type": "groupBy"
            }
          ],
          "limit": 50
        }
      }
    ],
    "datasource": {
      "type": "grafana-postgresql-datasource",
      "uid": "openfdd_timescale"
    },
    "options": {
      "reduceOptions": {
        "values": false,
        "calcs": [
          "lastNotNull"
        ],
        "fields": ""
      },
      "orientation": "auto",
      "textMode": "auto",
      "wideLayout": true,
      "colorMode": "value",
      "graphMode": "area",
      "justifyMode": "auto",
      "showPercentChange": false,
      "percentChangeColorMode": "standard"
    }
  }
```

77 Grafana dashboards

78 Grafana dashboards (optional)

Grafana is optional. Install with `./scripts/bootstrap.sh --with-grafana`. The React frontend provides equivalent timeseries, fault, and system-resource views. Use Grafana when you want custom SQL dashboards or existing Grafana workflows.

No dashboards are provisioned by default. Only the **datasource** (TimescaleDB, uid: `openfdd_timescale`) is prebuilt.

To build your own dashboards using SQL and the Open-FDD database, see the **Grafana SQL cookbook**.

You can also place dashboard JSON files in `stack/grafana/dashboards/` (or the directory mounted at `/var/lib/grafana/dashboards` in the Grafana container) and restart Grafana to load them.

78.1 React frontend parity

The React UI provides equivalent time-series and stat views for FDD workflows:

- **Trending** — Multi-point time-series (site/equipment/point picker, date range, dual Y-axis). Uses the same `timeseries_readings` data as the Grafana BACnet timeseries dashboard.
- **Faults** — Active fault table (with **Device** and **Sensor/point** columns so you can see which bad sensor was found on which device), fault definitions, fault count (period) stat, and fault flags over time chart (from `fault_results` via `GET /analytics/fault-timeseries` and `GET /analytics/fault-summary`).
- **Plots (trend + fault overlay)** — Same idea as the Grafana "BACnet Plus Fault Data" panel: one chart with point time-series (left axis) and fault data overlaid. In the React UI, select a site, add points and **Add faults**; fault activity is drawn as **vertical colored bands** on the same plot (data from `GET /analytics/fault-timeseries`, i.e. `fault_results`). So fault data does show on the plot in parity with the Grafana version.

- **Overview** — Site cards with fault counts, equipment/points stats, and FDD status.
- **System resources** — A dedicated sidebar page that mirrors the Grafana "System Resources (Host & Containers)" dashboard: host memory used/available, load average (1m/5m/15m), swap used (stat cards with threshold colors), host memory and load time-series charts, container memory (MB) and CPU % charts, **Containers (latest)** table (CPU %, Mem MB, Mem %, PIDs), and **disk usage** per mount (used/free/total GB, % used, progress bar). Data comes from `host_metrics`, `container_metrics`, and `disk_metrics` via `GET /analytics/system/*`. Requires the **host-stats** service to be running so those tables are populated.

If you prefer building custom SQL dashboards in Grafana, use the cookbook and the same tables (`host_metrics`, `container_metrics`, `disk_metrics`, `timeseries_readings`, `fault_results`, etc.).

79 Open-FDD integrity sweep

80 Open-FDD integrity sweep

The integrity sweep is a recurring, operator-style check that verifies Open-FDD platform health, graph integrity, and representative telemetry trustworthiness.

80.1 Cadence

- Daytime recurring sweep target: every 20 minutes.
- Overnight window (`18:00-06:00` local): reduce duplicate low-signal chatter and prioritize richer overnight review artifacts.

80.2 Source-of-truth order

1. Live backend data model (`/data-model/check`, `/data-model/sparql`, `/data-model/export`)
2. Current launch/auth/runtime context
3. Repo docs and playbooks

80.3 Sweep checklist

1. Verify authenticated backend reachability.
2. Validate data model integrity and SPARQL query path.
3. Discover representative points from the current model (not hard-coded point lists).
4. Validate representative BACnet reads for model-derived devices/points.
5. Classify failures cleanly.

80.4 Failure classes

- `auth_config_drift`
- `graph_model_drift`
- `bacnet_device_state_drift`
- `testbench_limitation`
- `likely_openfdd_product_behavior`
- `likely_ui_api_parity_bug`

80.5 Mode detection

- `TEST_BENCH`: fake device semantics and deterministic schedule artifacts present.
- `LIVE_HVAC`: occupied-building semantics and live operational telemetry.

The sweep should always report mode + basis, then reason accordingly.

81 Overnight review

82 Overnight review

Overnight review is the deeper process window for quality and drift detection. It complements daytime integrity sweeps.

82.1 Window

- Recommended local window: `18:00-06:00`.

82.2 Goals

- Catch regressions that only appear over longer runtime windows.
- Distinguish product defects from auth/env/testbench drift.
- Create durable context updates for future OpenClaw clones.

82.3 Expected outputs

- Environment mode and mode basis.
- High-signal changes in BACnet behavior, graph behavior, and fault outputs.
- Candidate docs/playbook updates when operating process improves.
- If durable bench/operator lessons emerge, capture them in versioned notes under `openclaw/references/` so they can later be curated into broader docs.

82.4 Alerting discipline

- Suppress repeated low-signal notices.
- Escalate only meaningful drift or new high-impact findings.
- Convert durable lessons into versioned docs, not chat-only memory.

83 OpenClaw context bootstrap

84 OpenClaw context bootstrap

This page defines how to keep context durable and portable for future OpenClaw sessions and new machines.

84.1 Store in Git (durable)

- Operating philosophy and sweep cadence.
- Failure classification standards.
- SPARQL/query patterns that generalize.
- Operator playbooks and runbooks.
- API workflow contracts and tool usage patterns.

84.2 Keep local/private (do not commit)

- API keys, bearer tokens, `.env` secrets.
- Raw auth stores and private local databases.
- Full raw transcript exports with sensitive state.

84.3 Portability principle

Same tools, any building: repo stores reusable process, while site-specific truth lives in the Open-FDD live model.

84.4 Minimal bootstrap read list for fresh clones

OpenClaw + lab (files in repo — read in order on first session):

1. `openclaw/HANDOFF_PROTOCOL.md` — mailbox handoff with `issues_log.md` and log files.
2. `openclaw/SKILL.md` — agent behavior, bootstrap modes, MCP, security scope.
3. `openclaw/references/testing_layers.md` — where pytest vs bench vs `bootstrap.sh` live.
4. `openclaw/references/legacy_automated_testing.md` — redirect from deprecated **open-fdd-automated-testing** if anything still points there.
5. `openclaw/references/session_status_summary.md` — **5-bullet** lab snapshot format when agents must not dump logs into provider chat.

Product and operations (published docs paths):

1. OpenClaw integration
2. Open-FDD integrity sweep

3. Operator framework
4. AI PR review playbook

AI data modeling (when the stack includes model/API): LLM workflow, AI-assisted tagging, plus `GET /data-model/export` and `PUT /data-model/import` as in OpenClaw integration.

85 Operator framework

86 Operator framework

The canonical machine-readable framework lives at `config/ai/operator_framework.yaml`.

This page explains how to use it.

86.1 Intent

Provide a portable operator reasoning contract for Open-FDD automation:

- source-of-truth ordering
- mode detection (`TEST_BENCH` vs `LIVE_HVAC`)
- seasonal reasoning bias
- failure classification
- overnight improvement loop

86.2 Usage

- Humans: review this page + YAML before changing automation behavior.
- Agents: treat YAML as policy contract and derive site specifics from live Open-FDD model/data.
- Repos: keep this file generic; do not hard-code one-site tribal assumptions.

87 MCP RAG service

88 MCP RAG service

Open-FDD can run an optional MCP-style retrieval service derived from canonical docs.

88.1 Canonical vs derived context

- Canonical human docs: `docs/` and generated `pdf/open-fdd-docs.txt`.
- Derived AI index: `stack/mcp-rag/index/rag_index.json`.

Never edit index artifacts as source-of-truth documentation.

88.2 Bootstrap

Run:

```
./scripts/bootstrap.sh --with-mcp-rag
```

This flow builds docs text when needed, builds retrieval index, and starts the MCP RAG service profile.

For module-focused operations, combine with bootstrap mode:

```
./scripts/bootstrap.sh --mode model --with-mcp-rag
```

88.3 Service endpoints

- `GET /health`
- `GET /manifest`
- `POST /tools/search_docs`
- `POST /tools/get_doc_section`
- `POST /tools/search_api_capabilities`
- `POST /tools/get_operator_playbook`

Optional guarded action tools are present but disabled by default via `OFDD_MCP_ENABLE_ACTION_TOOLS=false`.

89 Mode-aware runbooks

90 Mode-aware runbooks

Use bootstrap mode selection to run only the module you need.

90.1 Engine-only

```
./scripts/bootstrap.sh --mode engine
./scripts/bootstrap.sh --mode engine --test
```

90.2 Model-only

```
./scripts/bootstrap.sh --mode model
./scripts/bootstrap.sh --mode model --test
```

90.3 Collector + Model

```
./scripts/bootstrap.sh --mode collector
./scripts/bootstrap.sh --mode model
```

90.4 Full autonomous stack (OpenClaw + MCP)

```
./scripts/bootstrap.sh --mode full --with-mcp-rag
./scripts/bootstrap.sh --mode full --test
```

91 Testing plan

92 Testing plan

This is the evolving engineering plan for Open-FDD automated testing and the optional **OpenClaw lab** bench under `openclaw/README.md`.

92.1 Near-term priorities

92.1.1 0. Continuous PR and CI review

The OpenClaw workflow should continuously watch active PRs in the same spirit as CodeRabbit:

- detect new commits quickly
- re-check CI and review state
- inspect changed files directly
- run targeted local checks where possible
- write down risks, limitations, and next tests instead of relying on chat memory

See AI PR review playbook.

92.1.2 Daytime short-run counterpart

The overnight suite should have a daytime counterpart that exercises the same major paths without taking the full 12-hour window.

Current recommended shape:

- keep E2E, SPARQL/frontend parity, hot-reload, and BACnet/FDD checks
- use a shorter BACnet scrape profile that stays under 2 hours total
- use this as a daytime confidence pass before the real overnight run

Example:

```
python openclaw/bench/e2e/automated_suite.py \
  --api-url http://HOST:8000 \
  --frontend-url http://HOST \
  --long-run-check-faults \
  --long-run-short-day
```

Or on Windows: `openclaw/windows/run_daytime_short_suite.cmd` (edit URLs first).

92.1.3 1. Keep authenticated backend graph checks healthy

Latest status:

- direct authenticated backend access has been restored on the current bench context

- `GET /data-model/check` and authenticated `POST /data-model/sparql` are now working again

Ongoing risk:

- auth still depends on the real launch context, not just a remembered file location
- future overnight runs can regress back into auth/config drift if the bench shell or Python context loses `OFDD_API_KEY`
- the current `/data-model/sparql` response shape may be `{"bindings": [...]}` in some environments, so tooling should not assume strict SPARQL JSON `results.bindings`

Action:

- keep verifying that `OFDD_API_KEY` is available to the actual automated testing environment before overnight runs
- make SPARQL-parsing code tolerant to both `bindings` and `results.bindings`
- keep treating backend auth failure as a pre-overnight readiness issue, not a product regression by default

Why it matters:

- without authenticated SPARQL/API access, BACnet graph validation is incomplete
- without response-shape tolerance, a healthy graph can be misclassified as empty

92.1.4 2. Promote BACnet addressing to a first-class validation target

We need to explicitly validate:

- BACnet devices in the graph
- device instance and address visibility
- object identifiers for polling points
- semantic equipment type for those points

This is no longer optional background metadata. It is core operational context.

92.1.5 3. Prove fault calculation from end to end

The target standard is:

- fake BACnet device fault schedule is known
- DIY BACnet server RPC confirms source values
- Open-FDD data-model SPARQL queries confirm BACnet devices and point addressing
- Open-FDD scrape path receives those values
- YAML rules + rolling windows predict an expected fault
- Open-FDD fault outputs show that exact fault
- the overnight process writes a durable report

See BACnet-to-fault verification and the report template at `openclaw/reports/overnight-bacnet-verification-template.md` in the repo (not on GitHub Pages).

92.1.6 4. Preserve reusable context for future clones

The repo should keep visible documentation and machine-readable operator policy for:

- the operational states
- overnight review discipline
- BACnet graph context
- portability assumptions
- future optimization intent
- documentation guidance that works for both humans and AI agents

92.1.7 5. Add nightly docs and link review

The overnight workflow should also:

- validate important README and docs links
- make sure link checking is done against the correct target branches
- treat `master` as the primary target branch
- optionally check one active development branch when it is the intended docs destination for unreleased fixes
- avoid mixing findings from unrelated feature branches
- identify missing docs pages or thin areas
- suggest documentation improvements that make the system easier for both humans and AI agents to understand
- record those suggestions in durable repo docs or review notes

92.2 Future role in live HVAC systems

In a live HVAC deployment, the same testing and validation assets should support:

- confidence in FDD outputs
- confidence in model/rule applicability
- future optimization and supervisory logic
- operator- or facility-manager-facing monitoring summaries

The repo is not only a test harness. It is becoming a reproducible engineering context pack.

93 Continuous context backup loop

94 Continuous context backup loop

This is the non-site-specific process for making the virtual operator improve over time without turning the repo into one building's notebook.

94.1 What gets backed up

Back up only **durable, reusable process knowledge**:

- operator heuristics
- failure classification rules
- dashboard expectations
- overnight review discipline
- SPARQL/query patterns that discover site context from the model
- how to separate product bugs from setup/auth drift

94.2 What does not get backed up

Do not back up:

- bearer keys or API keys
- raw `.env` contents
- device pairing secrets
- local SQLite conversation history
- one-off private notes that should stay local
- hard-coded site assumptions that belong in the Open-FDD model instead

94.3 Trigger for an update

During overnight review, update the repo when any of these are true:

- the process changed materially
- the operator framework got smarter
- a failure pattern should be remembered by future clones
- a new query pattern or heuristic is now part of normal operation
- the dashboard or alert contract changed

94.4 Update sequence

1. Distill the lesson from local context
2. Put it into the best durable home:
   - docs
   - query template under `openclaw/bench/sparql/README.md`
   - YAML policy file (e.g. `config/ai/operator_framework.yaml`)
3. Commit the change professionally
4. Push to GitHub
5. Rebuild/publish the docs PDF when the docs set changed materially

94.5 Why this matters

This makes the OpenClaw/Open-FDD operator behave more like a continuously improving engineering system and less like a chat session with amnesia.

The repo becomes the transferable brainstem. The live Open-FDD model remains the site-specific truth.

95 Usage budget and throttling

96 Usage budget and throttling

Open-FDD automation and OpenClaw-assisted workflows should behave responsibly when model/API budget is tight.

That applies to:

- OpenClaw chat usage
- Codex / OpenAI model budget
- overnight review loops
- daytime smoke runs
- every-20-minute integrity sweeps
- weather/support API calls

96.1 Goal

Do not let automation burn expensive model budget on low-value repetition.

When usage pressure is high, the system should degrade gracefully:

- keep the most operator-relevant checks
- reduce frequency of expensive AI/browser work
- preserve durable findings to GitHub/docs
- stay useful without going silent too early

96.2 Practical throttle ladder

96.2.1 Level 0 — Normal

Use the full intended workflow:

- 20-minute integrity sweep
- daytime smoke when requested
- full overnight review
- browser parity when it adds real value
- docs/PDF rebuilds on meaningful changes

96.2.2 Level 1 — Moderate conservation

Use when model budget is healthy but burn rate is rising.

Actions:

- keep the 20-minute sweep concise
- avoid repeating large narrative summaries in chat
- batch docs updates into fewer larger commits
- prefer one focused rerun over repeated full-suite reruns
- avoid unnecessary weather fetches when test-bench mode is already obvious

96.2.3 Level 2 — High conservation

Use when account/session budget is tight or approaching a hard cap.

Actions:

- keep the 20-minute sweep to auth + graph + one or two model-derived BACnet checks
- suppress low-signal chatter
- avoid broad browser parity passes unless specifically needed for release confidence
- prefer direct backend/API checks before UI replays
- do not rerun full daytime smoke or overnight review repeatedly for the same known failure mode
- write the failure classification once, then work the fix

96.2.4 Level 3 — Survival mode

Use when usage is critically constrained or a weekly cap is effectively exhausted.

Actions:

- only run high-signal checks
- prefer local/backend checks over browser/UI checks
- skip optional weather/history enrichments
- skip long narrative output
- skip docs/PDF rebuilds unless the change is critical
- notify the human clearly that budget pressure is affecting coverage

96.3 Priority order under throttling

If budget is tight, preserve this order:

1. auth verification
2. backend reachability
3. data-model / SPARQL integrity
4. BACnet readability
5. high-signal product regressions
6. browser parity extras
7. weather/context enrichments
8. long prose summaries

96.4 Applying this to the Open-FDD workflows

96.4.1 Every-20-minute integrity sweep

Under pressure:

- keep it short
- use model-derived checks only
- do not repeat the same alert every cycle
- if a failure is already known and unchanged, stay quiet unless the state materially worsens or recovers

96.4.2 Daytime smoke

Under pressure:

- prefer targeted reruns instead of full reruns
- fix the first blocker before repeating the whole suite
- use the short-day BACnet profile (`openclaw/bench/e2e/README.md`)

96.4.3 Overnight review

Under pressure:

- keep the overnight run focused on evidence gathering and high-signal failures
- avoid chat spam

- distill durable lessons into docs once instead of narrating every small step

96.5 What can be inferred from usage tools

Useful signals include:

- session/model usage status
- visible Codex/OpenAI usage dashboard state
- repeated provider failures or hard-cap symptoms
- evidence that a task is re-spending tokens on the same known issue

The system should use those signals to reduce repeated model-heavy work.

96.6 Important limitation

OpenClaw can adapt its own behavior, but it does not directly control provider billing/quotas.

So the right design is:

- observe usage pressure
- throttle the workflow behavior
- preserve the most valuable engineering checks
- keep the human informed when coverage is reduced by budget pressure

97 Operations

98 Operations

Operational playbooks for recurring Open-FDD verification, OpenClaw context durability, and operator-style review loops.

Page	Description
Open-FDD integrity sweep	20-minute operator-style health and model integrity checks.

Page	Description
Overnight review	6 PM-6 AM deeper review pattern for logs, behavior drift, and durable context updates.
OpenClaw context bootstrap	What durable context should be stored in-repo vs kept local/private.
Operator framework	Canonical operator reasoning contract and source-of-truth order.
MCP RAG service	Derived AI retrieval service from canonical docs + generated docs text.
Mode-aware runbooks	Collector/model/engine/full orchestration flows for humans and OpenClaw.
Testing plan	Engineering plan for bench testing, auth/SPARQL checks, and overnight evidence.
Continuous context backup loop	What to commit to the repo vs keep local (no secrets).
Usage budget and throttling	Throttle ladder when model/API budget is tight.

99 Configuration

100 Configuration

100.1 Platform config (RDF + CRUD)

Platform config lives in the **same RDF graph** as Brick and BACnet (`config/data_model.ttl`). No YAML file. **Where:** Bootstrap seeds via PUT /config; API GET/PUT /config and POST /data-model/sparql use the graph. The **entire app is bootstrapped from this data model** (sites, equipment, points, and platform config such as `ofdd:rulesDir`, `ofdd:bacnetScrapeIntervalMin`, etc.). **`rules_dir` (ofdd:rulesDir) remains required**: it is the path where FDD rule YAML files are stored; the frontend upload/download UI manages files *in* that path and does not replace the need for the path itself. **Individual rule YAML files are**

not stored in the data model—only the single directory path (e.g. `ofdd:rulesDir "stack/rules"`) is; the files themselves live on disk under that path.

- **Bootstrap:** `./scripts/bootstrap.sh` seeds config via PUT /config (defaults or `OFDD_*` from `stack/.env`).
- **API:** GET /config, PUT /config; query via POST /data-model/sparql.
- **Env at seed:** Set `OFDD_*` in `stack/.env` before first bootstrap to customize the seed. **Docker:** Set env in `stack/docker-compose.yml` or `stack/.env`; bootstrap uses these when calling PUT /config.

Rename / multiple configs: Use different env files (e.g. `platform-prod.env`, `platform-building-a.env`) and start with `docker compose --env-file platform-building-a.env up -d`, or set `OFDD_*` in that file. No built-in "config name" selector; use env files or env vars per deployment.

100.2 Platform keys (RDF / GET-PUT /config)

{: #platform-keys-config }

Example keys (GET/PUT /config or OFDD_* at bootstrap seed):

Key	Default	Description
`rule_interval_hours`	3	FDD loop run interval (when no trigger file: run every N hours)
`lookback_days`	3	Days of data loaded per run (each run pulls last N days into the rule engine)
`rules_dir`	stack/rules	**Single directory for your rules** (hot reload)
`brick_ttl_dir`	—	Optional. Directory containing Brick model TTL (e.g. `config/`); platform uses first `.ttl` or brick_ttl path for FDD column mapping. Optional if using points `brick_type`/fdd_input. See Data modeling.
`bacnet_enabled`	true	Enable BACnet scraper
`graph_sync_interval_min`	5	Minutes between serializing the **full** in-memory graph to

Key	Default	Description
		`data_model.ttl` (API background thread). That write runs `sync_brick_from_db` first, so Brick **`ref:`** external references reflect the DB at serialize time. Env: `OFDD_GRAPH_SYNC_INTERVAL_MIN`. Edit in the React app under **OpenFDD Config** (not Overview). For an immediate refresh after discovery or DB edits, use **Data model → Serialize to TTL** or GET `/data-model/ttl` with save.
`bacnet_scrape_interval_min`	5	Poll interval (minutes)
`bacnet_site_id`	default	Site to tag when scraping (use on **remote gateways** so data is attributed to the right building on the central DB)
`bacnet_gateways`	—	Optional. **Central aggregator:** JSON array of `{url, site_id}`; scraper polls each remote diy-bacnet-server in turn. Env: `OFDD_BACNET_GATEWAYS`.
`open_meteo_enabled`	true	Enable weather; when true, **FDD loop runs a weather fetch at the start of each run** (same cadence as rules, every `rule_interval_hours`).
`open_meteo_interval_hours`	24	**Standalone weather-scraper only.** Poll interval (hours) when using the optional weather-scraper container. Ignored when weather is run from the FDD loop.
`open_meteo_latitude`	41.88	Site latitude
`open_meteo_longitude`	-87.63	Site longitude
`open_meteo_timezone`	America/Chicago	Timezone
`open_meteo_days_back`	3	

Key	Default	Description
		Standalone weather-scraper only. Days of archive to fetch per run. When weather runs with FDD, a 1-day lookback is used (no separate config).
`open_meteo_site_id`	default	Site ID for weather points

Weather fetch: Weather is normally fetched **with each FDD run** (same interval as fault rules, e.g. every 3 h), using a **1-day lookback** so the API is not over-used. A standalone weather scraper is only for setups that do not run the FDD loop; do not run both to avoid redundant fetches. The standalone scraper reads config from **GET /config** (like the BACnet scraper) when available.

Where rules live: Platform config includes **`rules_dir`** (e.g. `stack/rules`). This path is **required**: the FDD loop and the rules API both use it. Rule YAML files are stored there; you can manage them in two ways. (1) **React frontend (Faults page):** upload, download, delete YAML and **Sync definitions** so the fault_definitions table updates without waiting for the next FDD run. (2) **Files on disk:** edit or add files under the configured path (e.g. `stack/rules/`). Both target the same `rules_dir`; the frontend is the preferred path when you have UI access. See Fault rules overview and the Expression Rule Cookbook.

Hot reload (AFDD tuning): The FDD loop loads rules from `rules_dir` on every run (no cache). Edit YAML on disk or upload via the frontend → trigger a run (or wait for the schedule) or click **Sync definitions** in the UI → results and the definitions table reflect the new rules. The `rules_dir` path is **RDF-driven** (GET/PUT `/config`, same graph as the rest of platform config). Unit tests: `open_fdd/tests/platform/test_rules_loader.py`, `open_fdd/tests/platform_api/test_rules.py`.

Rolling window (per rule): Set `params.rolling_window` in each rule YAML; see Fault rules for HVAC.

100.3 Model context endpoint (external agents)

Open-FDD can serve its own documentation as plain-text model context for external AI agents (for example an OpenAI-compatible tool like Open-Claw).

The endpoint is `GET /model-context/docs`.

By default it returns a truncated excerpt of `pdf/open-fdd-docs.txt` (or the file pointed to by `OFDD_DOCS_PATH` if set). If you need specific sections, pass `query=...` for keyword retrieval and control output size with `max_chars`.

Open-FDD does not embed or run an LLM; you supply the LLM provider externally.

Security note: when `OFDD_API_KEY` is enabled, this endpoint requires Bearer auth like other API routes.

100.4 Services that read config from the API (BACnet scraper)

Platform config (e.g. **Scrape interval (min)**) is stored in the data model and served by **GET /config**. Some services need to call the API to get that config so changes in the Config UI take effect.

BACnet scraper and **weather scraper (standalone):** Both can read config from **GET /config** (with a short cache) so the Config UI and data model control their behaviour. The BACnet scraper uses `bacnet_scrape_interval_min`; the standalone weather scraper uses `open_meteo_enabled`, `open_meteo_interval_hours`, `open_meteo_days_back`, and the geo/site fields. If GET /config fails (e.g. no `OFDD_API_KEY`), they fall back to env vars.

BACnet scraper: The scraper runs on a fixed interval (e.g. every 1, 5, or 10 minutes). It can get that interval in two ways:

1. **From the API (dynamic)** — The scraper calls **GET /config** (with a short cache). It then uses `bacnet_scrape_interval_min` from the response, so whatever you set in the Config UI is what the scraper uses. **This only works when the scraper can authenticate:** the API requires Bearer auth when `OFDD_API_KEY` is set, so the scraper

must have **`OFDD_API_KEY`** in its environment (same value as in `stack/.env`). The stack's `docker-compose.yml` passes `OFDD_API_KEY: ${OFDD_API_KEY:-}` to the bacnet-scraper service for this reason.
2. **Fallback (env)** — If GET /config fails (e.g. 401 because no API key, or API unreachable), the scraper falls back to **`OFDD_BACNET_SCRAPE_INTERVAL_MIN`** from its environment (e.g. 5 in compose). **So if you set an interval in the Config UI but the scraper does not have `OFDD_API_KEY`, it will ignore the UI and keep using the env default.** Rebuild/restart the bacnet-scraper after ensuring `OFDD_API_KEY` is in `stack/.env` and in the scraper's env: `./scripts/bootstrap.sh --build bacnet-scraper`.

Summary: For the Config UI (and data model) to control the BACnet scrape interval, the API must have `OFDD_API_KEY` set and the bacnet-scraper container must receive the same key so it can call GET /config successfully.

100.5 BACnet: single gateway, remote gateways, central aggregator

- **Single building (or one remote gateway):** Set `OFDD_BACNET_SERVER_URL` and optionally `OFDD_BACNET_SITE_ID` (e.g. `building-a`). The scraper tags all readings with that site. On a **remote** gateway (diy-bacnet-server + scraper on another subnet), point `OFDD_DB_DSN` at the central Open-FDD DB and set `OFDD_BACNET_SITE_ID` to the site name/UUID used on the central API so data is attributed to that building.
- **Central aggregator (multiple remote gateways):** On the central host, do **not** run local bacnet-server/bacnet-scraper; run only DB, API, Grafana, FDD loop, and (optional) weather. Set `OFDD_BACNET_GATEWAYS` to a JSON array, e.g. `[{"url":"http://10.1.1.1:8080","site_id":"building-a"},{"url":"http://10.1.2.1:8080","site_id":"building-b"}]`, and run one scraper container (or cron) that polls each URL and writes to the central DB with the given `site_id`. Alternatively, deploy one scraper per building elsewhere, each with `OFDD_DB_DSN=central` and `OFDD_BACNET_SITE_ID=<that building>`.

Open-Meteo weather points (stored in `timeseries_readings`, `external_id`):

Point	Description
`temp_f`	Temperature (°F)
`rh_pct`	Relative humidity (%)
`dewpoint_f`	Dew point (°F)
`wind_mph`	Wind speed (mph)
`gust_mph`	Wind gusts (mph)
`wind_dir_deg`	Wind direction (degrees)
`shortwave_wm2`	Shortwave radiation (W/m²)
`direct_wm2`	Direct radiation (W/m²)
`diffuse_wm2`	Diffuse radiation (W/m²)
`gti_wm2`	Global tilted irradiance (W/m²)
`cloud_pct`	Cloud cover (%)

In the **data model** and **Points** UI these appear under equipment **Open-Meteo** (type **Weather_Service**); the RDF graph marks that equipment with `ofdd:dataSource "open_meteo"`.

100.6 Edge / resource limits

For edge deployments with limited disk, set these at bootstrap (or in `stack/.env`). See Getting Started — Bootstrap options.

Setting	Default	Bootstrap arg	Env (stack/.env)
Data retention	365 days	`--retention-days N`	`OFDD_RETENTION_DAYS`
Docker log size	100m per file	`--log-max-size SIZE`	`OFDD_LOG_MAX_SIZE`
Docker log files	3	`--log-max-files N`	`OFDD_LOG_MAX_FILES`

Data retention: TimescaleDB drops chunks older than the configured interval from `timeseries_readings`, `fault_results`, `host_metrics`, `container_metrics`. Set at first run, e.g. `./scripts/bootstrap.sh --retention-days 180`, or set `OFDD_RETENTION_DAYS` in `stack/.env` before running bootstrap.

Log rotation: Containers use `json-file` driver; size and file count come from the table above. Restart the stack after changing so new values apply.

Accessing logs: `docker logs <container> --tail 50` (e.g. `openfdd_weather_scraper`, `openfdd_bacnet_scraper`).

100.7 Environment

`OFDD_`-prefixed vars override YAML:

Variable	Description
`OFDD_DB_HOST`	TimescaleDB host
`OFDD_DB_PORT`	TimescaleDB port
`OFDD_DB_NAME`	Database name
`OFDD_DB_USER`	Database user
`OFDD_DB_PASSWORD`	Database password
`OFDD_BACNET_SERVER_URL`	diy-bacnet-server base URL (e.g. http://localhost:8080)
`OFDD_BACNET_SITE_ID`	Site to tag when scraping (default: default; use on remote gateways)
`OFDD_BACNET_GATEWAYS`	JSON array of {url, site_id} for central aggregator
`OFDD_RULES_DIR`	Rules directory (default: stack/rules)

101 Contributing

102 Contributing to Open-FDD

First off, thanks for taking the time to contribute!

Open-FDD is in **Alpha**. The most valuable contributions right now are **bug reports** and **FDD rules**—especially from mechanical engineers and building professionals who can add and refine fault-detection rules using the Expression Rule Cookbook. All types of contributions are encouraged.

Phase focus: Alpha emphasizes platform stability, driver implementation beyond BACnet, and API changes for specific integration needs. Beta (planned) will focus more on Brick data modeling (e.g. ASHRAE 223P), mechanical engineering and consulting input into the expression rule cookbook, and better default Grafana dashboards for HVAC analytics. See the Table of Contents for different ways to help and how this project handles them. Please read the relevant section before contributing; it helps maintainers and keeps things smooth for everyone.

> If you like the project but don't have time to contribute, that's fine. Other ways to support it: - Star the repository - Share it with colleagues or at meetups - Refer to Open-FDD in your project's readme or documentation

---

102.1 Table of Contents

- Code of Conduct
- I Have a Question
- I Want To Contribute
- Reporting Bugs
- Suggesting Enhancements
- Contributing Rules (Expression Rule Cookbook)
- Your First Code Contribution
- Improving the Documentation
- Styleguides
- Git workflow (branch → PR → sync)
- Commit Messages
- Join the Project Team

102.2 Code of Conduct

This project expects everyone to be respectful and constructive. By participating, you agree to uphold a positive environment. Please report unacceptable behavior by opening an issue or contacting the repository maintainers.

102.3 I Have a Question

Before asking, please check the documentation and search existing issues to see if your question is already answered.

If you still need help:

- Open an issue.
- Provide as much context as you can (what you're trying to do, what you ran, what happened).
- Include relevant versions: Python, Docker, OS (e.g. Ubuntu, Linux Mint), and Open-FDD commit or release if known.

We'll respond as soon as we can.

102.4 I Want To Contribute

102.4.1 Legal notice

When contributing, you agree that you have authored 100% of the content, have the necessary rights to it, and that your contribution may be provided under the project license (MIT).

102.5 Reporting Bugs

Bug testing is a top priority in Alpha. Good bug reports help us fix issues quickly.

102.5.1 Before submitting a bug report

- Use the latest version (main branch or latest release).

- Confirm the bug is in Open-FDD and not in your environment (e.g. wrong Python version, missing config). Check the documentation and I Have a Question first.
- Search issues to see if the bug is already reported.
- Collect:
  - **Stack trace** (Traceback) if applicable
  - **OS and platform** (e.g. Linux, macOS, Windows; x86, ARM)
  - **Versions**: Python, Docker, Docker Compose, and how you ran Open-FDD (pip, Docker Compose, etc.)
  - **Steps to reproduce** and, if possible, sample input/config
  - Whether you can reproduce it every time and on an older version

102.5.2 How to submit a good bug report

- **Security issues:** Do **not** report security vulnerabilities in public issues. Email the repository owner or open a private security advisory on GitHub.
- Open a new issue. Don't assume it's a bug yet—avoid using the word "bug" in the title until it's confirmed.
- Describe **expected behavior** vs **actual behavior**.
- Provide **reproduction steps** so someone else can recreate the issue. Isolate the problem when possible (e.g. minimal rule YAML, minimal config).
- Paste the information you collected above.

After you file:

- Maintainers will label the issue. If we can't reproduce it, we may ask for more steps and mark it `needs-repro`. Reproducible issues may be marked `needs-fix` and opened for implementation.

102.6 Suggesting Enhancements

Enhancements are tracked as GitHub issues.

102.6.1 Before submitting

- Use the latest version and read the documentation to see if the behavior already exists or can be configured.
- Search issues to see if the enhancement was already suggested; if so, add to that discussion.

- Consider whether the idea fits Open-FDD's scope (edge AFDD, rules, API, BACnet, Grafana). Make a clear case for why it would help most users.

102.6.2 How to submit a good enhancement suggestion

- Use a **clear, descriptive title**.
- Describe the **current behavior** and what you **want instead**, and why.
- Provide **step-by-step detail** where useful; screenshots or example YAML/config can help.
- Explain why this would be useful to the community.

102.7 Contributing Rules (Expression Rule Cookbook)

We especially welcome contributions from mechanical engineers and building professionals who can add or improve FDD rules.

- **Where rules live:** Fault rules overview — put project rules in **`stack/rules/`** (YAML). The FDD loop reloads them every run; no restart needed.
- **How to write rules:** Expression Rule Cookbook — expression-type rules use YAML with BRICK-style inputs, params, and pandas/NumPy expressions. The cookbook includes AHU-style rules (e.g. GL36-inspired) and patterns you can adapt.
- **What to contribute:**
  - **New rule YAMLs** for common faults (AHU, VAV, plant, sensors) that others can reuse.
  - **Improvements to existing cookbook rules** (thresholds, logic, descriptions).
  - **New cookbook sections or examples** (e.g. for a specific equipment type or standard).

To contribute a rule or cookbook change:

1. Add or edit files under `stack/rules/` for new/updated rules, or under `docs/expression_rule_cookbook.md` (and related docs) for cookbook content.
2. Follow the existing YAML style and BRICK naming used in the cookbook.

3. In a PR, describe the fault the rule targets and any assumptions (e.g. Brick types, units). If it's from a standard (e.g. ASHRAE), note the reference.

102.8 Your First Code Contribution

- Look for issues labeled `good first issue` or `help wanted` (if we've added them).
- The codebase is Python 3.11+, with FastAPI (API), pandas/NumPy (rules), and Docker for the platform. See Getting Started and the README for setup.
- For rule changes, run the test suite and, if possible, the FDD loop locally with sample data before submitting a PR.

102.9 Improving the Documentation

Documentation lives in the `docs/` directory and is published at bbartling.github.io/open-fdd (Just the Docs theme). Improvements are welcome:

- Fix typos, clarify wording, or update steps.
- Add examples or how-tos (e.g. for the Expression Rule Cookbook, How-to Guides, or BACnet).
- Keep code blocks and links in sync with the codebase.

Building a PDF: To generate a single PDF of the docs (e.g. for offline use or to push to the repo), run `python3 scripts/build_docs_pdf.py`. Requires pandoc and either **weasyprint** (`pip install weasyprint`) or LaTeX. Output is `pdf/open-fdd-docs.pdf` by default; use `--output path` to change it. You can commit and push the PDF if you want it in the repo.

Open a PR with your changes; for large edits, an issue first can help align with maintainers.

After changing README or high-traffic doc links, spot-check the live site (GitHub Pages builds extensionless URLs like `/modeling/llm_workflow`; the same path with a **trailing slash** often 404s because Jekyll emits `page.html`, not `page/index.html`):

```
BASE=https://bbartling.github.io/open-fdd
curl -sS -o /dev/null -w "%{http_code}" "$BASE/modeling/
```

```
llm_workflow"    # expect 200
        curl -sS -o /dev/null -w "%{http_code}" "$BASE/modeling/
llm_workflow/"   # often 404 — avoid trailing slash in external
links
        curl -sS -o /dev/null -w "%{http_code}" "https://github.com/
bbartling/open-fdd/blob/master/pdf/open-fdd-docs.pdf"  # expect
302 or 200
```

102.10 Styleguides

- **Python:** We use Black for formatting. Run `black` on changed files before committing.
- **YAML:** Match the style in `stack/rules/` and the expression rule cookbook (indentation, key order).
- **Markdown:** Use clear headings and lists; link to existing docs where relevant.

102.11 Git workflow (branch → PR → sync)

Create the branch **before** committing so that after the PR is merged, `git pull` on `main`/`master` fast-forwards and you avoid divergent-branch errors (protected `main`/`master`).

1. **Create a branch**

```
git checkout -b feature/short-name
```

2. **Stage, commit, and push**

```
git add .
git commit -m 'short note describing the change'
git push -u origin feature/short-name
```

3. **Open a PR manually on GitHub** from `feature/short-name` into `main` (or `master`). Create the PR in the GitHub UI; we do not use a CLI for this step.

4. **After the PR is merged** — fetch prune, switch to default branch, and pull to sync:

```
git fetch --prune
git checkout main
git pull
```

Use `master` instead of `main` if that is your default branch. If you ever committed on `main`/`master` before creating the branch, local and remote can diverge after the merge; run `git reset --hard origin/main` (or `origin/master`) to match the remote.

102.12 Commit Messages

- Use a clear, short summary line. Optionally add a body with details.
- Reference issues/PRs when relevant (e.g. `Fix bounds rule when input is all NaN (#123)`).

102.13 Join the Project Team

If you're interested in ongoing maintenance or a larger role, say so in an issue or reach out to the maintainers. We're especially interested in folks who can triage bugs, review rule contributions, or improve docs and the cookbook.

102.14 Attribution

This contributing guide was inspired by the contributing.md project. You don't need to pay to create or use a CONTRIBUTING file—it's just a markdown file in your repo that helps contributors and maintainers.

103 API Reference

104 API Reference

The Open-FDD platform exposes a **REST API** (port 8000) for CRUD, config, data model, bulk download, analytics, BACnet proxy, faults, and jobs. **Use the React frontend** (http://localhost:5173) for normal workflows—sites, points, config, faults, plots. Use the API when integrating scripts or cloud export.

Interactive docs: When the API is running, open **Swagger UI** at http://localhost:8000/docs or **ReDoc** at http://localhost:8000/redoc. Full **OpenAPI 3.1** spec: http://localhost:8000/openapi.json. When `OFDD_API_KEY` is set (e.g. in `stack/.env`), click **Authorize** in Swagger and paste the key so **Try it out** works.

104.1 High-level overview

Area	Purpose
CRUD	**Sites** — list, create, get, update, delete (cascade to equipment, points, timeseries, fault results). **Equipment** — list, create, get, update, delete (cascade to points). **Points** — list, create, get, update, delete (cascade to timeseries). Points can include `bacnet_device_id` and `object_identifier` so the BACnet scraper polls them.
Config	**GET/PUT /config** — platform config (rule interval, BACnet scrape interval, Open-Meteo, etc.) stored in the RDF graph. The React Config page uses these.
Data model	**GET /data-model/export** — BACnet discovery + DB points (for LLM tagging). **PUT /data-model/import** — bulk create/update points and equipment. **GET /data-model/ttl** — Brick + BACnet TTL. **POST /data-model/sparql** — run SPARQL. **POST /data-model/reset** — reset graph to DB-only.
Download	**GET/POST /download/csv** — timeseries CSV (wide or long). **GET /download/faults** — fault results (CSV or JSON). Excel-friendly; timestamps in UTC (Z).
Analytics	**GET /analytics/fault-summary**, **GET /analytics/fault-timeseries** — fault counts and time-series for charts. **GET /analytics/motor-runtime** — fan/VFD runtime (data-model driven). **GET /analytics/system/*** — host/container/disk metrics.

Area	Purpose
BACnet	**GET /bacnet/gateways** — list gateways. **POST /bacnet/server_hello**, **POST /bacnet/whois_range**, **POST /bacnet/point_discovery**, **POST /bacnet/point_discovery_to_graph** — proxy to diy-bacnet-server; discovery-to-graph feeds the data model. **POST /bacnet/write_point** — write value (audited; third-party integrations use this for audited writes).
Faults	**GET /faults/active**, **GET /faults/state**, **GET /faults/definitions** — active state, full state, definitions.
Jobs	**POST /jobs/bacnet/discovery**, **POST /jobs/fdd/run** — async BACnet discovery and FDD run. **GET /jobs/{job_id}** — status.
Run FDD	**POST /run-fdd** — trigger FDD run now. **GET /run-fdd/status** — last run.
Model context	**GET /model-context/docs** — serve Open-FDD documentation as plain-text model context for external agents (supports excerpt/full/slice and keyword retrieval via `query`).
MCP-style discovery	**GET /mcp/manifest** — JSON manifest listing logical resources (e.g. `openfdd://docs`) and HTTP mappings for common "tools" (docs, export, import, capabilities). Not a full MCP JSON-RPC server; see Open-Claw integration.

Base URL: `http://localhost:8000` (use your host or IP when remote).
Auth: When `OFDD_API_KEY` is set, send `Authorization: Bearer <key>`; the React frontend does this when built with `VITE_OFDD_API_KEY`.

104.2 MCP manifest (`GET /mcp/manifest`)

Agents can use this endpoint to **discover** stable URLs without hand-copying from Swagger:

- **Resources**: e.g. documentation as `openfdd://docs` → `GET /model-context/docs`
- **Tools**: JSON metadata with `http.method` and `http.path` for docs fetch, data-model export/import, and `GET /capabilities`

When `OFDD_API_KEY` is set, the manifest endpoint is protected like other routes (Bearer required).

104.3 Model context docs endpoint

Open-FDD can serve its documentation as plain text model context via:

- `GET /model-context/docs`

This endpoint is designed for external LLM agents (OpenAI-compatible providers like Open-Claw). Open-FDD does not embed or run an LLM; it only provides documentation context.

By default, `mode=excerpt` returns a truncated excerpt of `pdf/open-fdd-docs.txt` (or `OFDD_DOCS_PATH` if set). For more control:

- `mode=full` returns the entire file.
- `mode=slice&offset=...` returns a substring.
- `query=...` returns keyword-retrieved relevant doc sections (simple lexical matching) to help keep prompts smaller.

When `OFDD_API_KEY` auth is enabled, this endpoint requires Bearer auth like other API routes.

105 AI PR review playbook

106 AI PR review playbook

Use this checklist when an AI agent reviews Open-FDD pull requests.

106.1 Priority order

1. Behavior regressions and runtime risks.
2. Auth/security exposure or secret handling mistakes.
3. Data-model/API contract breaks.
4. Missing tests for changed behavior.
5. Documentation drift for operationally meaningful changes.

106.2 Review output contract

- Findings first, ordered by severity.
- Include concrete file references and reproducible rationale.
- Keep summary brief after findings.
- If no findings, state residual risks/testing gaps explicitly.

107 Appendix

108 Appendix

Technical deep dives and maintainer reference material.

Page	Description
API Reference	REST API at a glance: CRUD, config, data model, download, analytics, BACnet, faults, jobs. Full spec at Swagger /docs and /openapi.json.
Technical reference	Directory structure, environment variables, unit tests, BACnet scrape, bootstrap, database schema, LLM tagging workflow.
Developer guide	Front-end development, database schema (migrations, tables, cascade deletes), where to go next.
AI PR review playbook	Review checklist focused on regressions, risks, and missing validation before merge.

109 Cloning and porting

110 Cloning and porting

The **open-fdd** repo should be portable to another lab, another workstation, or another Open-FDD deployment with minimal conceptual changes. Optional bench assets live under `openclaw/README.md`.

110.1 Core portability idea

Same tools, any building — only the knowledge graph changes.

That means the repo carries the reusable process, while the live Open-FDD model carries the site-specific truth.

110.2 What should transfer cleanly

- the test phases (`./scripts/bootstrap.sh --test` and optional `openclaw/bench/e2e/README.md`)
- the BACnet fake-device approach (`openclaw/bench/fake_bacnet_devices/README.md`)
- the overnight review discipline
- the SPARQL validation set (`openclaw/bench/sparql/README.md`)
- the operator framework (`config/ai/operator_framework.yaml`)
- the continuous context-backup loop
- the idea of proving telemetry-to-fault correctness rather than only checking page loads

110.3 What usually changes per environment

- frontend URL
- API URL
- API auth setup
- site IDs / names
- BACnet gateway hostnames or IPs

- active Open-FDD rules directory (under `stack/`, mounted into containers)
- Docker/container naming
- LAN / OT network topology
- the actual HVAC system, naming conventions, and semantic model shape
- SPARQL queries or filters needed for that environment

110.4 What to do when deploying to another site

1. Resolve the target frontend/backend/BACnet endpoints from the real launch context.
2. Confirm auth works from the shell or runtime that will actually run the checks.
3. Query the Open-FDD model first:
    - sites
    - equipment
    - BACnet devices
    - representative outdoor / plant / air / zone points
4. Let the model decide what should be checked at that site.
5. Keep repo docs generic; put site-specific truth into the Open-FDD model instead of hard-coding it into Markdown.

110.5 Recommended first-pass deployment flow for a new building

Use this order on a fresh site:

- verify backend auth and reachability
- run SPARQL/model sanity checks
- discover representative operator-relevant points from the model
- run the daytime smoke suite first (`openclaw/bench/e2e/README.md`)
- fix auth/model/BACnet issues found there before trusting the overnight 12-hour run
- only then move into recurring integrity sweeps and overnight review

110.6 Same-bench OpenClaw clone checklist

If OpenClaw is cloned onto another machine for the **same current test bench**, the new clone should read these first:

1. Root `README.md`
2. `openclaw/README.md`
3. OpenClaw context bootstrap
4. Open-FDD integrity sweep
5. BACnet-to-fault verification
6. `openclaw/bench/fake_bacnet_devices/README.md`
7. Monitor the fake fault schedule

And it should know these durable facts immediately:

- the fake devices intentionally inject faults on a **UTC** schedule
- the 180°F spike is expected only during the shared out-of-bounds window
- the correct way to judge that spike is to compare live BACnet RPC reads against `openclaw/bench/fake_bacnet_devices/fault_schedule.py`
- the integrity sweep should classify graph drift, auth drift, BACnet drift, and product behavior separately
- durable reasoning belongs in this repo, not only in local OpenClaw chat memory

110.7 Portability goal

A clone of this repo should make it easy for another engineer to answer:

- Check whether Open-FDD is healthy.
- Confirm BACnet scraping is working.
- Verify the building model is usable.
- Are faults being computed here?
- Are regressions visible here before they affect a real deployment?

110.8 Engineering principle

Keep environment-specific values configurable and keep the verification logic reusable.

In practice, deployment to another site usually looks like this:

- Open-FDD runs on some other server (often a Linux box on the OT LAN)
- tooling is cloned onto another machine
- the tooling is pointed at the target Open-FDD URL, auth, BACnet gateway, and rule/model context for that environment

The tooling should therefore be robust to:

- different LAN IP schemes
- different Open-FDD hosts
- different HVAC systems and point naming
- different site/equipment modeling shapes
- different SPARQL needs per deployment

The goal is portability with context, not a one-off lab setup.

111 OpenClaw subscription setup (Codex OAuth)

112 OpenClaw setup: ChatGPT subscription / Codex OAuth (not API key)

This note captures a setup path that worked for using **OpenClaw with the OpenAI Codex OAuth / ChatGPT subscription path** instead of the plain OpenAI API-key provider.

112.1 Why this matters

A normal ChatGPT subscription and a normal OpenAI API key are **not** the same thing.

That means OpenClaw can appear partly configured, but still fail with errors like:

```
401 Incorrect API key provided: sk-proj-...
```

if stale `openai/...` API-key settings are still present.

112.2 The key distinction

In OpenClaw, these are separate provider paths:

- **Subscription / OAuth path:** `openai-codex/...`
- **API-key path:** `openai/...`

For the subscription/OAuth setup, the model should look like:

```
openai-codex/gpt-5.4
```

Not:

```
openai/gpt-5.4
openai/gpt-5.1-codex
```

112.3 First commands to run

Use this basic command ladder first:

```
openclaw status
openclaw models status --probe
openclaw config get agents.defaults.model
openclaw config get agents.defaults.models
```

What you want to see is the default model set to something like:

```
{
  "primary": "openai-codex/gpt-5.4"
}
```

112.4 If OpenClaw is still trying to use the API-key provider

If you still see `openai/...` models or a 401 against `sk-proj-...`, the most likely cause is stale config.

112.5 What worked

112.5.1 1. Re-auth the OAuth provider

```
openclaw models auth login --provider openai-codex
```

That should complete the browser-based OAuth flow for the subscription path.

112.5.2 2. Make sure the default model points to `openai-codex/...`

```
openclaw models set openai-codex/gpt-5.4
```

112.5.3 3. Clean `openclaw.json`

The important cleanup is to remove all stale `openai/...` references and keep only `openai-codex/...`.

A working pattern looked like this:

```
{
  "auth": {
    "profiles": {
      "openai-codex:default": {
        "provider": "openai-codex",
        "mode": "oauth"
      }
    }
  },
  "agents": {
    "defaults": {
      "model": {
        "primary": "openai-codex/gpt-5.4"
      },
      "models": {
        "openai-codex/gpt-5.4": {}
      }
    },
    "list": [
      {
        "id": "main",
        "model": "openai-codex/gpt-5.4"
      }
    ]
  }
}
```

112.5.4 4. Remove stale API-key provider entries

Problematic leftovers included things like:

- `openai:default`
- `openai/gpt-5.1-codex`
- aliases like `GPT -> openai/gpt-5.1-codex`
- per-agent model overrides pinned to `openai/...`

Those kept the old API-key path alive even though OAuth was configured.

112.5.5 5. Check the agent auth file too

Also inspect:

```
notepad $HOME\.openclaw\agents\main\agent\auth-profiles.json
```

If that file still contains a stale `"openai:default"`, remove it and keep only `"openai-codex:default"`.

112.5.6 6. Restart the gateway

```
openclaw gateway stop
openclaw gateway
```

112.5.7 7. Verify again

```
openclaw models status --probe
```

What you want to see:

- default = `openai-codex/gpt-5.4`
- no `openai:default (api_key)`
- no `openai/gpt-5.1-codex` in configured models
- healthy `openai-codex:default (oauth)`

112.6 One more gotcha: old sessions

Even after config is corrected, an older session/thread can still behave like it is using the old provider.

So after restarting:

```
openclaw dashboard
```

start a **fresh** chat thread and ask:

```
Tell me exactly which provider and model you are using right now.
```

You want the answer to be:

```
openai-codex/gpt-5.4
```

112.7 Practical summary

The OAuth login can work **and** the system can still fail if stale `openai/...` API-key config remains in:

- `openclaw.json`
- `agents.list[].model`
- `agents.defaults.models`
- `auth.profiles`
- `agents/main/agent/auth-profiles.json`

The successful fix was:

1. keep only `openai-codex/...`
2. remove stale `openai/...` references
3. restart the gateway
4. test from a fresh session

112.8 Operational context to save for future setups

When OpenClaw is used with Open-FDD in the field, the agent should expect to ask for and record at least:

- the **Open-FDD base URL** the system is running at
- the **OFDD API key** / **Bearer token** if API auth is enabled
- whether the Open-FDD host is a **Linux box on the OT LAN** (this is the likely default deployment shape)

Those values should be treated as setup context, not tribal knowledge. They should be written into durable repo notes when appropriate, with secrets handled carefully.

112.9 Why this doc exists

This was learned through actual setup/debugging and should not live only in chat history. It is the kind of practical onboarding/context that future humans and agents will need again.

113 Monitor the fake fault schedule

114 Monitor the fake fault schedule

This page explains the right way to interpret spikes from the Open-FDD **fake BACnet bench** (`openclaw/bench/fake_bacnet_devices/README.md`).

114.1 Why this matters

On this bench, a value like **SA-T = 180°F** is not automatically a product bug.

The fake BACnet devices intentionally inject deterministic faults so Open-FDD can be verified end to end. That behavior lives in:

- `openclaw/bench/fake_bacnet_devices/fault_schedule.py`
- `openclaw/bench/fake_bacnet_devices/fake_ahu_faults.py`
- `openclaw/bench/fake_bacnet_devices/fake_vav_faults.py`

114.2 The shared UTC fault schedule

The fake AHU and fake VAV both follow the same repeating **UTC minute-of-hour** schedule:

Minute (UTC)	Mode	Expected behavior
0-9	normal	scheduled points should stay in their rough normal band

Minute (UTC)	Mode	Expected behavior
10-49	flatline	scheduled points should hold constant
50-54	bounds	scheduled points should jump to **180.0°F**
55-59	normal	scheduled points should return to normal variability

114.2.1 Scheduled points

The current fake devices intentionally schedule faults on these points:

- fake AHU: `SA-T`, `RA-T`, `MA-T`
- fake VAV: `ZoneTemp`

That means the 180°F spike should be treated as:

- **expected bench behavior** during UTC minutes **50-54**
- **unexpected drift or misalignment** outside that window

114.3 Recommended monitor

Use the repo helper:

```
python openclaw/bench/scripts/monitor_fake_fault_schedule.py
```

What it does:

- loads the same `fault_schedule.py` module used by the fake devices
- determines the current expected mode from UTC minute-of-hour
- reads the scheduled BACnet points directly through the DIY BACnet JSON-RPC API
- explains whether the observed values match the expected schedule

114.3.1 Example output

```
UTC time: 2026-03-24T12:52:10+00:00
Scheduled mode: bounds (minute 52 UTC)
Expected windows: normal 0-9, flatline 10-49, bounds 50-54, normal
55-59
- SA-T: PASS value=180.000 | matches scheduled bounds value 180.0
F
```

```
- RA-T: PASS value=180.000 | matches scheduled bounds value 180.0
F
- MA-T: PASS value=180.000 | matches scheduled bounds value 180.0
F
- ZoneTemp: PASS value=180.000 | matches scheduled bounds value
180.0 F
```

Configure the DIY BACnet gateway URL in the script or environment if it is not the default bench host.

114.4 How to reason about anomalies

114.4.1 Case 1: 180°F during UTC minutes 50-54

Treat this as **expected testbench fault injection**, not an unexpected product failure.

Next question:

- did Open-FDD surface the expected `bad_sensor_flag` in the corresponding fault window?

114.4.2 Case 2: 180°F outside UTC minutes 50-54

Treat this as a likely anomaly in one of:

- fake-device schedule alignment
- fake-device runtime state
- testbench orchestration / timebase drift
- Open-FDD ingest lag or stale-value behavior

114.4.3 Case 3: flatline window but value keeps changing

During UTC minutes 10-49, scheduled points should stop moving. If they do not, that points more toward:

- fake-device scheduling drift
- device restart / process interruption
- wrong host timezone / no UTC alignment on the Pi

114.5 How this connects to Open-FDD verification

The better verification chain is:

1. use this monitor to confirm the fake BACnet source state
2. use SPARQL to confirm the points are modeled and addressable
3. use Open-FDD fault APIs or frontend fault views to verify the expected fault flags
4. compare the observed fault windows against `fault_schedule.py`

That is much stronger than treating a single spike as mysterious.

114.6 Clone note

If OpenClaw is cloned onto another machine on the same bench, this page plus:

- `openclaw/bench/fake_bacnet_devices/README.md`
- BACnet-to-fault verification
- OpenClaw context bootstrap
- `openclaw/bench/scripts/monitor_fake_fault_schedule.py`

should be enough to explain why the 180°F spike exists and how to monitor it correctly.

115 Frontend Evaluation

116 React frontend evaluation

This document summarizes the quality and feature coverage of the Open-FDD React frontend (CRUD UI), its alignment with the API and the Grafana cookbook, and recommendations for WebSockets and the legacy static UI.

116.1 Quality and stack

- **Stack:** React 19, TypeScript, Vite, TanStack Query, React Router, Recharts, Tailwind CSS, shadcn-style UI (Card, Table, Badge, Skeleton). Fits a modern SPA setup.
- **Structure:** Clear separation of pages, hooks (data + WebSocket), contexts (site, theme), and shared UI. API types in `src/types/api.ts` match the OpenAPI schemas (Site, Equipment, Point, FaultState, FaultDefinition, etc.).
- **Auth:** Bearer token from `VITE_OFDD_API_KEY` is sent on all API requests and as `?token=` on the WebSocket; no auth UI (key is build-time env).
- **Missing pieces (added in this pass):** `src/lib/api.ts` (apiFetch with Bearer), `src/lib/csv.ts` (fetchCsv, parseLongCsv, pivotForChart for trending), and `src/lib/utils.ts` (cn, timeAgo, severityVariant) were not in the PR and are required for the app to build and run.

116.2 Feature coverage vs OpenAPI CRUD

API area	Frontend coverage
Sites	✅ List, select site; overview cards (equipment/points/faults per site). Create/update/delete not in UI (API-only or future).
Equipment	✅ List by site or all; table with point count and fault count.
Points	✅ List by site or all; table with external_id, equipment, brick_type, fdd_input, unit.
Faults	✅ Active faults list; fault definitions table + count (Grafana cookbook parity).
Trending	✅ Site-scoped point picker, date presets (24h, 7d, 30d, custom), line chart via POST /download/csv (long format) and pivot.
FDD status	✅ Last run in banner; run FDD is API-only (could add button).
Config, data-model, BACnet, jobs	❌ Not in UI; use Swagger or API.

So the frontend covers the main “operator” workflows: sites, equipment, points, active faults, fault definitions, and trending. Advanced flows (config, data-model export/import, BACnet discovery, jobs) stay in Swagger or the API.

116.3 Grafana cookbook parity

The Grafana SQL cookbook describes:

- **BACnet:** Variables (site, device, point), timeseries panel, BACnet + fault overlays.
  Frontend: Site selector + point picker + trending chart (same data via `/download/csv`). No separate “device” (bacnet_device_id) filter in the UI; points are chosen by site/equipment. Fault overlays on the chart could be a future enhancement.
- **Host stats:** Memory, load, swap, disk, containers.
  Frontend: No host-stats views; those remain in Grafana (or a future “System” page).
- **Weather:** Temp/RH/Dewpoint, wind, solar, cloud.
  Frontend: Covered indirectly: weather points (e.g. temp_f, rh_pct) appear in the point list and can be trended like any other point.
- **Faults:** Fault definitions table, fault definition count.
  Frontend: Implemented: “Fault definitions (N)” table (fault_id, name, category, severity, target equipment) and active faults table.

So the frontend is capable of the main cookbook workflows that are data-model/CRUD driven (sites, points, faults, trending). Grafana remains the place for raw SQL, host metrics, and pre-built dashboard JSON.

116.4 WebSockets: keep or remove?

- **Current use:** The React app uses `/ws/events` with token auth; on `fault.*`, `fdd.run`, `crud.*` it invalidates TanStack Query so lists and banners refresh. The HA integration optionally uses the same WebSocket for coordinator refresh.
- **Recommendation: keep WebSockets.** They give the React UI live updates without polling. If you later add MQTT for some integrations, you can keep WebSockets for the browser and use MQTT elsewhere; they are complementary (WS for real-time UI,

MQTT for pub/sub or constrained clients). No need to remove the WebSocket feature.

116.5 Legacy static UI and BACnet tree

- The old config UI in `open_fdd/platform/static` (data-model tree Site → Equipment → Points, BACnet test/whois/discovery) was removed so the stack uses a single frontend: the React app, served from its own container and via Caddy.
- **BACnet tree / BAS-style tree:** The static UI had a tree that mirrored the data model and building automation. To get that in the React app, add a "Data model" or "Tree" page that:
  - Fetches `/sites`, then for each site `/equipment?site_id=`, then for each equipment `/points?equipment_id=`,
  - Renders a collapsible tree (sites → equipment → points) with the same behavior (expand/collapse, optional right-click delete).
  - Optionally add a "BACnet" section: server URL, Test (server_hello), Who-Is range, Point discovery, using the same API endpoints the static UI used (`/bacnet/server_hello`, `/bacnet/whois_range`, `/bacnet/point_discovery`). That would bring the legacy config + BACnet flows into the React app without serving static files from FastAPI.

116.6 Caddy and security

- With the React frontend in its own container, **Caddy** is the single entry point (e.g. :8088): basic auth, then proxy to API (and `/ws/*`) and to the frontend (`/*`). Optional Grafana at `/grafana` when started with `--with-grafana`. See Security and Caddy.

116.7 Summary

- **Quality:** Solid React/TS/Vite setup; missing `lib` (api, csv, utils) was added so the app builds and runs.
- **Features:** Good coverage of sites, equipment, points, faults, definitions, and trending; config/data-model/BACnet/jobs stay in API/Swagger.

- **Grafana cookbook:** Parity on fault definitions and trending; host stats and raw SQL stay in Grafana; optional to add fault overlays and a data-model/BACnet tree page later.
- **WebSockets:** Keep; they support real-time UI and HA. MQTT can be added alongside.
- **Static UI:** Removed; React is the primary UI, served by Caddy. BACnet tree and discovery can be reimplemented as a React page if desired.

117 Plan Rules Crud And Hot Reload

118 Plan: Default Rules, Fix Phantom Definitions, Rules CRUD + Frontend (Hot Reload)

118.1 Goals

1. **Fix phantom "AHU Short Cycling"** — Definitions table and Faults matrix show only what exists in `rules_dir`; remove stale rows when a rule file is gone.
2. **Default rules** — Ship with only `sensor_bounds.yaml` and `sensor_flatline.yaml` in `stack/rules`; no extra files that create phantom definitions.
3. **Rules CRUD + frontend** — Users can upload, download, and delete rule YAML files via the UI; changes hot-reload (next FDD run picks them up).

118.2 1. Fix phantom definitions (backend)

Problem: `fault_definitions` is upserted from YAML on each FDD run but never pruned. If a rule file is removed (or never existed on this server), its row remains and the UI shows e.g. "AHU Short Cycling".

Change: In `open_fdd/platform/loop.py`, extend `_sync_fault_definitions_from_rules(rules)` so that after upserting from the current rule list it **prunes** definitions that no longer have a corresponding rule:

- Collect `fault_id` for every rule in `rules` (same logic: `r.get("flag") or f"{r.get('name', 'rule')}_flag"`).
- After the upsert loop, run: `DELETE FROM fault_definitions WHERE fault_id != ALL(%s)` (or equivalent) with the list of current fault_ids.
- No FK from `fault_results` / `fault_state` to `fault_definitions` (they store `fault_id` as text), so deleting definitions is safe; historical results keep their fault_id for display/analytics.

Result: Definitions list and Faults matrix only show faults that have a YAML file in `rules_dir`. One-time fix for existing DBs: next FDD run will prune "AHU Short Cycling" (and any other orphan).

118.3 2. Default rules (no code change)

- **Repo:** `stack/rules` already contains only `sensor_bounds.yaml` and `sensor_flatline.yaml`. Keep it that way; do not add an AHU Short Cycling YAML unless you want that rule.
- **Server:** If the Linux server has extra files in `stack/rules` that you don't want, remove them; after the prune logic above and one FDD run, definitions will match the remaining files.
- **New deployments:** Same two files as defaults; anything else is added by users via the new CRUD (or manually).

118.4 3. Backend: Rules CRUD (upload / delete)

Existing:

- `GET /rules` — list `.yaml` filenames and `rules_dir` (from config).
- `GET /rules/{filename}` — return file content (plain text). Use for **download**.
- `POST /rules/test-inject`, `DELETE /rules/test-inject/{filename}` — test-only (gated by `OFDD_ALLOW_TEST_RULES`); keep for automation.

Add (production, no env gate):

Method	Endpoint	Purpose
POST	`/rules` or `/rules/upload`	Upload a new or replace an existing rule file.
DELETE	`/rules/{filename}`	Delete a rule file from `rules_dir`.

Upload (POST):

- **Body:** Either `multipart/form-data` (file) or JSON `{ "filename": "my_rule.yaml", "content": "..." }`. JSON is easier for "copy/paste YAML" in the UI and matches test-inject.
- **Validation:**
  - `filename`: must end in `.yaml`, no `..`, `/`, `\`.
  - Resolve path under `_rules_dir_resolved()`; ensure path stays inside `rules_dir` (path traversal check).
  - Optional but recommended: parse YAML and check for minimal keys (`name` or `flag`) so invalid rules are rejected.
- **Write:** `path.write_text(content)` (same as test-inject). Create `rules_dir` if missing (optional; currently API returns 503 if not a dir).
- **Response:** 200 + `{ "filename": "...", "path": "..." }` or 201 for create.
- **Auth:** Same as rest of API (Bearer when `OFDD_API_KEY` set).

Delete (DELETE):

- **Path:** `DELETE /rules/{filename}` (filename in path; same safety as GET: `.yaml` only, no path traversal).
- **Behavior:** `path.unlink()`. Return 404 if file doesn't exist.
- **Optional:** After delete, call a small "sync definitions from rules_dir" helper that runs `load_rules_from_dir` + `_sync_fault_definitions_from_rules` so the definitions table (and matrix) update immediately instead of waiting for the next FDD run. If you skip this, the next FDD run will prune the definition anyway.

Optional helper: Expose `POST /rules/sync-definitions` that loads all YAML from `rules_dir` and runs `_sync_fault_definitions_from_rules` (no FDD run). Then upload/delete can call it so the UI updates without waiting for the next run. Low priority if FDD runs every few hours.

118.5 4. Frontend: Upload, Download, Delete

Current (Faults page – FDD rule files section):

- Lists filenames from `GET /rules`.
- Click filename → fetch `GET /rules/{filename}` and show content in a `<pre>`.
- Shows `rules_dir` from the list response.

Add:

1. **Download**
   - Per file: "Download" link that opens `GET /rules/{filename}` (e.g. in a new tab or via `fetch` + blob download). Already supported by existing endpoint.
2. **Upload**
   - "Upload rule" (or "Add rule file"):
     - Option A: File input → read file as text → POST JSON `{ filename: file.name, content }` (normalize filename to `.yaml` if needed).
     - Option B: Textarea "Paste YAML" + filename input → POST same JSON.
   - On success: invalidate `GET /rules` (and definitions query if you add sync), show success toast; optionally refetch rules list and select the new file.
3. **Delete**
   - Per file: "Delete" button (with confirmation: "Remove rule file X? Definition will be removed after next FDD run.").
   - Call `DELETE /rules/{filename}`; on success invalidate list and definitions, clear selection if the deleted file was selected.
4. **Copy from existing**
   - Optional: "Duplicate" that copies current file content into the upload form with a new name (e.g. `sensor_bounds_copy.yaml`) so users can duplicate and edit.

UX: Keep the existing "view content" flow; add Upload above the list and Download/Delete next to each filename. Use existing auth (e.g. `apiFetch` with API key).

118.6 5. Hot reload (no change)

- FDD loop already loads rules from `rules_dir` on **every** run (`load_rules_from_dir(rules_path)` in `run_fdd_loop`). No caching.

- So: user uploads a new YAML → file is on disk in `rules_dir` → next FDD run loads it and runs it; new rule appears in definitions (after sync) and in results. Same for delete: file gone → next run doesn't load it → prune step removes the definition.
- Optional `POST /rules/sync-definitions` only makes the definitions table (and matrix) update immediately; it doesn't change hot-reload behavior of the FDD engine.

118.7 6. Implementation order

Step	Task	Effort
1	Backend: Prune stale definitions in `_sync_fault_definitions_from_rules`	Small
2	Backend: `POST /rules` (upload) with filename + content, validation	Small
3	Backend: `DELETE /rules/{filename}`	Small
4	Optional: `POST /rules/sync-definitions` and call after upload/delete	Small
5	Frontend: Download link per file	Small
6	Frontend: Upload form (file or paste + filename), call POST /rules	Medium
7	Frontend: Delete button + confirm, call DELETE /rules/{filename}	Small
8	Tests: API tests for upload/delete; e2e or manual for UI	Small

Total: Backend ~half day; frontend ~half day; testing ~half day. Optional sync-definitions adds a bit more.

118.8 7. Security / validation

- **Path traversal:** Reject `..`, `/`, `\` in filename; resolve path and ensure `path.relative_to(base)` (already done in test-inject).
- **Overwrite:** Upload can overwrite existing files (same as test-inject). Optional: return 409 if file exists and require explicit overwrite flag.

- **YAML validation:** At least parse YAML; optionally require `name` or `flag` and reject if missing so broken rules don't get into the matrix until the next run fails.

118.9 8. Summary

- **Fix:** Prune `fault_definitions` in sync so only current rule files have rows → phantom "AHU Short Cycling" goes away after one FDD run.
- **Defaults:** Keep `stack/rules` with only `sensor_bounds.yaml` and `sensor_flatline.yaml`.
- **CRUD:** Add POST (upload) and DELETE for rule files; reuse GET for download; list already exists.
- **Frontend:** Add upload (file or paste), download link, delete with confirm; reuse existing list + view.
- **Hot reload:** Already works; no change. Optional sync-definitions endpoint improves UX by updating definitions immediately after upload/delete.

119 Security

120 Security and Caddy Bootstrap

This document describes how to protect Open-FDD endpoints with **Caddy** (reverse proxy), which services are **unencrypted by default**, and **hardening** best practices. The project defaults to **non-TLS**; TLS (including self-signed certificates) is optional.

120.1 Reverse proxy: current file vs future hardening

- **Canonical config in the repo:** `stack/caddy/Caddyfile` — checked-in Caddy config for a **single HTTP entry point** on port **80**: `handle /api*` (with `uri strip_prefix /api`), `/auth*`, `/ws*`, and `/ai*` are proxied to the API; `handle /*` serves the React frontend. This matches the frontend's default **`VITE_API_BASE=/api`** in Docker Compose: browser calls go to same-origin paths under `/api`, `/auth`, and `/ws`.

- **Status:** Use **`http://localhost`** (or `http://<lan-host>`) as the main URL when Caddy is up; use **http://localhost:8000** (API) and **http://localhost:5173** (frontend direct) when debugging without the proxy.
- **Future hardening:** Optional Caddy **basic auth**, **TLS**, rate limiting, and stricter perimeter defaults are documented below as **targets**; they are not all enabled in the committed file.

What is Caddy? A small web server in front of the API and the **React frontend**. The **committed** `stack/caddy/Caddyfile` is a lightweight starting point; this page also documents **target** patterns (basic auth, `X-Caddy-Auth`, full API routes) for when you extend or harden the proxy. Optional **Grafana** (when started with `--with-grafana`) can be exposed via Caddy (e.g. `/grafana`) once the proxy config is extended. Bootstrap starts the Caddy service with the compose file under `stack/`; see Quick bootstrap with Caddy.

120.2 Architecture: frontend, API, and Caddy

- **React frontend** runs in its own container (production build on **5173**; Caddy's `/*` routes there). **App login:** users sign in at **`/login`**; the API returns a short-lived **JWT access token** (browser **sessionStorage**) and an **HttpOnly** refresh cookie (`/auth/*`). REST uses **`Authorization: Bearer <access_token>`**; the WebSocket uses **`/ws/events?token=<access_token>`** (or the API-key token path in Frontend and API authentication).
- **API** (FastAPI) requires auth when **`OFDD_API_KEY`** and/or **app-user** config is set (`OFDD_APP_USER`, `OFDD_APP_USER_HASH`, **`OFDD_JWT_SECRET`** — all three together). Valid credentials are **`Bearer`** matching **`OFDD_API_KEY`** or a valid **access JWT**. Exempt paths include `/`, `/health`, `/docs`, `/redoc`, `/openapi.json`, `/app` (and `/app/*`), and **`/auth/*`**. Partial app-user configuration is rejected explicitly: **`/auth/login`** and related routes return **503** (`AUTH_CONFIG_ERROR` / `AUTH_NOT_CONFIGURED`), while the global API middleware returns **500** with `AUTH_CONFIG_ERROR` for other protected paths—never a silent fallback.
- **Machine clients** (scrapers, curl, **Open Claw** on a Windows bench) should rely on **`OFDD_API_KEY`** and **`Authorization: Bearer`**, not the browser cookie flow — see Open-Claw integration.
- **Caddy** can be extended with **basic auth** + **`X-Caddy-Auth`** / **`OFDD_CADDY_INTERNAL_SECRET`** (optional); see Caddyfile for protecting

the entire API. **Best practice:** keep the frontend in its own container; use a reverse proxy as the operator entry point.

120.3 Frontend and API authentication

{: #frontend-and-api-authentication }

Mode	Typical use	How it works
Dashboard login	Human operators via browser	`POST /auth/login` → access JWT in session + HttpOnly refresh cookie; `apiFetch` attaches `Bearer` access token; WebSocket uses access token in query string.
`OFDD_API_KEY`	Scrapers, scripts, agents, LAN test bench	`Authorization: Bearer <OFDD_API_KEY>` on REST; WebSocket may use `?token=<key>` when no JWT is in use.

Configure dashboard login with **`./scripts/bootstrap.sh --user NAME`** and a **secret password source** (`--password-file`, `--password-stdin`, **`OFDD_APP_PASSWORD`**, or interactive prompt — see `bootstrap.sh --help`). **`--no-auth`** removes **`OFDD_API_KEY`** and app-user keys from `stack/.env` as applicable.

Bootstrap guardrail: Starting the **full** stack or **`--mode model`** fails fast if **`OFDD_APP_USER`**, **`OFDD_APP_USER_HASH`**, and **`OFDD_JWT_SECRET`** are not all present in `stack/.env` (unless you pass **`--allow-no-ui-auth`** or **`--no-auth`**). **`--mode collector`** and **`--mode engine`** are exempt.

Examples (password not on the argv):

```
        # Set or rotate dashboard user + hash + JWT secret in stack/.env,
then start/restart stack as usual:
        printf '%s' 'YOUR_PASSWORD' | ./scripts/bootstrap.sh --user ops --
password-stdin

        # Same pattern on a full maintenance run (pull, rebuild, verify,
tests, fresh frontend node_modules):
        printf '%s' 'YOUR_PASSWORD' | ./scripts/bootstrap.sh --
```

```
maintenance --update --verify --force-rebuild --test --user ops --
password-stdin --frontend
```

After **`OFDD_JWT_SECRET`** or the app user changes, sign in again in the browser so tokens match the new secret; use **Sign out** (or `/logout`) if the UI still holds an old session.

`VITE_API_BASE`: In Docker Compose the frontend defaults to **`/api`** so same-origin requests work behind Caddy. Override with a full URL (e.g. `http://api-host:8000`) only if you deploy without path-based routing; see the README note.

Older docs referred to baking **`VITE_OFDD_API_KEY`** into the frontend at build time; the supported operator path is now **login** + **JWT** (or unauthenticated local dev with auth disabled). A static API key in the bundle is discouraged for production.

HTTPS behind a reverse proxy: The API sees plain HTTP from the proxy. Set **`OFDD_TRUST_FORWARDED_PROTO=true`** on the API container when the edge proxy terminates TLS and sends **`X-Forwarded-Proto: https`**, so refresh cookies get the **`Secure`** flag. Leave **`false`** (default) for HTTP-only lab stacks so cookies stay usable over HTTP.

120.4 Stack hardening (database, Caddy, secrets)

{: #stack-hardening-db-caddy-secrets }

Related tracking: **Stack security hardening** and **dashboard / API authentication**.

Architecture rule: Frontend → API → database. The React app must not talk to Postgres; only the API uses `OFDD_DB_DSN` on the Docker network (`db:5432`).

Area	Repo default / guidance
Postgres publish	`docker-compose.yml` binds the DB port to **`127.0.0.1:5432`** only — tools on the **host** can use `psql` locally; **remote machines cannot** reach the DB via that mapping. For stricter production, **remove** the `ports:` block under `db` so only containers on the compose network can connect (admin via `docker exec` or a throwaway `psql` container).

Area	Repo default / guidance
API / frontend host bind	Compose uses **`OFDD_API_HOST_BIND`** and **`OFDD_FRONTEND_HOST_BIND`** (default **`127.0.0.1`**) for published ports **8000** and **5173**. **`./scripts/bootstrap.sh --bacnet-address …`** without **`--caddy-self-signed`** sets both to **`0.0.0.0`** for a standard **HTTP lab** (LAN access to `/docs` and the React app). **`--caddy-self-signed`** forces **loopback** so the LAN should use **Caddy HTTPS**, not raw **:8000** / **:5173**.
Edge TLS	The committed Caddyfile serves **HTTP on :80** with security headers. For **HTTPS**, use a public DNS name and either Caddy automatic TLS or mounted certs; start with `stack/caddy/Caddyfile.https.example` and set **`OFDD_TRUST_FORWARDED_PROTO=true`** on the API. **Do not** send `Strict-Transport-Security` on plain HTTP; the example adds HSTS only on the HTTPS site block.
Caddy headers	The bundled **`stack/caddy/Caddyfile`** sets `X-Content-Type-Options`, `X-Frame-Options`, and `Referrer-Policy`. Optional: **CSP**, **rate limiting**, Caddy **basic auth** as a second layer — see Caddyfile for protecting the entire API.
Secrets	**`stack/.env`** is **gitignored** (and listed in `.gitignore`). Bootstrap creates or updates it; store **Argon2 hash**, **JWT secret**, and optional **`OFDD_API_KEY`** (machine-only — not for the browser UI) there. Never commit secrets. For Caddy **`X-Caddy-Auth`** flows, set **`OFDD_CADDY_INTERNAL_SECRET`** in `.env` (compose default is dev-only). Optional Grafana: **`GF_SECURITY_ADMIN_USER`** / **`GF_SECURITY_ADMIN_PASSWORD`** in `.env` instead of compose defaults.

WebSocket: Same auth model as HTTP (access JWT or API key); no DB access from browser WebSocket code.

120.5 Quick bootstrap with Caddy

1. **Start the stack** (from repo root):

```
./scripts/bootstrap.sh
```

This starts DB, API (8000), frontend (5173), BACnet server (8080), scrapers, FDD loop, and **Caddy** (see `stack/docker-compose.yml`). Caddy publishes **host port 80** → container port 80 using **`stack/caddy/Caddyfile`**. **Grafana is not started by default**; use `./scripts/bootstrap.sh --with-grafana` to include it (then use http://localhost:3000, or add a `/grafana` route when you extend the Caddyfile).

2. **Access through the committed Caddyfile** (**no basic auth** in the repo file today):
   - **URL:** `http://localhost` (port **80**)
   - **`/api*`** → API (prefix stripped), **`/auth*`** → API, **`/ws*`** → API, **`/ai*`** → API, **`/*`** → frontend (static build). From another machine on the LAN, use **`http://<server-ip>/`** the same way.

3. **Hardened entry point (future / manual):** To put the **entire** UI and API behind one login and optional TLS, follow Caddyfile for protecting the entire API below and test thoroughly. Example credentials like `openfdd` / `xyz` apply only after you add **basic_auth** to your Caddyfile—not in the default committed file.

4. **Without Caddy:** You can still use the services directly (no auth if `OFDD_API_KEY` is unset):
   - Frontend: http://localhost:5173
   - API: http://localhost:8000/docs
   - Grafana (if started): http://localhost:3000
     Do not expose these ports to untrusted networks without a reverse proxy and auth.

5. **Browser "Not secure" or "Your connection is not private":** Over plain HTTP (e.g. `http://localhost/` or `http://192.168.x.x/`), browsers show a warning. That is expected without TLS. Use **http://** (not https://) unless you have configured HTTPS in Caddy. Production should use TLS when you deploy with HTTPS.

120.6 Caddyfile for protecting the entire API

The **committed** `stack/caddy/Caddyfile` is intentionally **small** (see Reverse proxy: current file vs future hardening). The block below is an **example / target** for a **future hardened** setup: one entry point, optional **basic auth**, and most API routes (plus WebSocket) proxied with **`X-Caddy-Auth`**. It is **not** drop-in verified for every workflow yet—validate in your environment before relying on it for production. Use

the same **secret** in `header_up X-Caddy-Auth` and in the API container env **`OFDD_CADDY_INTERNAL_SECRET`** so the API trusts requests that passed Caddy's basic auth.

```
# Listen on port 80 (or use :8088 and map 8088:8088 in docker-
compose).
:80 {
    # Optional: basic auth (generate hash: docker run --rm caddy:2
caddy hash-password --plaintext 'your_password')
    # basic_auth /* {
    #   openfdd <bcrypt_hash>
    # }

    # API: REST, Swagger, WebSocket. Set
OFDD_CADDY_INTERNAL_SECRET in the API container to match the value
below.
    handle /ws/* {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /docs {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /redoc {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /openapi.json {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /health {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /config* {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /sites* {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /equipment* {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /points* {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
```

```
    }
    handle /faults* {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /data-model* {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /download* {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /analytics* {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /bacnet* {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /jobs* {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /capabilities* {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }
    handle /ai* {
        header_up X-Caddy-Auth "your_internal_secret"
        reverse_proxy api:8000
    }

    # Optional: Grafana when started with --with-grafana
    # handle /grafana/* {
    #   reverse_proxy grafana:3000
    # }

    # Frontend (React)
    handle /* {
        reverse_proxy frontend:5173
    }
}
```

Then restart Caddy and recreate the API so it has `OFDD_CADDY_INTERNAL_SECRET` set: `./scripts/bootstrap.sh --build caddy api` (from repo root). See Troubleshooting if the API returns 401 or the WebSocket fails.

120.7 Default password and how to change it

The **default committed** `stack/caddy/Caddyfile` does **not** include **basic auth** today. The following applies **after** you add a `basic_auth` block (e.g. using the example Caddyfile above):

- Use a **bcrypt hash** for each password (e.g. default example password **`xyz`** only if you paste the matching hash from docs/examples—do not use defaults in production).
- **Change the password** (required before production):

  1. Generate a new hash:

     ```
     docker run --rm caddy:2 caddy hash-password --plaintext 'your_secure_password'
     ```

  2. Open **`stack/caddy/Caddyfile`** and update the `basic_auth` user line(s) with the new hash(es).

  3. Restart Caddy: `docker compose -f stack/docker-compose.yml restart caddy` (from repo root).

- **Advanced / multiple users:** Edit the `basic_auth` block in the Caddyfile and add more lines (one per `username hash`). Use `caddy hash-password` for each password. For SSO or advanced auth, consider Caddy's JWT or forward-auth and an IdP instead of basic auth.

120.8 Services: encryption and exposure

Service	Port	Encrypted by default?	Notes
API	8000	No (HTTP)	Put behind Caddy; do not expose directly. Bearer auth when `OFDD_API_KEY` is set.
Frontend	5173	No (HTTP)	React app; use Caddy to serve and protect.

Service	Port	Encrypted by default?	Notes
			Sends Bearer token to API.
Grafana	3000	No (HTTP)	Optional (–with-grafana); use Caddy at /grafana or Grafana's own auth.
TimescaleDB	5432	No (plain PostgreSQL)	Compose maps **`127.0.0.1:5432`** to the container so the host can use `psql` locally; remote clients do not get a LAN-wide DB port. Remove `ports` under `db` for internal-only DB.
BACnet server	8080	No (HTTP API)	Host network; protect with firewall or Caddy on host.

- **Recommendation:** When hardened, expose only Caddy (compose defaults to **port 80**; use **443** or another port with TLS when ready) to the building/remote network. Do not expose 5432, 8000, 5173, 3000, or 8080 to the internet. Plan for Caddy **basic auth** and **TLS** as security hardening matures; keep the database and BACnet server behind the firewall. The React frontend runs in its own container and is served by Caddy (or another reverse proxy), not as static files from the FastAPI process.

120.9 Hardening best practices

1. **Passwords**
   - When you enable Caddy **basic auth**, do not use example defaults like `openfdd` / `xyz` in production. Change via `caddy hash-password` and update `stack/caddy/Caddyfile`.
   - Use strong, unique passwords (or a secrets manager) for Grafana (`GF_SECURITY_ADMIN_PASSWORD`), Postgres (`POSTGRES_PASSWORD`), and Caddy basic auth.
2. **Network**
   - Run TimescaleDB and internal services on a private Docker network; bind DB to localhost or internal IP only if needed.
   - Restrict firewall so only Caddy's port (and optionally SSH) is reachable from outside.
3. **Reverse proxy**
   - Use Caddy in front of API and Grafana so one place enforces auth and (optionally) TLS.
   - Basic auth over plain HTTP is weak; prefer HTTPS when possible (see below).
4. **Principle of least privilege**
   - Run containers as non-root where possible; avoid sharing host network unless required (e.g. BACnet).
   - Limit Grafana sign-up and use strong admin credentials.
5. **Updates and hygiene**
   - Keep Caddy, Grafana, TimescaleDB, and the OS updated. Rotate credentials after compromise or periodically.

120.10 Throttling and rate limiting

120.10.1 1. No API rate limiting by default

Open-FDD does **not** rate-limit incoming HTTP requests to the API. There is no built-in cap on how often clients can call the API. If you need to limit how often external clients (e.g. a busy integration or cloud poller) can call the API, add rate limiting at the reverse proxy (e.g. Caddy with a rate-limit module) or with rate-limit middleware in front of the app.

120.10.2 2. Outbound: OT/building network is paced

The application **does** throttle its own outbound traffic to the building and OT network. We do not continuously hammer BACnet or other building systems. Load is paced by configuration **and by which points you scrape**:

Component	Config	Effect
BACnet scraper	`bacnet_scrape_interval_min` (e.g. 5)	Polls points on a fixed interval (e.g. every 5 minutes), not in a burst.
BACnet scraper	**Data model (DB)**	The scraper polls only points that have `bacnet_device_id` and `object_identifier` in the **data model** (CRUD, import, or after **POST /bacnet/point_discovery_to_graph**). The default Docker stack does **not** use a BACnet CSV for scrape config. Throttling depends on **how many points** are defined and the poll interval. Best practice: scrape only the points needed for FDD and HVAC health. See BACnet overview.
FDD rule loop	`rule_interval_hours` (e.g. 3)	Runs fault detection on a schedule (e.g. every 3 hours); each run pulls data from the DB, not from BACnet.
Weather scraper	`open_meteo_interval_hours` (e.g. 24)	Fetches weather once per interval (e.g. daily).

So outbound load on the OT network is predictable and tunable. **Define only the points you need** in the data model (CRUD, import, or discovery → graph), then adjust intervals via the Config UI / data model or `OFDD_*` environment variables. See Configuration and BACnet overview.

120.10.3 3. Inbound: rate limiting at the reverse proxy (e.g. Caddy)

If you need to **throttle incoming traffic** to the API—for example to protect the API and OT network from aggressive polling, misconfigured integrators, or abuse—enforce rate limiting at the reverse proxy or with middleware. Open-FDD does not implement this itself; use Caddy (with a rate-limit module), nginx, or application middleware.

Using Caddy for rate limiting

Caddy can throttle requests per client (e.g. per IP or per authenticated user) when configured with a rate-limiting capability. The standard Caddy build does not include an HTTP rate-limit directive; you add it by building Caddy with a module (e.g. `xcaddy build --with github.com/mholt/caddy-ratelimit`) or by using a distribution that includes one. Once available, you configure a rate-limit zone in the Caddyfile (e.g. limit by `{remote_host}` or by a header), then apply it to the blocks that proxy to the API (and optionally Grafana). Typical choices are a cap per IP (e.g. 60 requests per minute) or per client identity when using auth. Exceeding the limit returns `429 Too Many Requests`; the client can retry after the window resets.

Example (conceptual; syntax depends on the module you use):

```
:8088 {
  rate_limit {
    zone api { key {remote_host} events 60 duration 1m }
  }
  handle /api/* {
    rate_limit api
    reverse_proxy api:8000
  }
  # ... basic_auth and other handle blocks
}
```

Adjust the zone (events per duration) to match your OT network and integration requirements. For official options and current syntax, see Caddy's module documentation and the documentation for the rate-limit module you build or install.

120.11 TLS (optional): self-signed certificates

The project **default is non-TLS**. If you want HTTPS in front of Caddy:

1. **Option A – Caddy automatic HTTPS (public hostname)**
   If you have a public hostname and port 80/443, change the Caddyfile address to your domain; Caddy will obtain and renew Let's Encrypt certs automatically.

2. **Option B – Self-signed certificate for Caddy**
   For internal or lab use:

   - Generate a self-signed cert (e.g. with `openssl`).
   - In the Caddyfile, use `tls /path/to/cert.pem /path/to/key.pem` and listen on `:443` (or another port).
   - Mount the cert and key into the Caddy container and point Caddyfile to those paths.
   - Browsers will show a certificate warning; accept or install the CA for your environment.

Example Caddyfile snippet for self-signed (conceptual):

```
https://0.0.0.0:8443 {
  tls /etc/caddy/cert.pem /etc/caddy/key.pem
  basic_auth /* { ... }
  # same handle/reverse_proxy blocks as :8088
}
```

Generate self-signed (host):

```
        openssl req -x509 -newkey rsa:4096 -keyout key.pem -out cert.pem -
days 365 -nodes -subj "/CN=openfdd.local"
```

Mount `cert.pem` and `key.pem` into the Caddy container and reference them in `tls`.

120.12 Troubleshooting

API returns 401 (Unauthorized) after you pass Basic auth in the browser

The API trusts requests that carry the internal header `X-Caddy-Auth` (set by Caddy after Basic auth). The API only checks this when the env var `OFDD_CADDY_INTERNAL_SECRET` is set in the API container and matches the

value Caddy sends. If you added or changed this after the stack was already running, the API container may not have the variable. Recreate the API so it picks up the env:

```
cd stack && docker compose up -d --force-recreate api
```

Or from the repo root: `./scripts/bootstrap.sh --build api`. Then hard-refresh or use an incognito window and sign in again with Basic auth.

WebSocket to `/ws/events` fails or closes immediately

When access is through Caddy with Basic auth, the browser may not send credentials on the WebSocket upgrade. The API also accepts the same `X-Caddy-Auth` header on the WebSocket endpoint, so if Caddy is forwarding the request (after Basic auth), the WebSocket should work. Ensure the API has been recreated with `OFDD_CADDY_INTERNAL_SECRET` as above. If the WebSocket still fails (e.g. due to browser or proxy behavior), the UI will keep working; live updates will resume when the connection succeeds.

120.13 Security scanning (Trivy)

{: #security-scanning-trivy }

During development, use **Trivy** to scan container images and the repo for vulnerabilities and misconfigurations. Run it after building images, when changing Dockerfiles or dependencies, and optionally in CI. See **How-to: Trivy security scanning** for install, when to run, image and filesystem scan commands, and keeping docs up to date.

120.14 Summary

- **Bootstrap:** Start the stack; the committed Caddyfile listens on **`http://localhost`** (port **80**) with **no** basic auth, and routes **`/api`**, **`/auth`**, **`/ws`**, and **`/ai`** to the API. Use **8000** / **5173** for direct debugging.
- **Auth:** Optional dashboard login (JWT + HttpOnly refresh cookie) and/or **`OFDD_API_KEY`** for machine clients; both can be configured together.
- **Passwords:** Change default by running `caddy hash-password` and updating the Caddyfile; use strong passwords for Grafana and Postgres.

- **Unencrypted by default:** API, Grafana, TimescaleDB, and BACnet API are plain HTTP/TCP; protect them with network isolation and Caddy (and optional TLS).
- **Hardening:** Strong passwords, expose only Caddy, keep DB and internal services off the public internet, keep software updated.
- **Throttling:** (1) No API rate limiting by default. (2) Outbound traffic to the OT/building network is paced (BACnet scrape, FDD loop, weather intervals). (3) To throttle incoming API traffic, use the reverse proxy (e.g. Caddy with a rate-limiting module) or middleware.
- **TLS:** Optional; default is non-TLS. Add self-signed or Let's Encrypt via Caddy when you need HTTPS.